

INSIDE THE MACHINE LEARNING INTERVIEW

151 Real Questions from FAANG and
How to Answer Them

PENG SHAO

INSIDE THE MACHINE LEARNING INTERVIEW

Copyright © 2023 by Peng Shao

All rights reserved. No part of this book may be reproduced in any form by any electronic or mechanical means, including information storage and retrieval systems, without express written permission from the author.

For permissions, contact peng.m.shao@gmail.com.

*Dedicated to my training data.
Without you I'd have no clue.*

Contents

1. Introduction	1
How to Use This Guide	2
Prerequisites	3
Interview Process	4
Technical Phone Screen	5
ML Fundamentals Interview	5
ML Coding Interview	6
ML Design Interview	7
Other Interview Sessions	7
Elements of a Strong Answer	8
2. ML Fundamentals	11
Practice Interview	12
Datasets	12
Features	12
Modeling	12
Evaluation	13
Interview Solutions	15
Datasets	15
Features	22
Modeling	27
Evaluation	53
3. ML Coding	63
What Matters Most to Interviewers	63
Implement an ML Algorithm	65
Practice Interview	65
Study Strategy	66
Probability & Statistics Questions	71
Practice Interview	71
Study Strategy	72
Hash Table & Distributed Programming	77
Practice Interview	77
Study Strategy	78
Other Coding Questions	82

Practice Interview	83
Study Strategy	84
4. ML System Design: Part 1	97
Design Framework	98
Practice Interview	101
Clarify the Problem	101
High-Level Design	102
Data Collection	102
Candidate Generation	102
Pre-Ranking	103
Feature Engineering	103
Modeling & Evaluation	104
Deployment & Serving	104
Interview Solutions	105
Clarify the Problem	105
High-Level Design	111
Data Collection	115
Candidate Generation	125
Pre-Ranking	140
Feature Engineering	146
Modeling & Evaluation	152
Deployment & Serving	168
5. ML System Design: Part 2	177
Retrieval and Ranking	177
More Design Questions	179
Practice Interview	181
High-Level Design	181
Data Collection	182
Feature Engineering	182
Modeling & Evaluation	182
Deployment & Serving	183
High-Level Design	184
Document Parsing	185
Sentiment Analysis	187
Topic Modeling	189
Text Summarization	193
Natural Language Understanding	195

Data Collection	197
Supervised	197
Unsupervised	199
Feature Engineering	202
Discriminative	202
Generative	205
Modeling & Evaluation	206
Information Extraction	206
Classification	211
Regression	214
Topic Modeling	216
Clustering	219
Generative Modeling	223
Deployment & Serving	227
A Note about LLMs	231
6. ML Infrastructure Design	233
Practice Interview	234
Offline	234
Online	234
Interview Solutions	236
Offline	236
Online	247
7. Advanced ML Questions	261
Delayed Labels	262
Learning Without Labels	264
Pricing Models	268
Appendix	273
Generative Models: Noisy Channel to LLM	273
References	285
About the Author	306

1. Introduction

Machine Learning (ML) has gained immense popularity in recent years and has become a crucial component in the development of modern technology. As a result, the demand for talented ML practitioners has increased, and mastering the interview process is critical to success in this rapidly growing field. For many interview candidates, these interviews can be intimidating and challenging, requiring a different set of skills than traditional coding and design interviews.

Consider, for example, the following question: *"In solving classification problem X, how might one identify the most useful features?"*

To answer this question, candidates can tap into their academic or industry experience. Those with academic backgrounds may recommend statistical techniques such as chi-squared tests or information gain. Conversely, individuals with industry experience may suggest embedded approaches such as perturbing feature values or performing ablation studies. In addition, candidates may emphasize the importance of selecting features that are not only informative but also efficient to collect offline within the constraints of existing systems, as well as fast and reliable to hydrate for online inference in a production environment.

As an interviewee, transitioning from problem-solving in coding and design interviews to active recall in ML questions can feel jarring. It doesn't help that ML practitioners are often expected to meet the Software Engineering bar while also being proficient in ML knowledge. But don't worry! ML interviews don't have to be a grueling experience. In fact, they can be quite the opposite.

By identifying the most crucial ML topics and adopting a solid studying strategy, candidates can increase their confidence in tackling even the toughest ML interview questions. And that's where this guide comes in. We'll walk you through the ML interview process step-by-step, presenting questions that you're likely to encounter and providing solutions that cover all the essential elements of an answer.

The concepts presented in this guide are broadly applicable, so even if you face variations of the questions, you'll have the knowledge and skills to succeed in your next ML interview. So let's get started!

How to Use This Guide

In this guide, we will take you step-by-step through the ML interview process, identifying common interview questions you will encounter in each session, and walking you through strategies for formulating strong answers.

We understand that the vast amount of academic and applied ML material can be overwhelming, which is why our aim is to present condensed and easily understandable answers. Whether you're a seasoned ML professional or just starting, this guide is designed to provide valuable insights to help you navigate your next interview with confidence.

To best utilize this guide, first complete the [Prerequisites](#), as a certain level of fluency in ML is assumed and the rest of the chapters won't make a lot of sense otherwise – and might even be frustrating to read.

If you are relatively new to ML interviewing, briefly skim over the practice interviews contained in each chapter to get a feel for the types of questions asked in ML interviews. Then, go back and review each interview session in the [Interview Process](#), identifying your gaps and strengths. Use this information to determine which chapters of this guide will be most helpful in improving your interview performance.

More seasoned ML practitioners may treat this guide as a helpful study companion. With knowledge and experience under your belt, challenge yourself by taking the practice interviews included in each session. This may reveal areas to refresh your skills or uncover unexpected blind spots: for example, *“What are some ways to overcome label imbalance in a dataset?”*. Jump to [ML Fundamentals](#) for a solution.

Prerequisites

Interview candidates should have an academic background in ML and/or Natural Language Processing (NLP). A degree in a relevant field certainly helps, but is not strictly necessary. Having taken some courses, such as Andrew Ng’s ML courses (Ng, n.d.), may be sufficient. Make sure you can explain terms like “overfitting”, “gradient descent”, and “hyperparameters” – they are used throughout this guide and ML interviews will assume you know them.

The prepared candidate will have also built several models from scratch, having gone through the ML practitioner workflow in full at least once prior to the interview. The ML practitioner workflow consists of the following steps:

1. Offline data exploration
2. Offline creation of features
3. Offline creation of dataset, or join features with existing dataset
4. Offline model training
5. Offline model evaluation
6. Offline model hyperparameter tuning
7. Online hydration of features
8. Online serving the model
9. Online periodic model refresh

Familiarity with the ML practitioner workflow is important to mastering the ML interview, since strong candidates can apply ML to real-world industry problems and demonstrate proficiency in the entire ML lifecycle. If you haven’t completed the ML practitioner workflow before, hop over to the Tensorflow documentation and walk through some of their tutorials in full:

- TFX: Getting Started tutorials (*TFX Tutorials / TensorFlow*, 2021) walk through building an offline pipeline for data extraction, feature engineering, model training, model evaluation, and model export.
- Use the Tensorflow Serving tutorial (*Train and Serve a TensorFlow Model With TensorFlow Serving / TFX*, 2023) to deploy your trained model to a hosted environment, and try making some classification or regression requests.

- Play around with Tensorflow Serving Configuration (*Tensorflow Serving Configuration* / TFX, 2021) to monitor your model and enable periodic model refresh.

If you've never held an industry position in ML you likely won't be familiar with all the steps of the above workflow, particularly steps 7-9. Don't worry, as those steps are mainly evaluated in the [ML Infrastructure](#) interview session, which is more directed towards Senior+ candidates.

Interview Process

ML practitioners are assessed for proficiency in the following areas:

- **ML Fundamentals**, which refers to the foundational concepts and principles of machine learning, including supervised and unsupervised learning, model selection, regularization, and model evaluation.
- **Software Engineering** skills, such as proficiency in technical problem solving, coding, data structures and algorithms, testing, and potentially knowledge of distributed systems.
- **ML System Design**, which involves the integration of different system components such as data collection, model architecture, feature engineering, training, inference, and monitoring into a cohesive whole.

ML interviewers seek candidates who do not have any apparent weaknesses in these competencies, while the most successful candidates exhibit exceptional strength in one or more areas. In an ML interview, competencies are evaluated in individual interview sessions, which we outline below.

As you take in each interview session, ask yourself the following questions:

1. Do I have a weakness here? If so, attempt to pinpoint the specific areas of weakness. Navigate to the pertinent chapter of this guide and read the related practice questions. For instance, if you lack confidence in model serving at scale, refer to the [ML Infrastructure Design – Online](#) questions and review the solutions carefully. Follow references as needed to ensure you have built a firm grasp of its contents.

2. Can I highlight a strength for this session? For example, take a question like, “*Can you describe how logistic regression works?*”. If your strength is in ML fundamentals, provide a clear, textbook-like definition and demonstrate understanding of the mathematics. If your strength is more on the applied side, direct the discussion towards the use cases and tradeoffs of logistic regression compared to other approaches.

Here are the technical interview sessions comprising an ML interview:

Technical Phone Screen

Consists of: a [ML coding](#) or simple ML modeling problem.

Evaluates:

- What is the general ML knowledge of this candidate?
- Can they apply ML to a practical product problem?
- Can they convert basic ML algorithms into code?

Sample questions:

- How does k-means work? Can you implement it?
- What is overfitting?
- Describe a system, at a high-level, that recommends X.

Insider Tip

It's not recommended to prepare specifically for the Technical Phone Screen, since it typically involves a relatively easier question from one of the other interview sessions. Therefore, it is more beneficial to allocate your preparation time towards other interview sessions.

ML Fundamentals Interview

Consists of: one or two questions probing the breadth and depth of your ML knowledge.

Evaluates:

- How familiar are they with essential ML concepts?

- Can they describe approaches in literature and/or industry to common ML problems?
- Are they able to compare and contrast several ML algorithms?
- Can they detail ML algorithms and/or techniques step-by-step?
- Can they deep dive into feature or modeling issues?

Sample questions: see [ML Fundamentals](#).

ML Coding Interview

Consists of: one or two coding questions. Questions may ask you to (a) implement a familiar ML algorithm, (b) write code solving a Probability or Statistics question, or (c) write code not related to ML, but with emphasis on data structures, algorithms, and/or distributed systems.

Evaluates:

- Can they reason about and use appropriate data structures and algorithms?
- Can they design proper APIs?
- Can they write clean code and pertinent tests?
- Can they assess the runtime complexity of their code?
- Can they adapt their solution - rewriting if necessary - as the problem evolves?
- How do they prove their results are correct?

ML algorithm questions evaluate:

- Can they translate algorithms into code form?
- Are they familiar with mathematical libraries (e.g, numpy)?
- Are they comfortable doing math with code?

Probability & Statistics questions evaluate:

- Can they solve basic probability problems?
- Can they translate algorithms into code form?

Senior+ candidates:

- Are they familiar with distributed systems?
- Can they solve the problem at scale with a distributed design?

Sample questions: see [ML Coding](#).

ML Design Interview

Consists of: (a) designing an end-to-end solution for an ML problem, encompassing the entire ML lifecycle, and/or (b) practical questions about the training and productionisation of ML models at scale.

Evaluates:

- Can they apply ML to a practical industry problem?
- Can they navigate the entire lifecycle of ML systems – from data collection, to modeling, to productionisation?

Senior+ candidates:

- Can they identify the right technical strategy and articulate their approach to complex industry problems?
- Can they spot issues, reason about tradeoffs, and propose appropriate solutions in each step of the ML lifecycle?

Sample questions: see [ML System Design](#) (Parts 1 and 2) for questions related to the ML lifecycle, and [ML Infrastructure Design](#) for questions related to ML at scale.

Other Interview Sessions

Behavioral Interviews:

Most ML interviews also assess a candidate's soft skills and/or leadership qualities. Since company cultures vary significantly, review documentation from the company you are applying in order to identify the most important character traits to consider.

Probability and Statistics:

In contrast to Data Science interviews, ML interviews do not typically assess Probability and Statistics skills as a separate competency. Nevertheless, basic comprehension is assumed since discussing certain topics without it may prove challenging. It is therefore important to grasp the foundational principles, such as:

- What is a confidence interval?

- Describe p-values.
- What's a uniform distribution? Normal? Bernoulli?
- How to compute variance?
- What's an independent variable and a dependent variable?
- Describe the difference between likelihood and probability.

Elements of a Strong Answer

Interviewers seek candidates who not only possess theoretical knowledge but can also handle practical challenges in an industry environment. Excellent candidates are skilled problem-solvers, communicators, and pragmatists. How does this reflect in the ML interview? Here are some elements of a strong answer:

- **Communication:** You can explain ML techniques in a way that is easy to understand. You are comfortable explaining concepts in different levels of detail, and can delve into mathematical specifics or cite research when needed.
- **Detail oriented:** You pay careful attention to every aspect of the ML practitioner workflow, such as data collection, feature engineering, modeling, and evaluation. You are able to recognize and address potential issues throughout the process, and can prioritize the most important factors. Your answers reflect not only a solid theoretical understanding, but also include practical examples of how to implement your ideas..
- **Limitations:** You clearly outline the limitations of a given technical approach. You are able to address the issues and propose solutions to each limitation. You effectively communicate the associated costs of implementing each workaround.
- **Tradeoffs:** You possess the ability to articulate the tradeoffs between different approaches. You identify specific scenarios where certain solutions are more suitable than others. You acknowledge that a combination of approaches is the most appropriate solution in some cases.

- **Strategic:** You are able to think ahead and anticipate potential issues and limitations with the current approach. You have the ability to come up with a plan for ongoing improvements and determine the most effective technical strategy going forward. This includes brainstorming ideas, gathering requirements, and establishing criteria for success.

Let's now take a closer look at each ML interview session, starting with ML Fundamentals, then ML Coding, and finally, ML Design. To help you prepare, we've included practice interviews for each session that covers a range of frequently asked questions. You are encouraged to take the practice interview, in whole or in part, to gauge your relative strengths and weaknesses. Make an effort to provide the best answers you can, incorporating elements of a strong answer in your response.

2. ML Fundamentals

This interview session tests the candidate's ML knowledge in various areas, assessing both their breadth and depth. Breadth refers to their ability to produce multiple valid solutions to a common ML problem, while depth refers to their ability to explain the details of one such approach.

While it may seem daunting that, as a candidate, you may be asked literally anything about ML in general, in practice there are often only a handful of subjects that make for good interview discussions. Interviewers prefer to ask questions that nearly everyone has either studied in school, or has encountered – regardless of their industry – to maximize signal and enable interviewers to substitute for one another. We will leverage this fact to limit our study to a smaller set of questions.

You probably won't be tested on the most recent advancements in the field, as interviewers prefer to concentrate on – you guessed it – the fundamentals of ML. For example, the interviewer is more likely to ask about logistic regression and boosted trees than about transformers, BERT, or attention mechanisms.

In this chapter, we will present the commonly asked ML Fundamentals questions that you'll want to know for your upcoming interview. We provide broad yet concise answers to these questions that cover the essential talking points. It is recommended that you supplement these responses with your own academic knowledge and/or industry experience, diving deep where you feel comfortable. We also provide tips on additional resources for you to gain more in-depth knowledge through further reading.

We include images and mathematical formulas where we believe they will be illustrative. While we have provided them to aid in your comprehension, you will not be expected to reproduce them during an interview.

Practice Interview

We have compiled a list of commonly asked questions for the ML Fundamentals interview session. It covers the essential areas of ML knowledge that you should be familiar with prior to your interview.

Challenge yourself by choosing a sample of questions from below. Write your answer before consulting the Interview Solutions section!

Datasets

- 2.1: How does one collect data and prepare a dataset for training?
- 2.2: What are some common problems in collecting data for training a model?
- 2.3: How to determine whether collected data is suitable for modeling?
- 2.4: What are several ways to handle label imbalance in a dataset?
- 2.5: How to deal when labels are missing?

Features

- 2.6: Can you describe different types of input features and their intuitions?
- 2.7: What is feature selection / importance?
- 2.8: How to perform feature selection?
- 2.9: How would you deal with missing feature values?

Modeling

- 2.10: What are some common modeling algorithms? List 3 and compare them.

- 2.11: Deep dive into one. Explain how the algorithm works.
- 2.12: What's the loss function? How is it optimized?
- 2.13: Is there a faster way to optimize the algorithm? What are the tradeoffs?
- 2.14: What are some ways to optimize gradient descent?
- 2.15: What's a reasonable procedure for hyperparameter tuning?
- 2.16: What is overfitting? How can you tell if your model is overfitting?
- 2.17: How to deal with overfitting?
- 2.18: Regularization – what is it? List 2 approaches and compare them.
- 2.19: What's the difference between linear and logistic regression?
- 2.20: Compare several different activation functions in neural networks.
- 2.21: Can you explain the difference between a decision tree, a random forest, and GBDTs?
- 2.22: What's the difference between boosting and bagging?
- 2.23: What are some unsupervised learning techniques? How do they work?
- 2.24: Describe k-means. How to initialize? What's the stopping criterion?
- 2.25: What are some semi-supervised learning techniques? How do they work?

Evaluation

- 2.26: What are common loss functions? Compare and contrast.
- 2.27: How to determine whether a loss function is convex?

2.28: How to evaluate a classifier model? List 3 metrics and describe when to use them.

2.29: How to evaluate the quality of a regression model?

2.30: Why not optimize a model directly on evaluation metrics?

2.31: If your model is performing poorly, how to debug and improve performance?

Interview Solutions

Datasets

Solution 2.1: How does one collect data and prepare a dataset for training?

Constructing a dataset is a crucial step in developing ML systems. The following are the main steps to follow:

1. **Collect the data:** Gather data you need to train the model. The data can come from existing streams generated by your system, such as user behavior, or from new data collections through conducting user research. Make sure to employ appropriate **sampling techniques**— such as random or stratified sampling – based on the task you want to perform. Evaluate the significance of streaming data into the model versus batching.
2. **Clean the data:** Check for missing or duplicate data, identify outliers, remove irrelevant information and correct any errors. If feature values are missing, data points may be discarded or imputed using an approach like mean-value imputation, or more sophisticated techniques.
3. **Label the data:** Assign labels to the data if the task is supervised learning. This may involve collecting user interaction data, or possibly hiring human annotators.
4. **Split the data:** Divide the data into train, validation, and test sets. The training set is used to train the model, the validation set is used to evaluate the performance of the model during training (e.g., for early stopping, hyperparameter tuning), while the test set is used to evaluate model performance after training. Training and validation sets can be folded into a single dataset if employing techniques like cross-validation. The test set provides an unbiased estimate of model performance, which can also be used to compare different modeling approaches.

5. **Preprocess the data:** Apply necessary preprocessing steps to the data, such as normalization, scaling or transforming the data into a format suitable for the model.
6. **Check for balance:** Imbalanced data occurs when one class is represented by a disproportionately large number of samples relative to the other classes. This can lead to problems during training because the model is likely to focus on the majority class and ignore the minority class. When working with an imbalanced dataset, first try training on the true distribution. If the model is able to learn and generalize well on the true distribution, the problem is likely not severe and no further action may be required.
7. **Shuffle the data:** Shuffle the data to reduce bias and ensure the model is not learning any order-based information, unless that is the objective of your training.

While the process of constructing a dataset for machine learning typically follows a sequence of steps, it is not always a strict linear process. There may be situations where you need to revisit certain steps or make modifications as you progress through the process. For instance, you might split the data after you have labeled it. Similarly, you may not need to balance the dataset before labeling it.

Solution 2.2: What are some common problems in collecting data for training a model?

While we outlined the general steps for data collection in the previous answer, we will delve into problems you may encounter along the way.

1. **Collecting the data:** Be aware of the biases that your data collection technique introduces, such as serving bias, and apply appropriate countermeasures, such as bandit approaches. Improper sampling may result in the dataset becoming narrow or homogeneous, leading to poor generalization and limited diversity in predictions.
2. **Cleaning the data:** Techniques used to clean the data, such as outlier and duplicate detection, and missing value imputation, may affect the

relevance of the dataset – particularly if the data are not missing at random – and may negatively impact the training of the model.

3. **Labeling the data:** Beware of noise that this step introduces, such as ambiguous annotation instructions, or misalignment between annotators. Consider employing techniques like weighted voting or annotation validation, to reduce annotation error. Read (Callison-Burch, 2009) for more on this approach.
4. **Splitting the data:** Ensure proper separation of data to prevent contamination between the training, validation, and test sets. For instance, consider splitting the data by dates where the training data includes all data up to a date and the test data includes data from the following date. See also (*Data Split Example / Machine Learning*, 2022).
5. **Preprocessing the data:** Improper normalization may affect the utility of certain features. For instance, normalization of counting features where the vast majority of occurrences are zero may be detrimental to model training.
6. **Checking for balance:** The distribution of classes in the data can affect the model's performance. Models may converge slowly, or be biased towards the majority class and have difficulty recognizing minority classes. Techniques like oversampling and undersampling can be used to address this, but issues may remain, such as poorly calibrated probabilities from model outputs. One solution is to upweight the downsampled class to improve calibration.

Insider Tip

Read more about handling imbalanced datasets below under “2.4: What are several ways to handle label imbalance in a dataset?”.

Solution 2.3: How to determine whether collected data is suitable for modeling?

The first step in constructing a dataset for training is to gather the raw data that you will use to train and validate your model. This data can come from a

variety of sources such as logs, databases, web scraping, or even user studies. When gathering the raw data, there are several important considerations:

- **Quantity** is important because it directly impacts the model's performance. Having a large dataset allows the model to learn more patterns and generalize better. In general, a dataset should have an order of magnitude more data than the number of parameters in the model. Simple models trained on large datasets often outperform complex models trained on small datasets. Google has had success training simple linear regression models on large datasets (*The Size and Quality of a Data Set / Machine Learning*, 2022).
- **Quality** is equally important because unreliable data can negatively impact the model's performance. There are several reasons why a dataset might be unreliable:
 - Missing values in either the features or labels. This can be due to a variety of factors, such as human error in data collection or corrupted data.

Insider Tip

Gain a better understanding of how to handle missing values by reviewing the solutions below for “2.5: How to deal when labels are missing?”.

- Rows in the dataset may be duplicated, which can skew the results and lead to overfitting.
- Feature values may be incorrect, which can occur if there is a disparity between the training feature distribution and the prediction feature distribution. Reuse code between the training pipeline and serving pipeline where possible (Zinkevich, 2023).
- Labels may be incorrect, which can happen if there are mislabeled data or errors in the data collection process.

Insider Tip

Read more about handling mislabeled data in [ML System Design: Part 2 – Data Collection](#) under “5.6: What are some challenges when it comes to collecting labels?”.

- Choosing an appropriate **sampling strategy** may be important for constructing high-quality datasets. The sampling strategy that you choose should align with the specific goals and metrics of your model. For instance, if your objective is to build a ranking model, then random sampling may not be sufficient. Instead, you may need to sample data in a way that reflects the ranking metrics that you want to evaluate, such as at the query level.

Similarly, if your dataset is extremely imbalanced, this may result in slow convergence or poor predictive performance.

Insider Tip

Learn more about addressing label imbalance by referring to the solution provided below for “2.4: What are several ways to handle label imbalance in a dataset?”.

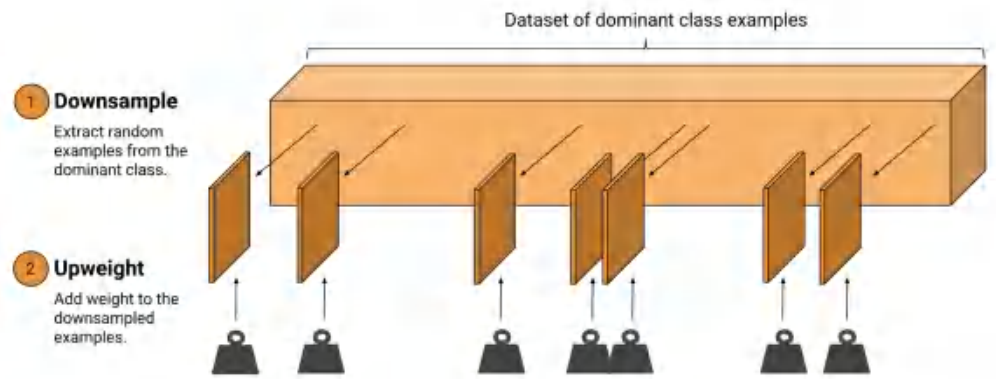
Solution 2.4: What are several ways to handle label imbalance in a dataset?

Handling label imbalance in a dataset is important to prevent a machine learning model from being biased towards the majority class. This is especially important when some classes have very limited representation. Here are several methods to handle label imbalance in datasets:

1. Resampling:

- **Oversampling the minority class:** Duplicating samples from the minority class until the classes are balanced.
- **Downsampling the majority class:** Removing samples from the majority class until the classes are balanced, which can lead to faster model convergence and reduced storage requirements.

Additionally, it may be beneficial to upweight the majority class after downsampling, as this can help to maintain model calibration throughout training.



Note. Downsample and upweight technique. Source: (*Imbalanced Data / Machine Learning*, 2022).

2. **Synthetic data generation:** Creating new synthetic samples of the minority class to balance the dataset. This can be done using techniques like Synthetic Minority Over-sampling Technique (SMOTE).
3. **Cost-sensitive learning:** Assigning different weights to samples from different classes to account for the imbalance in the loss function used to train the model.
4. **Ensemble methods:** Using multiple models with different sampling strategies or using an ensemble of models trained on different subsets of the data with boosting or bagging.
5. **Precision-oriented modeling:** Some models are better suited for evaluation metrics sensitive to imbalanced datasets, such as precision or precision@k. For example, in decision trees and their variants – like random forest, boosted trees, etc. – one can prune paths with high entropy or with sparse samples. The trees can also be broken down into decision lists. In certain Natural Language Understanding (NLU) scenarios like domain-constrained intent classification, even rule-based models can perform effectively. Alternatively, models that generate probability-based outputs, such as logistic regression, can be thresholded to attain the desired performance outcomes.

The method chosen to handle label imbalance will depend on the specific problem and dataset, and that different methods can be combined to get the best results. It's important to keep in mind that sometimes the best approach is to do nothing, particularly with enough data and a model with sufficiently large capacity.

Insider Tip

An informative survey on this work is (López et al., 2013).

Solution 2.5: How to deal when labels are missing?

Handling missing labels in a dataset can be achieved in several ways:

1. **Annotate the data:** This is the most direct approach for providing accurate labels, however, can be expensive and time consuming.
2. **Remove missing labels:** This approach is suitable for datasets with small amounts of missing data, but it may result in loss of information.
3. **Impute missing labels:** Replace missing values with estimated values based on the available data, such as mean or mode. For example, in a highly unbalanced dataset (e.g., web clicks) the majority class may be imputed. However, the imputed values may also introduce bias into the dataset, leading to a decrease in performance.
4. Predict missing labels with modeling (aka. “**Induction**”): This involves training a model (self-training), or multiple models (co-training), on the available data and then using them to predict missing labels. The downside is that the trained models may reinforce poor predictions.
5. Predict missing labels with separation (aka. “**Transduction**”): Instead of building a model only on labeled data, consider both labeled and unlabeled data to predict missing labels. This can be achieved several ways: (a) clustering algorithms with partial supervision, (b) graph-based methods like label propagation algorithms (LPA), and (c) manifold learning which encourages neighboring points in a lower-dimensional space to have similar predictions.

6. When the data is missing at the instance level but aggregated labels are available at the group level, several techniques can be employed to leverage group labels to build an instance level classifier. Read more in [Advanced ML Questions – Learning Without Labels](#).

Features

Solution 2.6: Can you describe different types of input features and their intuitions?

Broadly speaking, there are two fundamental types of features utilized in a ML model:

1. **Numerical features:** Features expressed by continuous or discrete numerical values, such as amounts, counts, rates, and durations.
 - Numeric features can be converted to categorical features. If the cardinality is small, numeric features may be treated as categorical features with identity values. Numeric features can also be bucketed (e.g., age) and discretized into categorical features.
2. **Categorical features:** Features expressed by categories, such as keywords, device, demographic, language, location, IDs, etc. Categorical features can be represented as one-hot, multi-hot, or weighted (e.g., frequencies).
 - Categorical features can be converted to numerical vectors, for instance, via embeddings. This is useful if the input vocabulary is large and there are latent relationships between values, such as with natural language text.
 - Categorical features can also be compressed into a smaller space by using a hashing function (called the “hashing trick”). This is useful for features where there can be many out-of-vocabulary values at inference time, such as IDs, as the model can incrementally fit new samples without retraining from scratch.

Many kinds of features can be reduced to one of the fundamental types above. Here are some examples:

1. **Binary features:** Features expressed by yes/no or true/false values, such as membership (“is-a”), traits (“has-a”), and thresholds (“greater-than”, “less-than”). These can be represented as either numerical or categorical features.
2. **Text features:** Text data are frequently transformed into embeddings, which are numerical vectors, or represented as one-hot or multi-hot categorical features.
3. **Image features:** Image data are preprocessed into numerical arrays, such as raw pixel values (e.g., RGB or HSV) or modified versions of the images created by applying various filters like thresholding, blurring, or equalization.
4. **Audio features:** Sounds are preprocessed into numerical representations such as frequency components of the audio signal, or Mel-Frequency Cepstral Coefficients (MFCCs).
5. **Sequential features:** These are features representing time-series data, such as stock prices. Sequential features can be numerical or categorical.
6. **Video features:** Video features can be represented as sequential image features. Another approach, optical flow features track pixel intensities across video frames.

Insider Tip

Read Google’s documentation on their built-in wide and deep algorithm (*Training Using the Built-In Wide and Deep Algorithm / AI Platform Training*, 2023) for practical guidance on when to use categorical or numerical features, vocabulary or hashing, and so on.

Solution 2.7: What is feature selection / importance?

Feature selection is the process of identifying a subset of the most relevant and useful features (predictive variables) from a larger set of features to be used in model building.

One goal of feature selection is to improve the model's predictive performance and reduce the complexity and overfitting of the model by removing irrelevant, redundant, or noisy features. Another goal is to reduce the footprint of the model and streamline the feature hydration process, thereby improving training and serving efficiency.

Feature importance refers to a numerical value or score that indicates the contribution of each feature to the prediction performance of the model. It is used to rank and select the most important features for a given task.

Solution 2.8: How to perform feature selection?

There are three approaches, at a high level, for performing feature selection:

- **Filter:** Features are selected based on statistical tests such as chi-squared test, ANOVA, mutual information, information gain, correlation coefficient, and others.
- **Intrinsic:** Features are selected as part of the modeling process. For instance, the Random Forest method (Palczewska et al., 2013) trains models on subsets of features where the expected fraction of samples contributed by a split is an estimate of the relative importance of the feature. An alternative approach is to incorporate L1 regularization during the modeling process, which reduces weights of less informative features to zero.
- **Wrapper:** A subset of features is selected based on their performance in a model. This usually involves training a model multiple times with different subsets of features. One method is to perturb individual feature values (such as randomly shuffling them) and evaluating the effect on the model's performance. Another method involves omitting features one by one, or in groups. The procedure can be performed bottom-up (Sequential Forward Selection) or top-down (Recursive Feature Elimination), or using non-greedy techniques such as simulated annealing.

When deciding on the best approach, there are certain tradeoffs to consider. For example, Filter methods are straightforward and inexpensive to compute, but they assume that statistical significance equates to improved predictive power. On the other hand, Wrapper methods are computationally intensive, but more directly assess the model's performance. To gain a better understanding of the relative importance of features, ML practitioners often employ a variety of approaches.

Approaches can also be combined, such as Filter and Wrapper. If several features are highly correlated, dropping an individual feature may not impact model performance even if it has a significant contribution to model performance, so highly correlated features should be dropped together.

Insider Tip

An excellent book on this topic is (Kuhn & Johnson, 2019).

Solution 2.9: How would you deal with missing feature values?

Some key questions to ask before imputing, or estimating, missing feature values are:

- Is the training data complete? It's possible that the data became corrupted in some way, and the problem could be resolved by improving the data collection process rather than through imputation.
- What kind of model are you training? Decision trees and ensembles, such as XGBoost, can handle missing values automatically without need for imputation (*Frequently Asked Questions — Xgboost 1.7.5 Documentation*, n.d.).
- Is the data missing at random? Many imputation techniques are only appropriate for scenarios where data are missing at random. Read more about this topic in (Puri et al., 2021). In some cases the data is intentionally missing, and missing values can be simply filled in: for instance, user interaction features (e.g., clicks) often omit values where the count is zero.

- How many values are missing? Visualize the severity of the missing values. For small amounts of missing data, values can be imputed. However, if a significant portion of data is missing at random, it may be better to delete the feature or fix the data collection process.
- What is the distribution for features with missing values? If the distribution is non-Gaussian, such as bimodal, using mean or median imputation will result in poor performance. Studies have also shown that single imputation techniques may underestimate the variance of the data (Alade et al., 2020).
- Are there highly correlated features? In the presence of collinearity with other features, missing features may be deleted entirely. Alternatively, correlation between features can make imputation easier if one or more highly correlated features is present, for instance using tree imputation. You can employ statistical tests such as pointwise mutual information or Pearson's R, or PCA to find underlying correlations.

With that, here are several ways to handle missing feature values in a dataset:

1. **Deletion:** Simply remove the records or instances that contain missing values. This method is useful when there is very little missing data, but it can lead to loss of information if there is a large amount of missing data. Another option is to delete the feature(s) itself.
2. **Mean/Median/Mode:** Replace missing values with the mean, median or mode of the feature. This method is simple and fast but often underestimates variance of the data (Zhang, 2016), produces bad estimates for non-Gaussian distributions (e.g., bimodal distributions), and introduces bias if the data is not missing at random. That said, it is often the de facto method used in ML toolsets, such as BigQuery ML (*Automatic Feature Preprocessing* / BigQuery, 2023).
3. **Interpolation:** In time-series data, missing feature values can be interpolated using adjacent values. For instance, stock prices are not available during weekends. Linear interpolation averages the values from Friday and Monday. An alternative approach is to use the value on Friday (technique called LOCF) or the value on Monday (called NOCB).

4. **kNN (aka. “Hot Deck”)**: Replace missing values with values from the k nearest neighbors. This may be resource intensive as kNN requires storing the entire dataset in memory.
5. **Regression**: Train a regression model to predict the missing values based on the other values of other features in the dataset. This approach can also be combined with kNN, for example, the Euclidean distance can be used as a weight. One downside is that regression analysis may underfit the distribution.
6. **Trees**: Decision trees and ensembles are a method of training non-linear models for imputation. The idea is to use trees to identify features that are highly correlated with the missing value and use those features to predict the missing value. Trees are fit on the available features and the prediction is the imputed value.
7. **Multiple imputation**: Multiple imputation involves generating several complete datasets by randomly filling in missing values from an estimated distribution. Each dataset is then used to train a separate model. During inference, the predictions from each model are pooled (e.g., by taking a weighted average) to produce a final estimate. One popular technique is Multiple Imputation with Chained Equations (MICE) (Errickson, n.d.).

Modeling

Solution 2.10: What are some common modeling algorithms? List 3 and compare them.

Three common modeling algorithms seen in industry are (1) logistic regression, (2) gradient boosted trees, and (3) deep neural networks.

Logistic regression:

Popular classification algorithm that models the probability of an event occurring based on one or more input variables (the “features”). In binary logistic regression, the features are combined linearly and then transformed using the logistic function, also called the sigmoid function, to model the

probability of the event. In multinomial logistic regression (also called softmax regression), the softmax function is used in place of the logistic function.

The logistic function is defined as:

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$

where x is the input vector, θ is the weight vector, and $h_{\theta}(x)$ is the predicted probability that the dependent variable takes the value 1 given x .

The weights in the logistic function are estimated from the data by maximizing the likelihood of observing the dependent variable given the inputs and weights. This is equivalent to minimizing the negative log-likelihood, which is expressed as the Log Loss, or Cross-Entropy loss.

Log Loss can be minimized using various optimization algorithms such as gradient descent, stochastic gradient descent, or quasi-Newton methods (e.g., BFGS) (Minka, 2003). During training, the model is repeatedly presented with training examples, and the model weights are updated in the direction of the negative gradient of the loss function with respect to the weights. This process is repeated until the loss converges to a minimum, according to a tolerance threshold.

Logistic regression can be regularized by adding a regularization term to the loss function. The most commonly used regularization techniques are L1 and L2, which add a penalty term to the loss function that encourages small weights.

Pros:

- Simple and efficient algorithm that can be trained quickly even on large datasets.
- Outputs probability estimates, which can be useful in certain applications such as risk assessment or fraud detection.
- Can be easily interpreted, and the weight vector can be used to identify the most important features for the classification task.

Cons:

- Assumes a linear relationship between the input variables and the target variable, which may not always hold true in practice.
- Prone to overfitting if the number of features is much larger than the number of training examples.

Gradient boosted trees (XGBoost is described below):

XGBoost (eXtreme Gradient Boosting) is an ensemble learning algorithm. It uses a gradient boosting framework that trains decision trees iteratively, adding new trees to the model to correct errors made by previous trees.

Commonly used loss functions are:

- Regression: Mean Squared Error (MSE) and Mean Absolute Error (MAE).
- Binary classification tasks: Log Loss (for binary) and Softmax Loss (for multi-class).

In traditional boosting, trees are added by reweighting data samples based on prediction errors of the current ensemble. In contrast, gradient boosting adds new trees by fitting the negative gradient of the loss function with respect to the predictions, also known as the pseudo-residuals.

XGBoost, rather than explicitly fitting the pseudo-residuals, aims to minimize the following objective at each iteration:

$$\sum_{i=1}^n l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \omega(f_t)$$

where $\hat{y}_i^{(t-1)}$ are the predictions of the current ensemble, f_t is the new tree, and ω is a regularizer term. XGBoost applies a Taylor expansion to the objective, reformulating it as:

$$\sum_{i=1}^n \left[l(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \omega(f_t)$$

where g_i and h_i are first- and second-order gradients of the loss function. The $l(y_i, \hat{y}_i^{(t-1)})$ is usually dropped since it's constant for a given iteration. New

trees are grown in a greedy fashion, enumerating over features at each node, and traversing possible splits¹. The split point is where the sum of loss from the child nodes most reduces the loss in the parent node (the “gain”), and is a function of the sums of first- and second-order gradients on either side.

One of the key innovations of XGBoost is the use of the second-order approximation of the loss function, which improves the accuracy of the gradient descent procedure. This is achieved by adding a second-order Taylor expansion of the loss function to the first-order gradient (Chen & Guestrin, 2016), resulting in more optimal estimates for the new tree.

Another feature of XGBoost is its ability to handle missing values, by assigning each missing value to a default direction in the decision tree, and learning the best direction during training (*Frequently Asked Questions — Xgboost 1.7.5 Documentation*, n.d.). This allows XGBoost to handle missing values without the need for imputation.

XGBoost provides several regularization techniques, such as L1 and L2 (known as alpha and lambda), gamma (the minimum loss reduction to create a tree-split), and max tree depth. Other regularization parameters include learning rate and data subsampling.

Pros:

- State-of-art performance on many regression and classification tasks.
- Robust to missing values and does not require scaling and normalization.
- Performs well on datasets of varying sizes, from small to large.
- Numerous hyperparameters, along with flexible options for parallelization and regularization, are available.

Cons:

- Doesn’t support categorical features natively, though there are ways to handle them (*Categorical Data — Xgboost 1.7.5 Documentation*, n.d.).
- Sensitive to outliers as new trees are fit to fix errors made by previous trees.
- Somewhat siloed from other modeling algorithms in industry settings. Integration of training and serving alongside other modeling algorithms can be a hassle.

¹ For larger datasets, XGBoost partitions features into quantiles instead of scanning all values.

Deep neural networks (DNN):

Deep neural networks (DNNs) are a class of modeling algorithms that consists of layers of interconnected neurons (also known as “units”). DNNs can learn complex and non-linear relationships between inputs and outputs, making them well-suited for a wide range of tasks.

In a DNN, the units in each layer are connected to the units in the previous and next layers. The inputs to the network are fed into the first layer, and the outputs of the last layer are the predictions of the model. Each unit applies an activation function to the weighted sum of its inputs, and passes the result to the units in the next layer. DNNs can have many hidden layers, which allows them to learn increasingly complex representations of the input data. DNNs with a large number of layers are often referred to as deep learning models.

A variety of loss functions are utilized in DNNs (see [ML Fundamentals – Evaluation](#) for a list), popular ones being MSE, Cross-Entropy, and Log-Cosh. DNNs are trained using a process called backpropagation, which involves iteratively updating the weights of the neurons to minimize the difference between the predicted and true outputs. Backpropagation works by computing the gradient of the loss function with respect to the weights of the neurons, and using this gradient to update the weights using a variant of gradient descent, such as stochastic gradient descent (SGD).

DNNs can be regularized to prevent overfitting, by adding L1 or L2 regularization, by using dropout, or with early stopping. Max-norm and batch normalization are other popular techniques.

Pros:

- Learns complex and non-linear relationships between inputs and outputs, and along with XGBoost, achieves state-of-the-art performance on many tasks.
- Scales well to high-dimensional and large-scale datasets, making them well-suited for big data applications.
- Versatility: models come in all shapes and sizes, feature interactions can be modeled explicitly (e.g., with crossing), support for a wide range of features and label types, pre-trained models can be easily transferred to new tasks, and so on.

Cons:

- Computationally expensive to train complex models.
- Requires large amounts of data to train effectively.
- Complex models are prone to overfitting on small to medium datasets.
- Can be difficult to interpret and understand.

In industry, ML practitioners often transition from one modeling algorithm to another, and they may also combine multiple modeling algorithms as well. For example, one might start model development with logistic regression and ramp up to DNNs, as appropriate. XGBoost models are also used in combination with logistic regression (He et al., 2014) and DNNs (Zhu et al., 2017).

Solution 2.11: Deep dive into one. Explain how the algorithm works.

As we described above, logistic regression models the probability of an event occurring based on one or more input variables. In **logistic regression**, the input variables are combined linearly, and transformed using the logistic function (also called the sigmoid function) to model a probability of the event, and then mapped to binary outcomes, such as 0 or 1. The logistic function is defined as:

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$

where x is the input vector, θ is the weight vector, and $h_{\theta}(x)$ is the predicted probability that the dependent variable takes the value 1 given x .

The logistic regression model is trained using a **maximum likelihood estimation** approach, which involves finding the values of the weight vector that maximize the likelihood of the observed data. The likelihood function is a product of the probabilities of each observation, given the input variables and weights. Maximizing the likelihood is equivalent to minimizing the negative log-likelihood, which is known as the Log Loss (or Cross-Entropy) function, defined as:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m y_i \log(h_{\theta}(x_i)) + (1 - y_i) \log(1 - h_{\theta}(x_i))$$

The goal of the optimization process is to minimize the loss function by finding the optimal values of the weight vector θ . This can be achieved using various optimization algorithms such as gradient descent, stochastic gradient descent, or quasi-Newton methods (e.g., BFGS) (Minka, 2003). During training, the model is repeatedly presented with training examples, and parameters of the weight vector are updated in the direction of the negative gradient of the loss function. This process is repeated until the loss function converges to a minimum.

Logistic regression can be regularized to prevent overfitting by adding a penalty term to the loss function that encourages small weights. The most commonly used regularization techniques are **L1 regularization**, which encourages sparsity by adding the absolute values of the parameters to the penalty term, and **L2 regularization**, which adds the square of the parameters to the penalty term.

Multinomial logistic regression is the generalization of logistic regression to the case of multiple classes. In multinomial logistic regression, the probabilities are estimated using a softmax function, which ensures that the probabilities for all classes add up to 1, defined as:

$$h_{\theta}(x) = \frac{e^{\theta^{(k)T}x}}{\sum_{j=1}^K e^{\theta^{(j)T}x}}$$

The loss function for multinomial logistic regression is defined as:

$$J(\theta) = - \sum_{i=1}^m \sum_{j=1}^K 1\{y_i = k\} \log \frac{e^{\theta^{(k)T}x}}{\sum_{j=1}^K e^{\theta^{(j)T}x}}$$

Solution 2.12: What's the loss function? How is it optimized?

Logistic regression is trained using a maximum likelihood estimation approach, which involves finding values of model coefficients (also known as weights) that maximize the likelihood of the observed data given the input variables and weights. The likelihood function is a product of the probabilities of each observation.

Maximizing the likelihood is equivalent to **minimizing the negative log-likelihood**, which is known as the Log Loss (or Cross-Entropy) function, defined as:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m y_i \log(h_{\theta}(x_i)) + (1 - y_i) \log(1 - h_{\theta}(x_i))$$

where y_i is the true label (0 or 1) and h_{θ} is the predicted probability for the positive class. The multinomial variant of this function is given in the previous answer.

Log Loss in logistic regression can be optimized using **gradient-based optimization** algorithms such as stochastic gradient descent (SGD) or its variants (e.g. mini-batch gradient descent).

The optimization algorithm updates the weights in the direction of the negative gradient of the loss with respect to the weights. The gradient is calculated by taking the derivative of the loss with respect to each weight, which tells us the rate of change of the loss with respect to a small change in that weight. By updating the weights in the direction of the negative gradient, the loss is reduced and the model is trained to make more accurate predictions.

The derivative of the loss function is straightforward to calculate. Since logistic regression uses the logistic, or sigmoid, function the first step of is to find the derivative of the sigmoid function, which is (derivation at (Tanner, 2020)):

$$h_{\theta}(x)(1 - h_{\theta}(x))$$

Plugging this in leads to the derivative of the loss function:

$$\frac{1}{m} \sum_{i=1}^m (h_{\theta}(x_i) - y_i)x_i$$

In each iteration of the optimization algorithm, the gradient of the loss is calculated based on a single random sample (for SGD) or a small batch of samples (in mini-batch gradient descent). This is achieved by computing the

dot product between the residuals $h_{\theta}(x_i) - y_i$ and the input variables x_i , and averaging over samples in the batch. Each weight is updated by a product of its gradient and a learning rate, which determines the step size for the update.

The optimization algorithm continues to update the weights until the loss converges to a minimum according to a tolerance threshold, or a maximum number of iterations is reached. The final weights provide a trained logistic regression model that can be used to make predictions on new data.

Solution 2.13: Is there a faster way to optimize the algorithm? What are the tradeoffs?

Second-order optimization methods are optimization algorithms that use information from the second derivative (also known as the Hessian matrix) of the objective function to optimize it. These methods are more efficient than first-order optimization methods (such as gradient descent), as they have a better estimate of the curvature of the objective function, leading to faster convergence and better solutions. Some examples of second-order optimization methods are **Newton's method**, **BFGS** and **L-BFGS** (Tan & Lim, 2019).

The main downsides of second-order optimization methods compared to first-order methods are:

1. **Computational cost:** Second-order methods require computation of the Hessian matrix, which can be expensive and time-consuming for large models, though quasi-Newton methods aim to reduce the computational burden.
2. **Memory requirements:** Storing the Hessian matrix can be memory-intensive for high-dimensional datasets, although there are approaches to approximate the matrix (e.g., L-BGFS).
3. **Saddle points:** Some second-order methods such as Newton's method are attracted to saddle points, though this can be mitigated with regularization.

For these reasons, first-order methods are often the preferred choice for large-scale problems, especially when there are computational or memory limitations.

Solution 2.14: What are some ways to optimize gradient descent?

Recall that gradient descent takes the form:

$$\theta = \theta - \eta \nabla_{\theta} J(\theta)$$

where η is the learning rate. We have already discussed techniques like stochastic gradient descent (SGD) which performs a weight update after each sample. However, SGD suffers from high variance in gradient estimation, due to the small sample size, which may lead to overshooting.

Mini-batch gradient descent reduces the variance of the gradient estimation by computing weight update on a small batch of samples (usually between 32 and 256), resulting in smoother convergence.

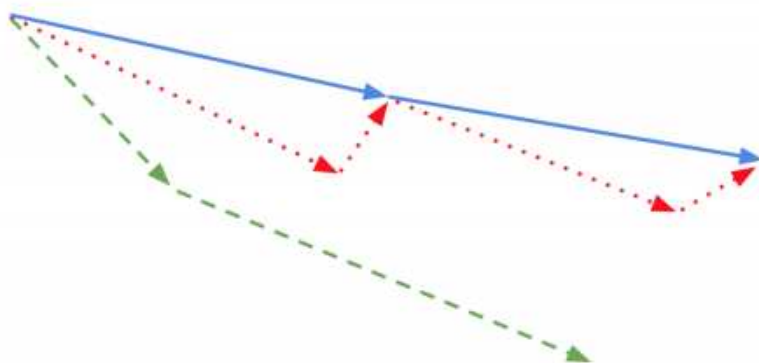
However, there are still drawbacks, particularly with the learning rate.

- Learning rates that are high lead to large fluctuations in weight updates or even to divergence.
- Learning rates that are low result in slow convergence.
- Learning rates that are fixed or adjusted on a schedule (e.g., annealing) fail to adapt to the model, dataset, or contour of the loss function.
- Learning rates applied uniformly to all weights don't account for feature sparsity.

Here are a few gradient descent optimization algorithms that address these issues:

- **Momentum:** Adds a fraction of the weight update from the previous time step to the current weight update. Can be viewed as adapting to the slope of the loss function. Encourages faster convergence but may also wander in the wrong direction.
- **Nesterov Momentum:** Version of Momentum that computes the gradient after applying the momentum update to the weights. It can be

seen as a correction to the momentum, to increase the accuracy of weight updates without sacrificing the speed.



Note. Momentum (dashed) combines the gradient with the previous update. Nesterov Momentum first approximates where momentum will take the parameters and computes the gradient with respect to that position (dotted), resulting in the update (solid).

- **AdaGrad:** Adapts weight updates according to feature sparsity. Features that activate frequently perform smaller weight updates than features that are infrequent. This is achieved by accumulating squared gradients per feature in the denominator of the learning rate, so learning rate decreases the more often features activate. The downside is that the learning rate may be reduced too much before the algorithm converges.
- **AdaDelta (and RMSProp):** Addresses the diminishing learning rates of AdaGrad by allowing the learning rates to speed up again. This is achieved by keeping an exponentially decaying moving average of squared gradients, rather than the entire history of the squared gradient. More memory efficient than AdaGrad. AdaDelta and RMSProp are commonly used by ML practitioners in the industry settings.
- **Adam:** Can be seen as adding momentum to Adadelta/RMSProp. In addition to accumulating an exponentially decaying average of squared gradients in the denominator, Adam also stores an exponentially

decaying average of gradients in the numerator, which acts as the momentum term.

In some cases the application of Momentum and Adam should be avoided, such as DNNs that direct training to different regions of the model architecture (e.g., via masking) conditioned on the input variables. Applying Momentum in these scenarios results in unintended weight updates.

Insider Tip

Sebastian Ruder has compiled a detailed breakdown of common gradient descent optimization algorithms in his blog (Ruder, 2016).

Solution 2.15: What's a reasonable procedure for hyperparameter tuning?

There are various methods for hyperparameter tuning, such as grid search, random search, Bayesian optimization, and genetic algorithms.

In **grid search**, a predefined set of hyperparameters are evaluated exhaustively, while in random search, hyperparameters are selected randomly from a predefined search space. These approaches can be inefficient, especially for models with a large number of hyperparameters or ones that take a long time to train.

Bayesian optimization leverages a probabilistic model to approximate the relationship between hyperparameters and the model performance, using an acquisition function that guides the search by balancing exploration (sampling new hyperparameters) and exploitation (evaluating promising hyperparameters).

In a **genetic algorithm**, models represent combinations of hyperparameters (“genes”). The process involves selecting the fittest models from each generation in an iterative manner. The surviving models undergo modification (“mutation”) or recombination (“crossover”) of their hyperparameters.

Bayesian optimization and genetic algorithms can be effective in finding hyperparameter values for complex models and large datasets.

For **logistic regression**, commonly tuned hyperparameters include:

- Optimization algorithm (aka. “solver”)
- Learning rate
- Regularization (aka. “penalty”): L1 and L2

Logistic regression also uses a “C” hyperparameter which controls the amount of regularization applied.

Here are some additional hyperparameters when working with DNNs:

- Number of hidden layers (aka. “depth”)
- Neurons (units) per layer
- Embedding size e.g., for text and categorical input features
- Activation functions
- Batch size
- Dropout

XGBoost employs a different set of hyperparameters, the chief one being the number of estimators (trees), or the number of boosting rounds.

The majority of other hyperparameters in XGBoost serve the purpose of regularization:

- Max depth of each tree
- Subsample: Fraction of data used to train on
- Colsample: Fraction of features available to train on
- Learning rate
- L1 (aka. “alpha”) and L2 (aka. “lambda”)
- Gamma: Minimum loss reduction to create a tree-split

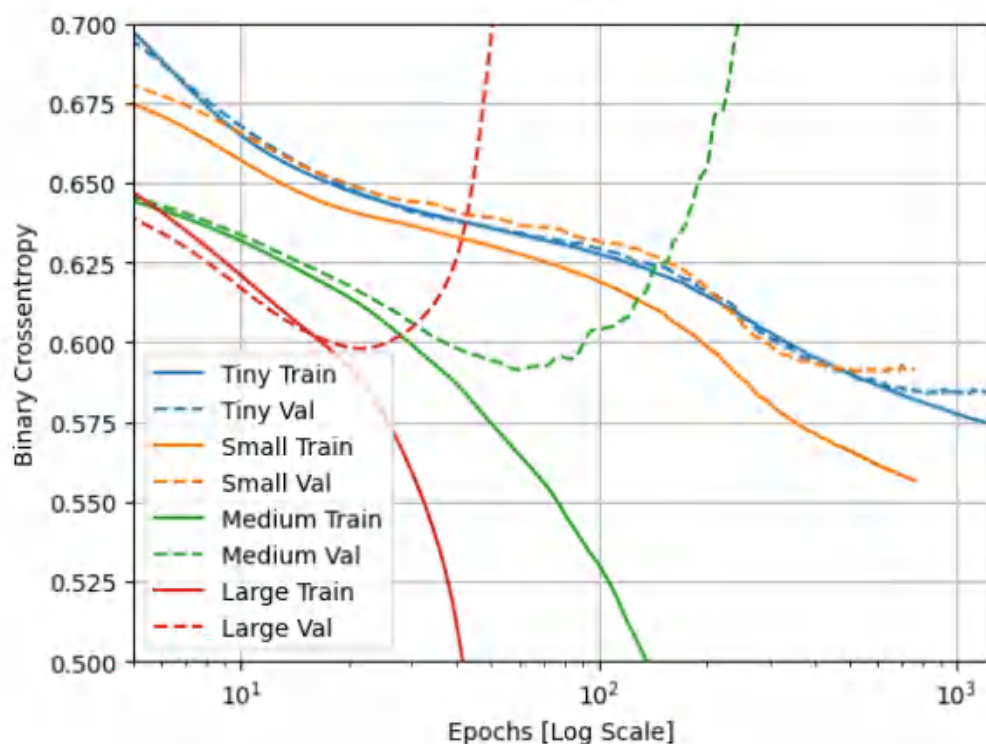
Solution 2.16: What is overfitting? How can you tell if your model is overfitting?

Overfitting is a phenomenon in machine learning where a model is trained to fit the training data too closely, to the extent that it starts to memorize noise or idiosyncrasies of the training data, rather than learning the underlying patterns that would generalize to new, unseen data.

This can lead to a model that performs well on the training data but poorly on new data. Overfitting can occur when a model is too complex (e.g., has too many parameters) relative to the size of the training data, is trained for too long, or model weights become too large. In such cases, the model captures noise or outliers in the training data, which can cause it to make incorrect predictions when presented with new data.

There are several signs and methods to detect overfitting in a machine learning model:

- **Accuracy gap:** If the accuracy of the model on the training data is significantly higher than its accuracy on the validation data, it may indicate overfitting.
- **Learning curve:** A learning curve that shows a rapid decrease in training loss and a slow decrease, or a plateau, in validation loss can also indicate overfitting. Validation loss that inverts – starts to increase – is also a sign of overfitting.



Note. Validation loss going in the wrong direction is a sure sign of overfitting. When validation loss plateaus while training loss improves, the model is close to overfitting. Source: (*Overfit and Underfit*, 2022).

Developing strategies to address overfitting is crucial. Models can often achieve strong performance on the training data, but the objective is to create models that generalize effectively to test data and unseen data.

Solution 2.17: How to deal with overfitting?

There are generally three levers for reducing overfitting:

1. **More complete training data:** Having more complete data necessitates less regularization and enables complex model architectures to reach their potential. However, it's important that the training data contains a comprehensive range of inputs. More data may not be helpful unless it covers novel cases.
2. **Regularization:** Techniques like L1 and L2 incorporate a penalty term into the loss function, decreasing the magnitude of weights, making it more difficult for the model to learn from noise in the training data. Another technique is Dropout, which randomly sets unit outputs in a given layer to zero. The intuition behind Dropout is to avoid “co-adaptation” (Srivastava et al., 2014), which discourages weight updates from systemically fixing errors produced by other units.
3. **Reducing model capacity:** A simpler model is less likely to overfit as it is naturally more constrained in the amount of information it can memorize. If the model can store a limited number of patterns, the training procedure forces it to learn the most important patterns, which increases the model's generalization ability. Reduce model capacity by reducing the number of input variables, the number of layers, and the size of each layer.

Understand that these three levers are complementary: more data may necessitate less regularization, larger capacity may require more regularization, and having more data can allow for larger capacity.

Beyond these key levers, here are some additional techniques:

- **Early stopping:** Interrupts the training process when the validation loss stops decreasing, since this is when the model usually begins to learn more from the noise in the training data rather than the signal.
- **Ensemble methods:** Combining multiple models, such as with bagging or boosting, can also help mitigate overfitting by reducing the variance of the model.

Insider Tip

Read through the Tensorflow tutorial on overfitting and underfitting (*Overfit and Underfit*, 2022) to see an example of the impact of model capacity and regularization on validation loss.

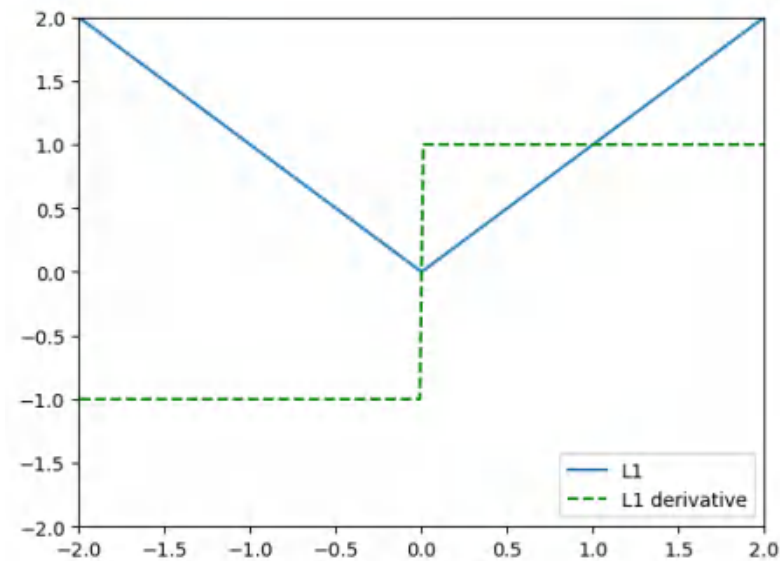
Solution 2.18: Regularization – what is it? List 2 approaches and compare them.

The goal of regularization is to prevent overfitting by reducing the complexity of the model. L1 and L2 regularization are two commonly used techniques which add a penalty term based on the magnitude of the model weights.

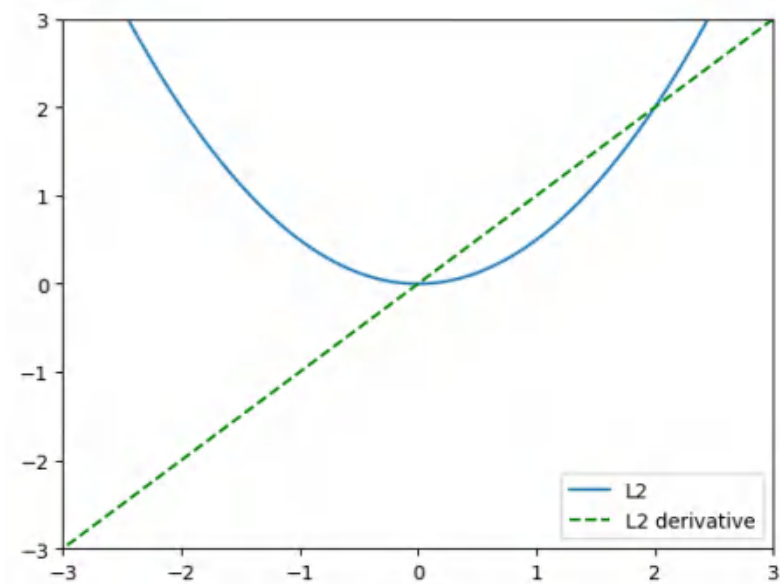
L1 regularization, also known as Lasso regularization, adds a penalty term to the loss function proportional to the absolute value of the weights. This leads to sparse models where some weights are exactly equal to zero, effectively reducing the number of features the model uses.

L2 regularization, also known as Ridge regularization, adds a penalty term to the loss function proportional to the square of the values of the weights. This results in small, non-zero weights, which reduces the complexity of the model without making it sparse.

Why does L1 make weights sparse while L2 does not? This is because the gradient of L1 is either -1 or 1 except where the weight is zero, so the penalty moves the value closer to zero by the same increment, regardless of the weight's current value. By contrast, the gradient of L2 is a linear function that decreases to zero as the weight approaches zero, resulting in a penalty that diminishes as the weight value becomes smaller.



Note. For L1 (solid), the gradient (dashed) moves weights towards zero with the same step size (-1 or 1). This encourages weights to become sparse.



Note. By contrast, with L2 (solid) the magnitude of the gradient (dashed) decreases as weights approach zero. In other words, step sizes get smaller.

In general, L1 regularization is used when the objective is to reduce the number of features the model uses (such as redundant features), while L2 regularization is used to reduce the magnitude of the weights, spreading them out more evenly, without necessarily setting them to exactly zero.

Solution 2.19: What's the difference between linear and logistic regression?

Linear regression models a linear relationship between a dependent variable and one or more independent variables by fitting a best line. Defined as:

$$y = \beta_0 + \beta_1 x_1 + \dots$$

Logistic regression, on the other hand, models the probability of a binary outcome (where the dependent variable can only take two values, such as 0 or 1) by fitting a logistic curve. The logistic regression model outputs probabilities which are then transformed into binary outputs. Softmax regression is the generalization of logistic regression to handle more than two classes. The logistic function is defined as:

$$p = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \dots)}}$$

Linear regression minimizes the sum of the squared errors between the predicted and actual values in the data. This is often done using the method of least squares (*Least Squares*, n.d.).

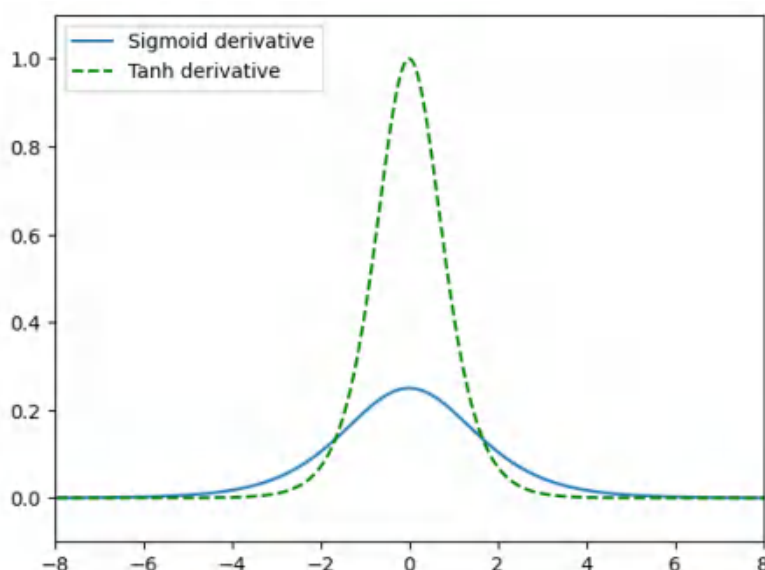
Logistic regression maximizes the likelihood of the observed data by minimizing the negative log-likelihood, known as the Log Loss (or Cross-Entropy) function, which can be achieved with various optimization algorithms such as gradient descent or quasi-Newton methods.

Solution 2.20: Compare several different activation functions in neural networks.

Here are some commonly used activation functions and their characteristics:

- **Sigmoid:** Maps input to the range of 0 to 1, making it useful for producing probabilities (such as for binary classification problems). Can lead to vanishing gradients as the function becomes saturated with large or small values. Not zero-centered, so gradient updates always have the same sign, which may result in slower convergence.

- **Softmax:** Often used as an output layer for multiclass classification problems, mapping input values to a probability distribution over multiple classes.
- **Tanh:** Maps input to the range of -1 to 1, meaning that it can output negative values which may be useful in some contexts. Compared to the sigmoid function, the gradient near zero is notably larger, leading to more substantial updates. However, tanh saturates as well, which makes it prone to vanishing gradients like the sigmoid function. Zero-centered, so it doesn't suffer from zigzagging gradient updates.

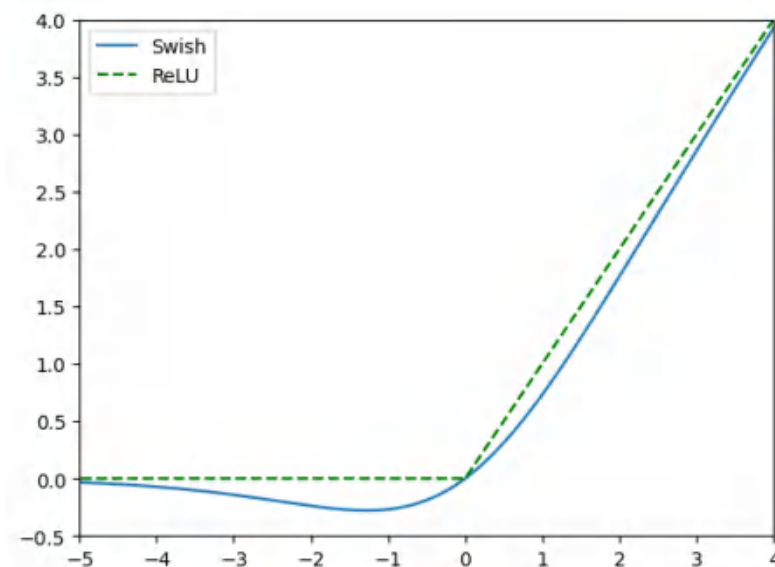


Note. Derivatives of sigmoid (solid) and tanh (dashed). Observe that tanh has larger gradients near zero, while both functions saturate rapidly.

- **Softsign:** Similar to tanh, bounded -1 to 1. Smoother than the tanh function, resulting in less saturation and fewer issues with vanishing gradients. As a result, convergence may be faster. However, computation is more expensive due to complex derivatives.
- **ReLU (Rectified Linear Unit):** ReLU is a piecewise linear function that replaces negative values with 0 and keeps positive values unchanged, which avoids saturation. Gradients do not vanish like they do in sigmoid and tanh. Fast to compute and has been shown to work well in practice for DNNs. However, one issue is the “dying ReLU” problem,

which can occur when inputs to the ReLU function become negative, making the unit inactive and causing the gradient to be zero during backpropagation. This problem can be difficult to recover from as a result.

- **Softplus:** Smoother version of ReLU, negative bounded at 0 and positive unbounded. The smoothness and non-zero nature of the gradient improve training stability (Zheng et al., 2015). However, computation is more expensive than ReLU due to complex derivatives (it's the sigmoid function).
- **Leaky ReLU:** Similar to ReLU, but with a small slope for negative inputs, to alleviate the problem of dying ReLUs. Negative values also encourage mean unit activations closer to zero.
- **PReLU:** Variant of Leaky ReLU that has a learnable parameter for the slope of inputs less than zero, rather than a predetermined value.
- **ELU (Exponential Linear Unit):** Popular among ML practitioners. Similar to Leaky ReLU and PReLU, with the main difference being saturation to a fixed negative value as the input gets smaller, making ELU negative bounded and positive unbounded. This enables ELUs to have variance dampening capabilities, which demonstrated faster convergence despite higher computational cost (Clevert et al., 2016).
- **SELU:** Expands the benefits of ELUs – ability to control the mean, to decrease and increase the variance of unit activations – but with theoretical convergence towards zero mean and unit variance (Klambauer et al., 2017). This self-normalizing property supports the training of DNNs with many hidden layers without vanishing or exploding gradients.
- **Swish:** Like ELU and SELU, Swish is an activation function that is negative bounded and positive unbounded. Additionally, it is non-monotonic and smooth. The Google Brain team found these properties to produce superior results to ReLU, ELU, and SELU in experiments (Ramachandran et al., 2017). GELU (Gaussian Error Linear Unit) is another activation function that shares the same characteristics.



Note. Swish (solid) and ReLU (dashed) activation functions are both positive unbounded – avoiding saturation – and negative bounded. Additionally, Swish is non-monotonic, with negative values for small negative inputs, and smooth, which is useful for optimization.

When choosing an activation function, consider its (1) **zero-centricity**, (2) **computational cost**, and (3) **gradient anomalies** such as vanishing or exploding gradients. As a result of these considerations, effective activation functions often generate both negative and positive values, and are unbounded on the positive side to avoid saturation.

Insider Tip

For an in-depth review of the various activation functions, see (Szandala, 2021).

Solution 2.21: Can you explain the difference between a decision tree, a random forest, and gradient boosted decision trees (GBDT)?

A **decision tree** is an ML algorithm that recursively partitions data into smaller subsets and builds a tree-like model of decisions and their consequences. Each internal node of the tree represents a split on a feature, each branch represents the outcome of the split and each leaf node represents an output.

Criteria for splitting are usually grounded on metrics of “purity”, such as the Gini index, or information gain, which is based on entropy.

A **random forest** is an ensemble of decision trees, where each tree is trained on a random subset of the data and a random subset of the features at each split. The predictions of all trees are combined to produce the final prediction, either by majority voting for classification problems or by averaging for regression problems. This combination of many decision trees leads to reduced variance and improved generalization performance, with a drawback of reduced interpretability. The training process is straightforward to parallelize.

Gradient Boosted Decision Trees (GBDT) are also an ensemble of multiple decision trees, where each new tree is trained to correct the errors made by the previous trees. GBDTs add new trees by fitting to the negative gradient of a loss function (e.g., MSE) with respect to the predictions, also known as the pseudo-residuals. The splitting criterion is typically based on the loss function. Unlike random forests, the trees in GBDT are dependent on each other, and the final prediction is made by combining the weighted predictions of all trees. Although trees must be trained sequentially, the construction of an individual tree – particularly the split-finding procedure – may be distributed.

GBDTs aim to take a more principled approach to tree building than random forests: using training loss instead of the information gain heuristic for splitting, employing regularization instead of outright pruning, and so on.

Solution 2.22: What’s the difference between boosting and bagging?

Boosting and bagging are two popular techniques used to improve the accuracy of predictive models. Both techniques involve building multiple models and combining their predictions to produce a final output.

A key similarity between boosting and bagging is that they are both ensemble methods that use voting to combine multiple models. In both cases, the goal is to improve the accuracy and reduce the variance of the model.

However, there are several key differences between the two techniques.

- **Boosting** is a sequential process where models are built iteratively, and each subsequent model is trained to improve the performance of the

previous ones by focusing on the misclassifications made by the previous models. However, this makes boosting more prone to overfit than bagging.

- In contrast, **bagging** involves building multiple models in parallel, each trained on a different subset of the training data, which improves robustness to outliers and noise. Bagging can be more readily distributed, which means that the training process can be split across multiple machines to speed up the training process.

Solution 2.23: What are some unsupervised learning techniques? How do they work?

Unsupervised learning is a type of ML where the algorithm learns patterns in data without the need for labeled examples. The goal is to discover structure or relationships within the data, such as clustering similar data points together or reducing the dimensionality of the data.

Some popular unsupervised learning techniques are:

- **k-means:** Unsupervised algorithm used for clustering. It works by partitioning a set of data points into k distinct, non-overlapping clusters such that the points within each cluster are as close as possible to each other and as far away as possible from points in other clusters.
- **EM (Expectation Maximization):** Iterative algorithm used to estimate parameters of statistical models. It works by alternating between computing expected values of the missing data given the current estimates of the parameters (the E step) and then updating the parameters to maximize the likelihood of the observed data (the M step). EM has many applications, including in clustering, natural language processing, computer vision, and speech recognition.
- **PCA (Principal Component Analysis):** Statistical technique used to reduce the dimensionality of high-dimensional datasets while maintaining a high degree of information. It works by finding the linear combinations of the original features that explain the most variance in the data. It works by first centering the data, then iteratively finding the directions of maximum variance in the data through the computation

of the eigenvectors of the covariance matrix (the principal components).

- **t-SNE (t-Distributed Stochastic Neighbor Embedding):** Algorithm for visualizing high-dimensional data in a low-dimensional space. It works by first measuring pairwise similarities between data points in the high-dimensional space, and then finding a lower-dimensional representation of the data that best preserves these similarities. The algorithm uses a probabilistic approach to map similar high-dimensional data points to nearby points in the low-dimensional space, while keeping dissimilar points far apart.
- **SVD (Singular Value Decomposition):** Reduced-rank approximation used to represent the high-dimensional data in a lower-dimensional space, while minimizing information loss. It works by decomposing the original matrix (a co-occurrence matrix) into three separate matrices – a left, right, and singular value matrix – and the product of these matrices approximates the original matrix. The left matrix is regarded as the “embeddings”, while the rank of the approximation is determined by the number of values used from the singular value matrix.
- **Autoencoder:** DNNs used for unsupervised learning that aim to learn a compressed representation of the input data. They work by encoding the input data into a lower-dimensional latent space, and then decoding it back into the original space. Autoencoders have many applications, including in image and video compression, anomaly detection, and dimensionality reduction.
- **VAE (Variational Autoencoder):** The main difference between Autoencoders and VAEs is that VAEs learn a probabilistic distribution of the encoded latent space, while Autoencoders learn a deterministic mapping. VAEs use a variational inference approach to learn this distribution, which involves optimizing a lower bound (ELBO) on the log-likelihood of the data, consisting of a reconstruction term and a regularization term.
- **RBM (Restricted Boltzmann Machine):** Generative models used for unsupervised learning that learn a compressed representation of input data. Consisting of two layers of neurons, one visible and one hidden,

where the visible layer corresponds to the input data, while the hidden layer represents a latent representation of the input. The layers are fully connected to one another. The RBM learns weights of these connections that minimize the difference between the input data and reconstructed data, which is generated by feeding outputs of the hidden layer back through the network.

- **GAN (Generative Adversarial Network):** Type of neural network consisting of two parts, a generator and a discriminator. The generator learns to create new data samples that are similar to a given training dataset, while the discriminator learns to distinguish between the generated samples and real data. The two parts are trained together in a game-like process, where the generator tries to fool the discriminator, and the discriminator tries to correctly identify the fake data, resulting in the generator producing more realistic data over time.

Solution 2.24: Describe k-means. How to initialize? What's the stopping criterion?

k-means is an unsupervised algorithm used for clustering. It works by partitioning a set of data points into k distinct, non-overlapping clusters such that the points within each cluster are as close as possible to each other and as far away as possible from points in other clusters.

The standard version of the algorithm proceeds as follows:

1. **Initialization:** Randomly select k data points to serve as initial centroids for the k clusters.
2. **Assignment:** Assign each data point to the nearest centroid based on the Euclidean distance between the data point and the centroid.
3. **Update:** Calculate the mean of all the data points assigned to each centroid and use it as the new centroid for that cluster.
4. Repeat steps 2 and 3 until the centroids **no longer move** (or move within a tolerance), assignments **no longer change** (or change within a tolerance), or a **maximum number of iterations** is reached.

The result of k-means clustering is a partition of the data points into k clusters, each represented by its centroid.

Drawbacks of k-means: manual selection of the value k, sensitivity to initial seeding (Celebi et al., 2013), issues clustering datasets with certain characteristics – high-dimensional datasets, datasets with many outliers. k-means also has trouble with clusters of varying sizes and densities, though k-means Gaussian mixture models addresses this issue (Guestrin, 2007).

Solution 2.25: What are some semi-supervised learning techniques? How do they work?

Semi-supervised learning is a type of machine learning where a model is trained on both labeled and unlabeled data. By leveraging the information in the unlabeled data, the model can improve its performance on the labeled data.

There are several approaches to semi-supervised learning:

- **Induction:** This involves training a model (self-training), or multiple models (co-training), on the available data and then using them to predict missing labels. The downside is that the trained models may reinforce poor predictions.
- **Transduction:** Instead of building a model only on labeled data, consider all the available data when training a model. This can be achieved several ways: (1) clustering algorithms with partial supervision, (2) graph-based methods like label propagation algorithms (LPA), and (3) manifold learning, which encourages neighboring points in a lower-dimensional space to have similar predictions. There are also models that learn with transductive properties, such as the Transductive SVM and the Laplacian SVM.
- **Hybrid:** Extend unsupervised learning with labeled data, and vice versa. For example, Variational Autoencoders (VAEs) are a type of generative model that can incorporate labeled data into the training procedure. In VAEs, the encoder network takes the input data and maps it to a lower-dimensional latent space, where it tries to represent

the underlying structure of the data, and the decoder maps it back to the original space.

In semi-supervised learning, labels can provide supervision on the encoder side. For instance, a supervised classifier can be trained on the latent space using labeled data. Alternatively, the decoder network can be conditioned so that it generates samples that are consistent with the labels, such as concatenating labels to the latent representation before passing through the decoder network. Read more about this approach in (Kingma et al., 2014). Another example of a hybrid approach is (Isen et al., 2019) which alternates between supervised learning and label propagation.

- When the data is unlabeled at the instance level but aggregated labels are available at the group level, several techniques can be employed to leverage group labels to build an instance level classifier. Read more in [Advanced ML Questions – Learning Without Labels](#).

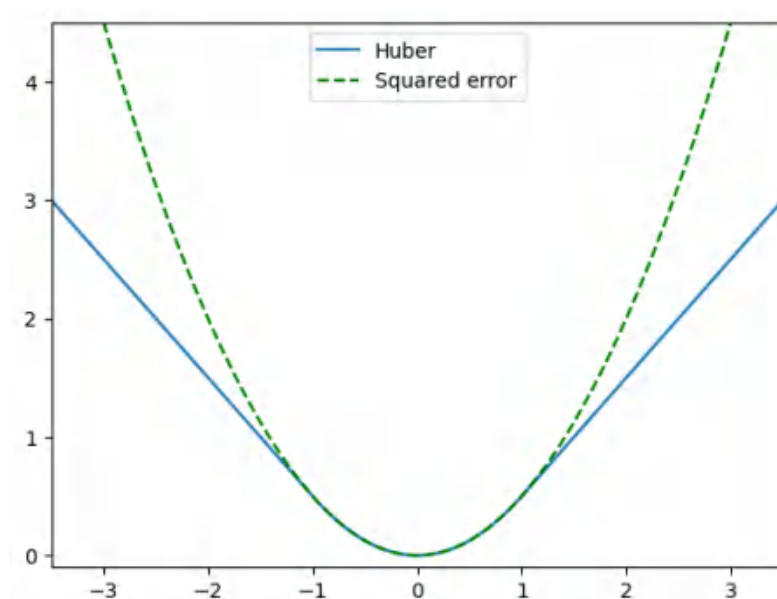
Evaluation

Solution 2.26: What are common loss functions? Compare and contrast.

Common loss functions in machine learning include:

- **Mean Squared Error (MSE):** Average of the squared prediction error. More sensitive to large errors. Gradient decreases as loss moves closer to 0, resulting in more precise convergence as training progresses. Also known as L2 loss.
- **Mean Squared Log Error (MSLE):** Average of the squared logarithmic differences between the predicted and actual values. Compresses large values, which makes MSLE more robust to outliers than MSE. However, there are some quirks. MSLE measures relative error between the predicted and actual values, not the absolute scale of the error. Penalizes underestimation more than overestimation.
- **Mean Absolute Error (MAE):** Average of the absolute prediction error. More robust to outliers. Useful when the magnitude of error for outliers is not a major consideration. Gradient does not decrease with loss, so adding a learning rate is important. Also known as L1 loss.

- **Mean Absolute Percentage Error (MAPE):** Average of the absolute prediction error, expressed as a percentage of the actual value. Similar to MSLE, only measures relative error between the predicted and actual values, not the absolute scale. Useful when dealing with data with varying scales. Avoid actual values of zero.
- **Huber loss:** Hybrid of MSE and MAE. Squared error for small differences, absolute error for larger differences. The cutoff is determined by the hyperparameter δ (delta). More robust to outliers than MSE, yet retains MSE's better precision when loss is small. The choice of δ is important.



Note. Plot of Huber loss (solid) versus squared error loss (dashed). The Huber loss is linear for larger values.

- **Log-Cosh loss:** Logarithm of the hyperbolic cosine of the prediction error. Similar to Huber loss but without the δ hyperparameter, Log-Cosh behaves like squared error for small values and absolute error for larger values. Twice differentiable, which is useful for algorithms like XGBoost.

- **Poisson loss:** Used in regression problems where the goal is to predict count data, such as the number of events in a given time period. Measures prediction error assuming distribution of events follows a Poisson distribution and that they occur independently.
- **Cross-Entropy (or Log Loss):** Used in classification problems, or regression problems where the actual value is between 0 and 1 (e.g., a probability). Measures the error between the predicted probability of the positive class and the actual value:

$$-\frac{1}{m} \sum_{i=1}^m y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)$$

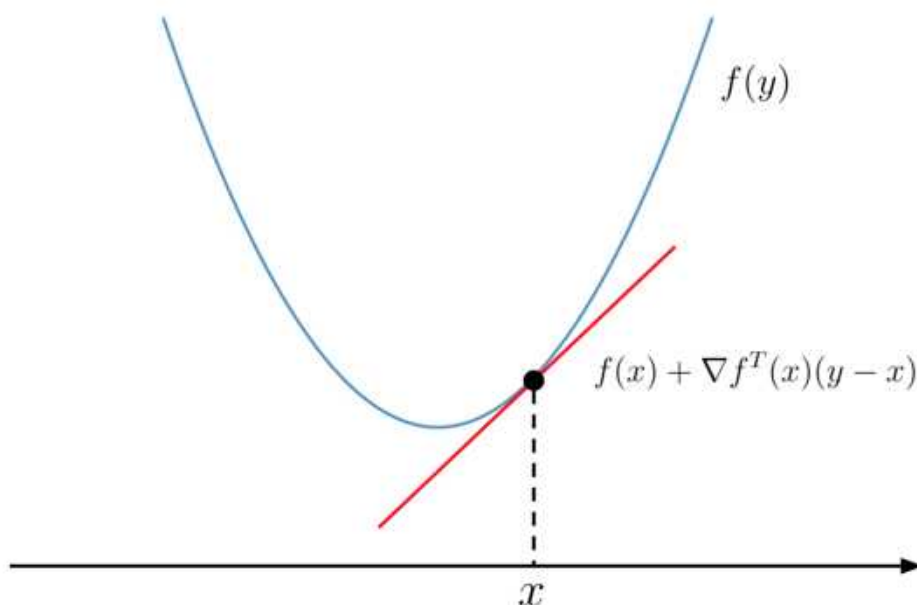
This loss function is used in logistic regression. Read this worked out example comparing Cross-Entropy loss with MSE on the same data (McCaffrey, 2013).

- **Kullback-Leibler Divergence (KLD):** Measure of the difference between two probability distributions, or the relative entropy. Similar conceptually to Cross-Entropy with several differences. First, KLD is always non-negative. Second, when the two distributions are identical, KLD is zero, whereas Cross-Entropy is equal to the distribution entropy. Cross-Entropy is more often used in practice as they solve equivalent optimization problems (Siddiqui, 2022), with the notable exception of Variational Autoencoders (VAE), where KLD is used as a regularization term (Prokhorov et al., 2019) in the loss function.
- **Hinge loss:** Absolute prediction error, except where the prediction for the actual class label greater than a margin incurs zero loss. Can be viewed as trying to maximize the margin between the decision boundary and actual values, and commonly used in Support Vector Machines (SVMs) as a result. While Hinge loss can produce accurate models, it is not as adept at estimating probabilities as Cross-Entropy loss.

Solution 2.27: How to determine whether a loss function is convex?

A loss function is strictly convex if it satisfies either the first-order or second-order condition for convexity. Intuitively:

1. **First-order condition:** If the first derivative of the function, also known as its gradient, is a monotonically increasing function. Alternatively, one can say that the function's tangent line (first-order Taylor approximation) is a global underestimator for the function.



Note. Under the first-order condition, the tangent line should be a global underestimator of the function.

2. **Second-order condition:** If the second derivative of the function is always positive.

Furthermore, the unique global minimum of a strictly convex function is guaranteed to occur at the point where its gradient is equal to zero.

Solution 2.28: How to evaluate a classifier model? List 3 metrics and describe when to use them.

There are many metrics that can be used to evaluate model performance, depending on the type of problem and the evaluation goals. Some of the most commonly used metrics include:

- **Accuracy:** Proportion of correct predictions among all predictions. Use when the specific type of error is not a significant concern, such as when classes are relatively balanced.
- **Precision:** Proportion of true positives among all positive predictions. Use when the goal is to minimize the occurrence of false positives.
- **Recall:** Proportion of true positive among all actual positive cases. Use when the goal is to minimize the occurrence of false negatives.
- **F1 Score:** Harmonic mean of Precision and Recall. Use when sensitive to either low Precision or low Recall.
- **Precision@k:** Version of Precision metric that is more pertinent for ranking problems. Measures the proportion of true positives among the top-k results returned by the system.
- **AUC (or PR AUC):** Precision vs Recall curve. Useful for visualization and thresholding. PR space is preferred to ROC when the number of positives is few, because a large change in false positives only leads to a small change in False Positive Rate in ROC analysis (Davis & Goadrich, 2006).
- **ROC (or ROC AUC):** True Positive Rate vs False Positive Rate curve. ROC space is particularly useful when the positive class is either the majority class or when the classes are relatively balanced.
- **DCG (and nDCG):** Graded relevance measuring the value (or “gain”) of retrieved results, discounted by the position of each result. nDCG is a normalized version of DCG which bounds the metric between 0 and 1. Commonly used in learning-to-rank problems where there is variance in the relative value of every result.

In many cases, metrics are used together to get a better understanding of the model's performance. Metrics sensitive to positives are the most popular for retrieval and ranking tasks, such as F1 Score, PR AUC, and nDCG.

Solution 2.29: How to evaluate the quality of a regression model?

Below are several commonly used evaluation metrics for regression:

- **Mean Absolute Error (MAE):** Calculates the average absolute difference between the predicted and actual values. Expressed in the same units as the dependent variable, making it easier to interpret.
- **Mean Squared Error (MSE):** Computes the mean of the squared differences between the predicted and actual values. Squaring the differences has several benefits. First, it eliminates the possibility of negative differences canceling out positive differences. Second, it penalizes large errors more heavily than small ones, which makes it more sensitive to outliers.
- **Root Mean Squared Error (RMSE):** Square root of the mean of the squared differences. Shares benefits with MSE, however, interpretation is more intuitive than MSE since value is scaled to that of the dependent variable.
- **Root Mean Squared Logarithmic Error (RMSLE):** Similar to RMSE, but it is applied to the logarithms of the predicted and actual values instead. More robust to outliers than RMSE. Measures relative error between the predicted and actual values, not the absolute scale of the error. Underestimation is penalized more than overestimation (Saxena, 2019).
- **R-squared:** Measures the ratio of residual sum of squares to the total sum of squares. Also known as “coefficient of determination” or “goodness of fit”. May be more intuitive than other metrics, since it’s expressed as a percentage.
- **Mean Absolute Percentage Error (MAPE):** Measures the average percentage difference between the predicted and actual values, relative to the actual values. Scale independent, useful when comparing series of different scales. Avoid values of 0 in the dependent variable.

Solution 2.30: Why not optimize a model directly on evaluation metrics?

Models are not directly optimized on evaluation metrics for several reasons:

- **Non-differentiable:** Some evaluation metrics, such as Accuracy or F1 score, are not differentiable. Differentiability is required for optimization using gradient-based methods.
- **Non-convex:** Direct optimization of evaluation metrics may lead to optimization difficulties, such as non-convexity and local minima.
- **Impractical:** Some evaluation metrics, such Precision or F1 score, require many samples to generate reliable values. Other evaluation metrics such as DCG require inputs to be grouped by query. This may be impractical for data extraction, transformation, and loading (ETL), or for model training.

A related question is, “*Why not use the model’s loss function as the evaluation metric?*”

Loss is in fact often used as one evaluation metric (Log Loss in particular), however, typically it’s better to have a separation of concerns. This gives model development the freedom to explore various loss functions, while focusing evaluation on metrics that are most intuitive or informative for the given task.

Solution 2.31: If your model is performing poorly, how to debug and improve performance?

Here are several steps that can be taken to debug and improve model performance:

1. **Define the problem:** Clearly define what is considered as “poor performance”. Is the model compiling? Are there runtime issues? Is the loss not converging, or simply too high? Are model predictions outside of expected bounds?
2. **Visualize the data:** Plot the data to check for patterns, outliers, corruption, and other issues that could be affecting model performance. Determine whether the dataset is being sampled properly, or if there are any ingestion issues. Check the data before and after preprocessing. For instance, missing values might be too

pervasive, handled incorrectly, or numerical features are not properly normalized. Read more about data validation in (*TensorFlow Data Validation* / TFX, 2023).

3. **Check model variables:** By variable, we mean a tensor whose value can be changed via operations (*Introduction to Variables*, 2022). Check variable scoping and proper reuse of variables. Verify variable names and shapes.
4. **Step through the model:** Inspect the model state at each layer for several steps. Print the outputs of all tensor operations, including outputs of activation functions. It is common to discover problems such as incorrect tensor operations, outliers, NaNs, and improper inputs at this stage.
5. **Loss:** Verify the loss computation is correct. Check the learning rate, gradient computation and layer weights. After several steps, verify optimizer behaviors like momentum.
6. **Simplify the model:** If model performance is still difficult to troubleshoot, simplify all components of the model. Ingest fewer features. Use a simple optimizer like SGD. Use fewer parameters, layers, and fewer tensor operations. Once the model is converging and metrics are moving in the right direction, incrementally add complexity back and verify once more.
7. **Debugging with TensorBoard:** Use TensorBoard to visualize the training process, such as loss over time, to help identify issues with the model. For instance, if the model shows signs of overfitting, apply regularization or early stopping.
8. **Model tuning:** To get better performance, experiment with various components of the model:
 - Feature selection
 - Feature preprocessing, see (*Working With Preprocessing Layers*, 2022)
 - Training label
 - Loss function

- Layer activations
- Hyperparameters
- Model architecture

Insider Tip

See (*Keras Debugging Tips*, 2020) for more debugging suggestions.

3. ML Coding

There are generally three types of questions that are asked in the ML Coding interview:

1. Implement a familiar ML algorithm.
2. Write code solving a Probability or Statistics question.
3. Traditional Software Engineering coding question – not directly related to ML.

Below, we discuss patterns you can apply to solve coding questions for (1) and (2). Since (3) is more of a catch-all for Software Engineering coding questions, broad coverage is beyond the scope of this guide. That said, we will review several SWE question templates that show up repeatedly in ML interviews.

Our main focus in this chapter is to help you formulate a plan to tackle these coding questions. Whenever appropriate, such as when demonstrating a specific approach, we will provide solutions in the form of code. In other instances, we may direct you to code solutions in alternative resources.

What Matters Most to Interviewers

In a coding interview, interviewers prioritize several aspects of the candidate's code, which we rank below in order of importance.

1. The first and foremost aspect that the interviewers care about is **whether the code solves the problem or not**. This is because the ultimate goal is to solve real-world problems efficiently and effectively. In many cases, candidates may come up with partial or incorrect solutions. This can happen due to a variety of reasons, such as an inadequate understanding of the problem, errors in logic, or lack of attention to detail. In such cases, even if the candidate has other positive qualities, it can become difficult to overcome an invalid solution.

Therefore, it is crucial for candidates to focus on understanding the problem statement thoroughly before attempting to write the code. It is preferable to have a functional solution rather than hyper-optimizing the code and getting it entirely wrong.

2. The second aspect is the use of **reasonable data structures and algorithms**. The interviewers look for code that uses data structures and algorithms that are appropriate for the given problem. For example, if the problem involves storing key-value pairs, using a hashmap for storing the mappings would be a reasonable choice. Similarly, if the problem requires adding and removing elements in order, using queues and stacks would be a reasonable choice.

These data structures have known time complexities and can be used to solve a wide range of problems. It is important for the candidate to understand the trade-offs between different data structures and algorithms and choose one that is appropriate for the given problem.

3. The third aspect is the **cleanliness of the code**. The interviewers look for code that is easy to understand and maintain. Descriptive and intuitive variable names can make the code more understandable and reduce the need for comments. The layout of loops and conditionals is another important aspect of code structure. If the code is not well organized, it can be difficult to follow the logic of the code, leading to errors and inefficiencies.
4. The fourth aspect is the candidate's **communication skills**. The interviewer looks for the candidate's ability to reason about the code. They might ask questions like the time and space complexity of the code, why a particular data structure was used, or ask the candidate to walk through the algorithm.
5. The fifth aspect is testing. Interviewers appreciate candidates who **proactively test their code**. It goes a long way if the candidate can test the main success scenarios and a few edge cases. Testing the main success scenarios involves testing the code with input values that are expected to produce the correct output. These inputs should cover the most common and straightforward cases.

Testing the edge cases involves testing the code with input values that are expected to produce unexpected or boundary cases. This demonstrates that the candidate has thought about potential edge cases and has taken steps to ensure that the code works as expected in those cases. This builds confidence in their coding abilities and demonstrates their attention to detail.

Implement an ML Algorithm

These questions lead with a discussion about a simple ML algorithm or evaluation metric, followed by some clarifications (e.g., stopping criteria, the mathematical formula if needed, etc.), and then jump straight into code.

In this session the interviewer may ask a couple of questions related to ML Fundamentals to prep the coding question, but is generally not interested in diving deep there, so feel free to ask details if you're fuzzy on the math. For example, if asked to implement the information gain metric, you are not expected to recall the exact formula by memory, and the interviewer should provide it without penalty.

Practice Interview

Here are some example ML algorithm coding questions:

- 7.1: What is k-means?
 - Follow ups:
 - 7.2: How to initialize?
 - 7.3: What's the stopping criterion?
 - 7.4: Can you implement it?
- 7.5: What is k-nearest neighbor?
 - Follow ups:
 - 7.6: What distance metric to use?
 - 7.7: Can you implement it?
- 7.8: What is a decision tree?
 - Follow ups:
 - 7.9: How are splits determined?

- 7.10: What's the stopping criterion?
- 7.11: Can you write the code for information gain?
- 7.12: Can you implement linear regression from scratch?
- 7.13: Code up an evaluation metric, such as nDCG or AUC-ROC.

Study Strategy

The key to improving your ability to implement ML algorithms is through some practice. Here are some suggestions to aid you.

1. Refresh yourself on simple, common ML algorithms:
 - Linear regression, logic regression, naive bayes, k-means, k-nearest neighbor, d-trees, and random forest.
2. Do the same for common evaluation metrics:
 - Mean absolute error, mean squared error, cross-entropy, f-score, AUC-ROC, average precision, recall@k, and nDCG.
3. Practice coding several of these algorithms from scratch. Although you may generally use any language of preference, it's recommended you learn to write these algorithms in Python, as it's most commonly adopted in industry, and knowledge of math libraries like NumPy will make algorithmic coding much easier.

Spend time learning to write vectorized versions of these algorithms with NumPy, it will save you significant interview time and coding effort, and win points with your interviewer as well. Learning how to use **broadcasting** (*Broadcasting — NumPy V1.24 Manual*, n.d.) is particularly helpful when comparing items or lists of different sizes.

Take the **k-nearest neighbor** algorithm, for example. With plain Python, your code may look something like this:

```
def k_nearest_neighbors(query, candidates, k):
    # compute Euclidean distances by looping through all candidates
    distances = []
    for i in range(len(candidates)):
        diff = 0
```

```

for j in range(len(query)):
    # sqrt is not necessary
    diff += (candidates[i][j] - query[j])**2
    distances.append(diff)

# get sorted indices
indices = sorted(range(len(distances)), key=lambda k: dist[k])
return indices[:k]

```

Using broadcast with NumPy is cleaner and more efficient:

```

def k_nearest_neighbors(query, candidates, k):
    # assumes np.arrays are passed in
    diff = candidates - query
    distance = sum(diff**2, axis=-1)

    # np.argsort(dist)[:k] works fine too
    return np.argpartition(distance, k)[:k]

```

another version that's even shorter:

```

def k_nearest_neighbors(query, candidates, k):
    distance = np.linalg.norm(candidates - query, axis=1)

    # np.argsort(dist)[:k] works fine too
    return np.argpartition(distance, k)[:k]

```

The Machine-Learning-Collection github (Persson, n.d.) has code samples for many of the above ML algorithms. Try coding up several ML algorithms from scratch and compare your solutions with those in the github – keep in mind, there isn't a single correct method for implementing these algorithms, and the code references serve as a resource to explore alternative or more optimal implementations.

Solution 7.1-7.4: k-means

k-means is an algorithm used to partition a dataset into distinct clusters based on similarity between data points. It aims to minimize the sum of squared distances between data points and their corresponding cluster centroids. To initialize k-means, we first specify the desired number of clusters k . Random initialization is a common approach, where initial centroids are randomly chosen from the dataset. Alternatively, we can use a more advanced technique like k-means++, which aims to spread out the initial centroids. The stopping

criterion for k-means is typically defined by either a maximum number of iterations or a threshold for the change in centroid positions.

Here's an implementation with maximum number of iterations:

```
def kmeans(input, k, max_iters=100):
    # randomly initialize k centroids
    centroids = input[np.random.choice(
        range(len(input)), size=k, replace=False)]

    for _ in range(max_iters):
        # assign each data point to the nearest centroid
        distances = np.linalg.norm(
            input[:, np.newaxis] - centroids, axis=2)
        labels = np.argmin(distances, axis=1)

        # update centroids
        new_centroids = np.array(
            [input[labels == i].mean(axis=0) for i in range(k)])

        if np.allclose(centroids, new_centroids):
            break

        centroids = new_centroids

    return centroids, labels
```

Solution 7.5-7.7: k-nearest neighbor

k-nearest neighbor (kNN) is an algorithm used for classification and regression tasks. It finds the k nearest data points in the dataset to a given query to predict the label or value of the query. The most commonly used distance metrics in kNN are Euclidean distance and Hamming distance (for discrete variables). Other distance metrics, like cosine similarity, can also be used based on specific requirements.

The sample code for k-nearest neighbor is given above.

Solution 7.8-7.11: Decision tree

A decision tree is an algorithm represented by a flowchart-like structure where each internal node represents a feature, each branch represents a decision rule based on that feature, and each leaf node represents the predicted value. Splits in a decision tree are determined using various criteria, with information gain being one commonly used measure. Information gain quantifies the reduction in entropy in the dataset achieved by splitting it based on a particular feature. The attribute that maximizes information gain is chosen as the split point at each node. Stopping criteria could be based on a maximum depth limit, minimum number of instances required in a node, or when further splits do not improve the classification accuracy significantly.

Here's the code for information gain:

```
def entropy(labels):
    _, counts = np.unique(labels, return_counts=True)
    probs = counts / len(labels)
    entropy = -np.sum(probs * np.log2(probs))
    return entropy

def information_gain(features, labels, split_index):
    feat_split = features[:, split_index]
    unique_values, counts = np.unique(
        feat_split, return_counts=True)
    probs = counts / len(feat_split)

    weighted_entropies = [
        (count / len(feat_split)) *
        entropy(labels[feat_split == value])
        for value, count in zip(unique_values, counts)
    ]

    return entropy(labels) - np.sum(weighted_entropies)
```

Solution 7.12: Can you implement linear regression?

For linear regression, use the normal equation $\theta = (X^T X)^{-1} X^T y$ to minimize the cost function (least square).

```
def linear_regression(features, labels):
    # add a column of ones to features for the bias term
    feat_with_bias = np.c_[
        np.ones((features.shape[0], 1)),
        features
    ]

    # use the normal equation to compute weights
    weights = np.linalg.inv(
        feat_with_bias.T.dot(feat_with_bias)).dot(
        feat_with_bias.T).dot(labels)

    return weights
```

Solution 7.13: Code up an evaluation metric, e.g., nDCG.

Here's code based on the $2^{rel_i} - 1$ definition of relevance for nDCG:

```
def ndcg(predictions, labels, k):
    # Calculate ideal dcg. Use rank + 2 since rank starts at 0
    ideal_labels = sorted(labels, reverse=True)
    ideal_dcg = sum((2 ** label - 1) / math.log2(rank + 2) \
                    for rank, label in enumerate(ideal_labels[:k]))

    # Calculate the dcg from predictions
    pred_indices = sorted(range(len(predictions)),
                          key=lambda x: predictions[x],
                          reverse=True)
    pred_labels = [labels[i] for i in pred_indices]
    dcg = sum((2 ** label - 1) / math.log2(rank + 2) \
              for rank, label in enumerate(pred_labels[:k]))

    if ideal_dcg != 0:
        ndcg = dcg / ideal_dcg
```

```

else:
    ndcg = 0.0

return ndcg

```

Probability & Statistics Questions

Fortunately, the ML interview does not typically involve as rigorous a focus on probability and statistics as might be encountered in a Data Science interview. The interviewer may briefly touch upon fundamental concepts like “p-value”. Probability and statistics concepts are instead woven into the interview process in a more general sense, such as when evaluating one's familiarity with ML fundamentals.

It is not uncommon to get a coding question involving the process of iteratively sampling data and subsequently determining the final state or outcome. There are two main reasons why this is assessed. First, sampling is a widely utilized technique for such purposes as data collection and model training. Second, it serves to evaluate the candidate's basic understanding of probability theory.

Practice Interview

Here are some sample questions you might encounter:

- 7.14: What is reservoir sampling?
 - 7.15: Can you implement it?
 - 7.16: Is there a more optimal way to implement it?
 - 7.17: How to adapt the solution to weighted reservoir sampling?
- 7.18: Draw lottery balls with replacement. Each ball has a value between 1 and N . Continue to draw lottery balls until a final value of V or more. What's the probability that I have a final value of U or less?
- 7.19: Given a $N \times N$ chessboard and a knight in the initial position (i, j) . The knight can take up to K moves. If the knight leaves the chessboard,

it can no longer reenter. What is the probability that the knight remains on the chessboard after K moves?

Study Strategy

Reservoir sampling:

Solution 7.14: What is reservoir sampling?

Reservoir sampling is a technique for selecting a random sample of k items from a large set of n items, where n is either unknown or too large to process in its entirety. The technique works by maintaining a reservoir of k items, and selecting items from the input stream randomly, with a decreasing probability of selection as more items are seen.

Solution 7.15: Can you implement it?

```
class Reservoir:
    def __init__(self, size):
        self.size = size
        self.reservoir = []

        # initialize count to -1 as it gets
        # incremented to 0 on first try_add
        self.count = -1

    def try_add(self, value):
        self.count += 1

        if self.count < self.size:
            self.reservoir.append(value)
            return True

        # sample incoming items with decreasing probability
        else:
            insertion_idx = random.randint(0, self.count)
            if insertion_idx < self.size:
                self.reservoir[insertion_idx] = value
                return True
            else:
                return False
```

Solution 7.16: Is there a more optimal way to implement it?

One disadvantage of the above implementation is that it requires generating a random number for every item in the input, even those that are ultimately discarded. This means that, for large inputs, the probability of selecting an item that is not already in the reservoir becomes very small, and most of the random numbers generated by the algorithm are wasted.

To optimize the algorithm for large inputs, we can take advantage of the fact that the interval of acceptance for each item follows a **geometric distribution**. This means that we can calculate the number of items to skip over before the next acceptance, enabling us to quickly move to the next item to accept into the reservoir. This implementation is called Algorithm L and details can be found in (Li, 1994).

Insider Tip

A Python implementation of Algorithm L can be found in this blog (Steadman, 2020).

Solution 7.17: How to adapt the solution to weighted reservoir sampling?

Weighted reservoir sampling is a variant where each item in the input is associated with a weight that represents its relative probability of being selected. To compute the acceptance probability for each item, we first calculate the sum of weights for all items seen so far. We then divide the weight of each item by this sum to obtain its relative weight. The probability of accepting an item is then proportional to its relative weight.

```
class WeightedReservoir:
    def __init__(self, size):
        self.size = size
        self.reservoir = []
        self.total_weight = 0

        # initialize count to -1 as it gets
        # incremented to 0 on first try_add
        self.count = -1

    def try_add(self, value, weight):
        if weight <= 0:
            return False
```

```

self.count += 1
self.total_weight += weight

if self.count < self.size:
    self.reservoir.append(value)
    return True

else:
    # add item with probability proportional
    # to its relative weight
    random_weight = random.randrange(0, self.total_weight)

    if random_weight < weight:
        insertion_idx = random.randrange(0, self.size)
        self.reservoir[insertion_idx] = value
        return True
    else:
        return False

```

Probability questions:

Dynamic programming is a frequently useful technique for solving coding questions that involve calculating the probability of certain events occurring after a series of steps.

Here are typical procedures for solving such problems using dynamic programming:

1. Set up the initial state: This is the starting point for the problem. For example, in the knight problem, initialize all squares with the probability that the knight stays on the chessboard after zero moves.
2. Take incremental steps: This involves defining some transition rules for going from one state to the next. For example, in the knight problem, the incremental step is to explore all the moves the knight can make to a new position on the chessboard.
3. Accrue the probability from previous states into next states: This involves computing the probability of the current state based on the probabilities of previous states. For example, in the knight problem, the probability of the knight being inside the chessboard after k moves

depends on its previous position and its probability of being inside the chessboard after $k-1$ moves.

4. Determine probability at the end state: This involves computing the final probability of the event(s) we're interested in. For example, in the lottery balls problem, the final probability of the ball being U or less is the sum of all probabilities for values between V and U .

Here are some code examples to help illustrate the steps above:

Solution 7.18: Draw lottery balls with replacement. Each ball has a value between 1 and N . Continue to draw lottery balls until a final value of V or more. What's the probability that I have a final value of U or less?

To solve the lottery balls question, we can define $prob[k]$ as the probability that the sum of all drawn lottery balls equals k . In order to determine the probability of any specific value $prob[k]$, we tally up the probabilities of all valid values that could reach k :

$$prob[k] = (prob[k-1] + prob[k-2] + \dots + prob[k-N]) / N$$

Then, to calculate the probability that the total sum is U or less, we sum up the probabilities for all values between V and U , since V is the stopping criterion. For readability, in the code below, we use `min_val` to represent V and `max_val` to represent U .

```
def probability_max_val_or_less(max_val, min_val, num_options):
    if (max_val <= 0 or min_val <= 0 or
        max_val < min_val or num_options <= 0):
        return 0.0

    if max_val >= min_val + num_options:
        return 1.0

    prob_sum = 1.0
    prob = [prob_sum] + [0.0] * max_val

    for idx in range(1, max_val + 1):
        # prob[idx] represents the sum of all
        # the probabilities of the elements that can
        # reach the value idx, normalized by num_options
```

```

prob[idx] = prob_sum / num_options

# maintain a sliding window such that prob_sum contains
# the sum of at most num_options elements
if idx - num_options >= 0:
    prob_sum -= prob[idx - num_options]

# accrue probabilities for each potential draw
# stop accruing when no more draws can be made
if idx < min_val:
    prob_sum += prob[idx]

return sum(prob[min_val:])

```

Solution 7.19: Given a $N \times N$ chessboard and a knight in the initial position (i,j) . The knight can take up to K moves. If the knight leaves the chessboard, it can no longer reenter. What is the probability that the knight remains on the chessboard after K moves?

For the knights question, let $prob[x][y][k]$ be the probability that the knight remains on the chessboard after k moves, starting from position $(start_x, start_y)$. The base case is when $k=0$, in which case the probability is 1.

For each subsequent k , we can calculate $prob[x][y][k]$ by summing the probabilities of all the valid moves the knight can make from $(start_x, start_y)$ at the k -th move. To calculate the probability of a single move, we need to check if the target position is within the bounds of the chessboard. If it is, we add the probability of that move to the current position to the running sum.

```

N = 8 # size of the board
directions =
[(-2,-1),(-2,1),(-1,-2),(-1,2),(1,-2),(1,2),(2,-1),(2,1)]
num_directions = len(directions)

def probability_on_chessboard(start_x, start_y, num_moves):
    if num_moves <= 0:
        return 1.0

    prob = [[0] * (num_moves + 1)
             for _ in range(N)]
    for _ in range(N):

        # for 0 moves, the probability is 1
        for x in range(N):

```



```

    for y in range(N):
        prob[x][y][0] = 1.0

# take moves incrementally
for move in range(1, num_moves + 1):

    # compute the chances that this move ends inside the board
    for x in range(N):
        for y in range(N):
            prob_sum = 0.0

            for dx, dy in directions:
                new_x = x + dx
                new_y = y + dy

                # if the move ends inside the board
                # accrue the probability mass
                if (new_x >= 0 and
                    new_x < N and
                    new_y >= 0 and
                    new_y < N):
                    prob_sum += prob[new_x][new_y][move - 1]

            # normalize by the number of directions
            prob[x][y][move] = prob_sum / num_directions

return prob[start_x][start_y][num_moves]

```

Hash Table & Distributed Programming

Hash table questions are not frequently asked in ML interviews, but if they do come up, they often serve as a precursor to follow-up questions about distributed computing (e.g., MapReduce).

This is due to the fact that the MapReduce programming model is utilized to extract and aggregate information from large volumes of logs and datasets, which is a common occurrence in real-world industry situations.

Practice Interview

Here are a few sample hash table questions which can be extended to distributed implementations:

- 7.20: Create a co-occurrence matrix from a corpus of text with window size of W .
- 7.21: Create a tf-idf dataset from a corpus of text, which outputs the format:

```
term    document    tf-idf score
```

Study Strategy

Hash table coding problems generally involve aggregating counts over keys, often in an incremental manner. For example, when calculating tf-idf, group the terms in each document and store the counts in a hash table to compute term frequency. To compute document frequency, group the terms from the term frequency hash table and store the count of documents. Finally, the tf-idf values can be computed by iterating over the terms and documents.

The challenge comes when thinking about how to extend these implementations to distributed programming. When extending any coding problem to distributed programming, it can be helpful to think about how to code it up in a **functional programming** style. Functional programming is well-suited for distributed programming for several reasons:

- **Immutability:** Functional programming emphasizes the use of immutable data structures, which means that once a value is set, it cannot be changed. This property is crucial in distributed systems, as it eliminates the need for complex synchronization mechanisms to handle shared mutable state.
- **Statelessness:** Functional programming discourages the use of stateful objects, making it easier to reason about the behavior of a program in a distributed context. Stateless functions can be easily parallelized without worrying about hidden state or side effects.
- **Referential transparency:** Functions in functional programming are referentially transparent, meaning that their outputs are solely determined by their inputs. This allows for better caching and memoization, which can lead to performance improvements in distributed systems.

- **Higher-order functions:** Functional programming languages support higher-order functions, which are functions that accept other functions as arguments or return them as results. This enables a more modular and composable approach to building distributed systems.
- **Fault tolerance:** The immutability and statelessness of functional programming make it easier to implement fault-tolerant systems. Since functions do not depend on shared mutable state, it becomes simpler to recover from failures, such as node crashes, by re-executing the failed function with the same inputs.

Insider Tip

Read more about functional programming through Scala's resources (*What Is Functional Programming?* / *Scala 3 — Book*, n.d.) on the topic.

For example, here is some code that calculates the sum of even integers in an array after doubling their values. Java is used for illustrative purposes.

Imperative style

```
int[] arr = {1, 2, 3, 4, 5};
int sum = 0;
for (int i = 0; i < arr.length; i++) {
    if (arr[i] % 2 == 0) {
        sum += arr[i] * 2;
    }
}
```

Functional style

```
int[] arr = {1, 2, 3, 4, 5};
int sum = Arrays.stream(arr)
    .filter(x -> x % 2 == 0)
    .map(x -> x * 2)
    .reduce(0, Integer::sum);
```

Adopting this perspective can help with structuring the flow in a distributed environment.

With that said, let's dive into the practice interview questions above. Now, MapReduce itself is a complex topic that comprises numerous topics. For simplicity, we will focus on the main stages involved.

- **Map stage:** The input data is split into chunks that are processed by map tasks in a parallel manner. Each map task reads the input data and converts it into (key, value) pairs.
- **Partitioning, sorting and shuffling:** The (key, value) pairs from the map tasks are partitioned and sorted based on their keys, and then shuffled to be sent to the appropriate Reducer tasks. This stage is responsible for grouping together all pairs with the same key.
- **Reduce stage:** The shuffled (key, value) pairs are processed by reduce tasks in a parallel manner. Each reduce task processes all the values for a given key and produces aggregated results for the corresponding key.

Solution 7.20: Create a co-occurrence matrix from a corpus of text with window size of W .

There are two common approaches to calculate co-occurrence counts in MapReduce: the Pairs approach and the Stripes approach (Lin & Dyer, 2010). Below we describe the Pairs approach.

In the Pairs approach, for each word in a given window, we emit key-value pairs representing all possible word pairs in the window. Then we count the number of times each word pair occurs together within the window.

Here are the steps to implement the Pairs approach:

- **Map:** The input data is a set of documents. For each document, split the text into individual words. Then, for each pair of words in a window, generate a key-value pair with the key being the word pair and the value being 1.
- **Reduce:** The input data is the output of the map stage. For each word, we sum up the counts for all the word pairs produced by the mapper. The output is a set of key-value pairs where the key is the word pair and the value is the co-occurrence count.

The pairs approach, while straightforward, can generate a large number of key-value pairs during the Map stage. An alternative method is the Stripes approach, where the Map stage creates a key-value pair with the word as the key and a hashmap of all other words that co-occur with that word, along with their respective counts, as the value.

Insider Tip

To see code for implementing the Pairs and Stripes approaches, check out this blog (Bejeck, 2012).

Solution 7.21: Create a tf-idf dataset from a corpus of text.

Now that we have a basic understanding of how MapReduce works for co-occurrence counts, we can move on to the next problem of computing tf-idf scores. There are several variations to the solution, a simple one is given below:

The approach first generates the term frequency (tf) of each term in each document, where the input is a document, and the output of the mapper is ((term, doc), 1). The reducer then either sums up the raw counts, or produces the document-length normalized counts to obtain the tf.

The inverse document frequency (idf) is then calculated, where the input is ((term, doc), tf), and the output of the mapper is (term, 1) for each document per term. The reducer sums up the counts to get the total number of documents containing the term, and then calculates the idf using the formula $\log(\text{number of docs} / \text{count})$.

Finally, the tf and idf values are joined on the term, and the input is (term, ((doc, tf), idf)). The output of the mapper is ((term, doc), tf*idf), which gives the tf-idf score for each term in each document.

To summarize, the process can be divided into three stages:

1. tf:
 - Input: document.
 - Mapper output: ((term, doc), 1) for every occurrence.

- Reducer sums up the counts $((\text{term}, \text{doc}), \text{tf})$.
 - Alternatively, reducer can produce the document-length normalized counts $((\text{term}, \text{doc}), \text{tf})$.
- In practice, another mapper may reformulate the tuple as $(\text{term}, (\text{doc}, \text{tf}))$ to simplify the joining process below.

2. idf:

- Input: $((\text{term}, \text{doc}), \text{tf})$.
- Mapper output: $(\text{term}, 1)$ for every occurrence, since $(\text{term}, \text{doc})$ is unique.
- Reducer sums up the counts as df and computes $(\text{term}, \log(\text{number of docs} / \text{df}))$, which is $(\text{term}, \text{idf})$.

3. tf-idf:

- Join tf dataset with idf dataset on term .
- Input is $(\text{term}, ((\text{doc}, \text{tf}), \text{idf}))$.
- Output of mapper is $((\text{term}, \text{doc}), \text{tf} * \text{idf})$.

Insider Tip

Explore the code for this and how it aligns with functional programming principles by referring to Spotify's Scala implementation on github (*Big-Data-Rosetta-Code/Tfidf.scala*, n.d.).

Other Coding Questions

In addition to the questions outlined previously, ML interviews may also ask traditional Software Engineering coding questions. As the subject of coding questions is extensively covered in numerous books, we will not repeat that content. Rather, we'll identify the types of traditional coding questions that are more frequently encountered in ML interviews and direct you to additional resources for practice and review.

Here are the traditional coding question types commonly found in ML interviews:

1. Graph questions

2. String questions
3. Array questions

Practice Interview

Sample graph questions:

- 7.22: Given a reference to a node in an undirected graph, perform a deep copy of the graph.
- 7.23: Given a reference to the root node in a directed graph, check whether the graph contains a cycle.

Sample string questions:

- 7.24: Given a list of keywords, find the shortest snippet of text containing all the keywords.
- 7.25: Find the longest substring without repeated words.
- 7.26: Given two strings, check whether s is a subsequence of t . For example: $s = \text{"abc"}$ and $t = \text{"a1b2c3"}$, return True.
- 7.27: Find and replace: given a string, replace all the substrings s with the substring t .

Sample array questions:

- 7.28: Given a 2-dimensional array of 0s and 1s, find the number of islands. An island is a group of connected 1s (up, down, left, right).
- 7.29: Given a 2-dimensional array of 0s and 1s, write a function that dilates, that is it sets every coordinate adjacent (up, down, left, right) to an existing 1 to the value of 1.
 - Example:
 [0 0 0]
 [0 1 0]
 after dilating would be

[0 1 0]
[1 1 1]

- 7.30: Given an array of integers, determine whether there is a majority element, that is an element that occupies over half of the indices.
 - 7.31: Does it make a difference if the array were sorted?
- 7.32: Given an array of integers, find out whether any three integers sum up to 0. This is also known as the “3-sum” problem.

Study Strategy

Graph questions:

There is an underlying strategy for tackling many graph-based coding problems. The strategy involves traversing the graph, either in a depth-first or breadth-first manner, and using some data structures to keep track of visited nodes and other information.

To summarize:

1. Traverse the graph using either DFS or BFS.
2. Use data structures such as a hashmap or a set to keep track of visited nodes.
3. Use additional data structures, such as in the case of detecting cycles, a container to keep track of the nodes on the search path.

Let's apply this strategy to solve a couple of the practice coding questions.

Solution 7.22: Given a reference to a node in an undirected graph, perform a deep copy of the graph.

In the case of the deep copy of an undirected graph, we can traverse the graph and create a new graph with the same structure as the original graph. To achieve this, we use a hashmap to keep track of visited nodes and to map the original nodes to the new nodes in the new graph.


```

class Node:
    def __init__(self, val = 0):
        self.val = val
        self.neighbors = []

visited = {}

def clone_graph(node: Node) -> Node:
    if node in visited:
        return visited[node]

    clone_node = Node(node.val)
    visited[node] = clone_node

    for neighbor in node.neighbors:
        clone_node.neighbors.append(clone_graph(neighbor))

    return clone_node

```

Solution 7.23: Given a reference to the root node in a directed graph, check whether the graph contains a cycle.

In the case of detecting cycles in a directed graph, we can use DFS to traverse the graph and keep track of all visited nodes as well as the nodes in the current search path. If we reencounter a node in the search path, it means that there is a cycle in the graph.

```

class Node:
    def __init__(self, val = 0):
        self.val = val
        self.neighbors = []

# stores all the nodes that have been visited
visited = set()

# stores only nodes that are being explored
search_path = set()

def has_cycle(node: Node) -> bool:
    if node in search_path:
        return True

    if node in visited:
        return False

    visited.add(node)
    search_path.add(node)

```

```

for neighbor in node.neighbors:
    if has_cycle(neighbor):
        return True

search_path.remove(node)
return False

```

The purpose of the two data structures can be demonstrated in the following test cases:

No cycle

```

node1 = Node(1)
node2 = Node(2)
node3 = Node(3)
node4 = Node(4)
node1.neighbors = [node2, node3]
node2.neighbors = [node4]
node3.neighbors = [node4]

```

In the above example, the code visits node4 twice, but there is no cycle. Therefore, search_path will remove node4 and node2. However, when node4 is reached again via node3, it will no longer be explored because of its existence in visited.

Cycle

```

node1 = Node(1)
node2 = Node(2)
node3 = Node(3)
node4 = Node(4)
node1.neighbors = [node2, node3]
node2.neighbors = [node4]
node3.neighbors = [node4, node1]

```

In this case, the code visits node4 twice again, but there is a cycle which is discovered when node3 reencounters node1, as node1 is still in the search_path (node1 → node3 → node1).

String questions:

The **two pointer approach** can be a useful technique for solving various types of substring problems. This approach involves the following steps:

1. Initialize a hashmap to store information about characters or words in the query. This depends on the exact problem.
2. Initialize two pointers, Begin and End, to point to the beginning of the string.
3. Move the End pointer towards the end of the string until a desirable condition is met. This desirable condition will be different for each problem.
4. When the desirable condition is met, move the Begin pointer towards the end of the string until the desirable condition is no longer met.
5. Update the match if it is better than the best solution thus far (e.g., shorter, or longer), depending on the problem.
6. Repeat steps 3-5 until the End pointer reaches the end of the string.

Let's use the above framework to solve the shortest matching snippet problem:

Solution 7.24: Given a list of keywords, find the shortest snippet of text containing all the keywords.

Following the two pointer approach, the concrete steps are as follows:

1. Initialize a hashmap to store the frequency of keywords in the query.
2. Initialize two pointers, Begin and End, to the beginning of the string.
3. Move the End pointer towards the end of the string until all the keywords are present in the substring.
4. When all the keywords are present, move the Begin pointer towards the end of the string until any of the keywords are missing from the substring.
5. Update the match if it is shorter than the current best match.
6. Repeat steps 3-5 until the End pointer reaches the end of the string.

This is an implementation for the shortest matching snippet problem. Note that this code assumes that the input is already split into words and does not perform any text preprocessing:

```

def shortest_matching_snippet(input, keywords):
    # store the count of each keyword
    keyword_counts = {}
    for keyword in keywords:
        keyword_counts[keyword] = keyword_counts.get(keyword, 0) + 1

    begin, end = 0, 0
    count = len(keywords)
    min_window = float('inf')
    min_begin = 0

    # move the end pointer until we have a valid window
    while end < len(input):
        if input[end] in keyword_counts:
            if keyword_counts[input[end]] > 0:
                count -= 1
            keyword_counts[input[end]] -= 1

        # if we have a valid window, move the begin pointer
        while count == 0:
            if end - begin + 1 < min_window:
                min_window = end - begin + 1
                min_begin = begin

            if input[begin] in keyword_counts:
                keyword_counts[input[begin]] += 1
                if keyword_counts[input[begin]] > 0:
                    count += 1

            begin += 1

        end += 1

    # if no valid window was found, return an empty string
    if min_window == float('inf'):
        return ''

    return input[min_begin:min_begin + min_window]

```

This approach can be used to solve many similar problems: for instance, to find the *longest* snippet of text containing all the keywords, simply modify step 5 to update the match only if it's longer than the current best match.

Solution 7.25: Find the longest substring without repeated words.

To find the longest substring without repeated words, let's make a few adjustments to the two pointer approach:

- Step 1: The hashmap stores the last seen index of each word.
- Step 3: Expand End until a repeated word is found.
- Step 4: Move Begin to remove the repeated word.
- Step 5: Update the match if it's longer than the current best match.

Solution 7.26: Given two strings, check whether s is a subsequence of t .

To solve the subsequence coding problem, we can simplify the two pointer approach. We don't need to keep a hashmap or track the length of the match. We can simply move the End pointer forward until we find a matching character. Once we find a match, we move the Begin pointer forward until we no longer have a match.

```
def is_subsequence(text, query):
    len_query, len_text = len(query), len(text)
    begin, end = 0, 0

    while begin < len_query and end < len_text:
        if query[begin] == text[end]:
            begin += 1
            end += 1

    return begin == len_query
```

Solution 7.27: Find and replace: given a string, replace all the substrings s with the substring t .

In terms of general string search (e.g., find and replace), two efficient algorithms are Boyer-Moore and Knuth-Morris-Pratt (KMP). You don't need to memorize details, except to know that both algorithms have in common the idea of preprocessing the query string to create tables that can be used during the search to avoid unnecessary character comparisons. However, they differ in the specific types of tables they create and the way they use them during the search.

For instance, **Boyer-Moore** makes use of two preprocessing steps before starting the search. The first step involves creating a “bad character” table that is used during the search to quickly shift the query to align with a mismatched character in the text. The second step involves creating a “good suffix” table that helps in finding the longest suffix of the query that matches a prefix of the

text. These two tables enable Boyer-Moore to skip over many irrelevant characters.

Insider Tip

We suggest reading the Wikipedia article on string searching algorithms (*String-Searching Algorithm*, n.d.) as a way to gain a general understanding of the topic.

Array questions:

Array-based coding problems encompass a variety of problem types and do not have a single definitive approach. The sample problems provided illustrate only a few of the possible scenarios that can be encountered.

For 2-D array problems, a reasonable strategy involves traversing the array row by row and column by column. When we reach a cell of interest, such as an “island” or a “pixel” in the sample questions, we **investigate the adjacent cells** to accomplish the desired outcome.

Solution 7.28: Given a 2-dimensional array of 0s and 1s, find the number of islands.

One approach to solving the island problem can be to use a depth-first search (DFS) algorithm to traverse the 2-D array and count the number of islands. Specifically, if an element is a 1, increment `num_islands` by 1, and start a DFS traversal from that element to find all connected 1s.

In the DFS traversal, mark the current element as visited and explore its adjacent cells (up, down, left, right). If an adjacent cell is also a 1 and has not been visited yet, mark it as visited and explore its adjacent cells recursively. Once the DFS is complete, go back and continue traversing the array until all elements have been visited.

```
def num_islands(grid):
    if not grid:
        return 0

    num_islands = 0
```

```

rows = len(grid)
cols = len(grid[0])

for x in range(rows):
    for y in range(cols):
        if grid[x][y] == 1:
            num_islands += 1
            dfs(grid, x, y, rows, cols)

return num_islands

def dfs(grid, x, y, rows, cols):
    if (x < 0 or x >= rows or
        y < 0 or y >= cols or grid[x][y] != 1):
        return

    # mark cell as visited to avoid double counting
    grid[x][y] = 0

    dfs(grid, x - 1, y, rows, cols)
    dfs(grid, x + 1, y, rows, cols)
    dfs(grid, x, y - 1, rows, cols)
    dfs(grid, x, y + 1, rows, cols)

```

Solution 7.29: Given a 2-dimensional array of 0s and 1s, write a function that dilates.

To solve the dilate problem, we also traverse the 2-D array row by row and column by column, similar to the approach used in the islands problem. Whenever we encounter a 1 in the array, we are tempted to set its adjacent cells to 1.

However, this approach is flawed as it can lead to assigning more 1s than necessary, including some that were previously 0. To avoid this issue, we first set the adjacent cells to 2, and then iterate over the array again to set all 2s to 1s, thereby dilating the original 1s without introducing new 1s.

1. Traverse the input array row by row, column by column.
2. For each element that is equal to 1, set its adjacent cells to 2. The adjacent cells are those above, below, to the left, and to the right of the current element.

3. Traverse the input array again and set all elements that are equal to 2 to 1.

```
def dilate(grid):
    rows, cols = len(grid), len(grid[0])

    # set adjacent cells to the value 2 temporarily
    for x in range(rows):
        for y in range(cols):
            if grid[x][y] == 1:
                if x > 0 and grid[x-1][y] == 0:
                    grid[x-1][y] = 2
                if x < rows-1 and grid[x+1][y] == 0:
                    grid[x+1][y] = 2
                if y > 0 and grid[x][y-1] == 0:
                    grid[x][y-1] = 2
                if y < cols-1 and grid[x][y+1] == 0:
                    grid[x][y+1] = 2

    # set all 2s to 1s
    for x in range(rows):
        for y in range(cols):
            if grid[x][y] == 2:
                grid[x][y] = 1

    return grid
```

Insider Tip

If you're interested in learning more about dilating by k , you may find this blog post (Ostermiller, n.d.) by Stephen Ostermiller informative.

When solving 1-D array problems, it's worth considering the **benefits of having a sorted array**. For example, take the majority element problem:

Solution 7.30: Given an array of integers, determine whether there is a majority element.

When dealing with the majority element problem, an unsorted array requires a linear time solution that iterates through the array and uses a hashmap to count the frequency of each element. The majority element is the element that appears more than half the length of the array. However, with a sorted array,

we can find the majority element more efficiently in $O(\log N)$ time. Let's see how.

Solution 7.31: Does it make a difference if the array were sorted?

If the given array is sorted, we can take advantage of certain properties. Specifically, if there exists a majority element, it must occupy the middle of the array. Here are the steps:

1. Locate the candidate element, which is the value at the middle index. For arrays with an odd number of elements, the middle index may be positioned to the left or right of the exact middle of the array.
2. Since the array is sorted, we can efficiently find the leftmost occurrence of the candidate element by performing a binary search.
3. Check whether the element at $\text{Left} + N/2$ is equal to the candidate element. If there is a majority element, then the candidate element must also occur at the index $\text{Left} + N/2$. Otherwise, there is no majority element.

```
def majority_element(sorted_arr):
    size = len(sorted_arr)

    if size == 0:
        return None

    # if the array contains one element, it is the majority
    if size == 1:
        return sorted_arr[0]

    # candidate is the middle element
    mid = size // 2
    candidate = sorted_arr[mid]

    # find the leftmost occurrence of the candidate
    left = binary_search(sorted_arr, 0, size, candidate)

    # check if the candidate occurs more than n/2 times
    if (left + size // 2 < size and
        sorted_arr[left + size // 2] == candidate):
        return candidate
    else:
        return None
```

```
def binary_search(arr, start, end, target):
    while start < end:
        mid = (start + end) // 2
        if arr[mid] < target:
            start = mid + 1
        else:
            end = mid
    return start
```

Solution 7.32: Given an array of integers, find out whether any three integers sum up to 0.

Sorting the array can also be helpful in solving the 3-sum problem. By sorting the array, you can apply a modified version of the two pointer approach that we discussed earlier to solve this problem.

In this implementation, we use the left and right pointers to find all unique triplets that sum to zero. If the sum of the current triplet is less than zero, we increment the left pointer to increase the sum. If the sum is greater than zero, we decrement the right pointer to decrease the sum. If the sum is zero, we add the triplet to the result list and increment the left pointer while decrementing the right pointer to find all other possible triplets with the same first element. We skip duplicate elements to avoid redundant triplets.

```
def three_sum(arr):
    size = len(arr)
    arr.sort()
    result = []

    for idx in range(size - 2):
        # if the current number is the same as the
        # previous one, skip it to avoid duplicates
        if idx > 0 and arr[idx] == arr[idx - 1]:
            continue

        left = idx + 1
        right = size - 1

        while left < right:
            # calculate the sum of three elements
            total = arr[idx] + arr[left] + arr[right]

            if total < 0:
                # if sum is less than zero, move the left
                # pointer to the right to get a bigger sum
```

```

        left += 1
    elif total > 0:
        # if the sum is greater than zero, move the right
        # pointer to the left to get a smaller sum
        right -= 1
    else:
        # if the sum is zero, add triplet to the results
        result.append([arr[idx], arr[left], arr[right]])

        # skip duplicates
        while left < right and arr[left] == arr[left + 1]:
            left += 1
        while left < right and arr[right] == arr[right - 1]:
            right -= 1
        # move left and right pointers to next unique elements
        left += 1
        right -= 1

return result

```


4. ML System Design: Part 1

In **ML System Design: Part 1**, we will (a) introduce a framework for tackling system design questions, and (b) walk through a comprehensive example of a system design question.

The ML System Design interview session consists of one end-to-end applied ML modeling question that covers the entire ML practitioner workflow. Recapping the steps of the ML practitioner workflow, introduced earlier in [Prerequisites](#):

1. Offline data exploration
2. Offline creation of features
3. Offline creation of dataset, or join features with existing dataset
4. Offline model training
5. Offline model evaluation
6. Offline model hyperparameter tuning
7. Online hydration of features
8. Online serving the model
9. Online periodic model refresh

For this question, the interviewer typically starts with, “*We would like to build a system that [performs a prediction task]*” and then asks you to walk through the construction of that ML system from initial design through launch. There are many possible variations that may be asked in the ML System Design interview session, here are several examples.

“We would like to build a system that ...”

- Extracts company names from financial documents.
- Identifies topics discussed in news articles.
- Classifies products into a taxonomy.
- Merges duplicate products in a catalog.
- Analyzes the sentiment of products on the web.
- Recognizes bot traffic patterns.
- Infers user demographics.
- Finds users that are similar to one another.

- Generates concise summaries of documents.
- Shows ads on a search results page.
- Recommends content based on user behavior.
- Understands natural language commands.
- ... and many more.

Design Framework

It may be tempting to jump straight into the modeling part when answering an ML System Design question, such as recommending the use of a specific ML algorithm with certain features. However, this is not the right way to approach the problem. Instead, it is important to recall the ML practitioner workflow and organize your response accordingly. When answering these questions in an interview, structure your response into sections following the general outline below:

1. **Clarify the problem:** Why do we want to do this? Who are the end users? How will they be consuming this data? What metrics do we expect to move? Besides this, understanding the operational parameters of the problem will help inform design decisions later in the model building process, such as size of the dataset, latency requirements, and so on.
2. **High-level design:** ML systems are complex and involve more than just model building. The process of producing desired outputs involves a series of steps that need to be considered. This includes identifying the inputs and candidate outputs of the system, as well as understanding the intermediate stages that the data undergoes during the transformation process into the final outputs. For example,
 - In recommender systems, there is (a) a candidate fetch stage, (b) filters and other business logic, (c) a pre-ranking stage to reduce burden on full ranking, (d) full ranking, and (e) reranking e.g., with heuristics applied in post-ranking.
 - For information extraction tasks, stages may include (a) content classification, (b) entity extraction, followed by (c)

entity resolution which itself involves a separate candidate fetch and ranking flow.

3. **Data collection & processing:** Data collection is a crucial aspect of designing ML systems and demands careful attention. What kind of data do we need? Where does the data come from? How much data? To ensure that the training data accurately represents a wide range of items, it may be randomly sampled or stratified to ensure diversity. Additionally, data must be denoised to remove outliers, duplicates, and missing values. And then there's the issue of labeling, annotators may be hired and trained, and where labels are available from user interactions, serving bias must be dealt with.
4. **Feature engineering:** Feature engineering includes exploring datasets, prototyping inputs, selecting relevant features while considering operational impacts, and transforming the data for model training. It can be challenging because the datasets and corresponding dataflows can be very large and complex, knowledge of the problem domain is required to create useful features, and experimenting with feature combinations to optimize model and serving performance can be time-consuming.
5. **Modeling & evaluation:** To design a successful ML model, practitioners must navigate various challenges. These challenges include experimenting with various model architectures, handling dataset quality and bias issues, dealing with model fit (overfitting and underfitting) and hyperparameter tuning, balancing model and dataset size with computational resources and training time, and selecting appropriate evaluation metrics. Additionally, it's not always clear what the training objective and evaluation metrics should be, as offline goals (e.g., minimize the loss) and online goals (e.g., maximize user engagements) can differ.
6. **Deployment & serving:** Deploying a model can be challenging due to various factors such as latency and infrastructure impacts, and added system complexity. In some cases, deploying a model may require system redesign and optimization to maintain the overall performance. Monitoring and maintenance is a major concern when deploying and/or serving a model. The system also needs to support an online experimentation framework, which involves procedures for model

update and rollback. Creating new models or modifying existing models could also affect the quality of training data and perpetuate biases.

By adhering to this framework for answering ML System Design questions, you can effectively guide your response, address all critical aspects of the system, and highlight your understanding of the complex aspects of system design and execution.

Practice Interview

Let's now walk through a practice interview for recommender systems, or RecSys for short. Recommender systems recommend content that is tailored to individual user preferences, leading to a more personalized experience.

Recommender systems come in various forms and are widely utilized across numerous industries. Some well-known examples include:

- YouTube feed
- Instagram Reels
- TikTok feed
- Twitter feed
- Google News
- LinkedIn: People You May Know
- Netflix movie and TV show recommendations
- Amazon product recommendations
- Online advertisements

The list of interview questions provided below is rather comprehensive, so it is important to note that in an actual interview, only a third or so of these questions may be asked due to time constraints.

Try to answer as many questions as you can on your own before consulting the Interview Solutions section!

Prompt: *“We would like to build a system that recommends content to users tailored to their individual interests and preferences.”*

Clarify the Problem

4.1: Why would we want to recommend content to the user?

4.2: What metrics do we expect to improve?

4.3: What kinds of content should we recommend?

4.4: How to blend different types of content (e.g., in-network, out-of-network, ads)?

4.5: Describe operational parameters of this recommender system (e.g., latency).

High-Level Design

4.6: Describe the high-level design for this recommender system.

4.7: How to generate recommendations for new users and items, aka. the cold start problem?

Data Collection

4.8: What datasets do we need to collect to build this system?

4.9: How to collect these datasets?

4.10: What sorts of biases might be found in the dataset?

4.11: How to mitigate serving bias?

4.12: What about position bias?

Candidate Generation

4.13: Where do candidates come from? List potential sources.

4.14: What are the benefits and drawbacks of each source?

4.15: Describe how candidate generation works at a high-level.

4.16: What are some algorithms for generating candidates?

- 4.17: How might you develop an embedding approach to generate candidates?
- 4.18: At scale, candidate generation cannot score every potential candidate in real-time. What are some ways to solve this problem?
- 4.19: How to index newly created content?
- 4.20: Suppose we need to limit the number of candidates that survive to ranking. How should we merge and prune candidates from different algorithms?
- 4.21: Why not use the candidate generator to rank items?

Pre-Ranking

- 4.22: What is a pre-ranking (or light ranking) model? What does it do?
- 4.23: What should the pre-ranking model learn?
- 4.24: Describe some evaluation metrics for this model.
- 4.25: What are some suitable algorithms for this model?
- 4.26: Suppose we need to pre-rank a large number of candidates. How to optimize the model to support this?

Feature Engineering

- 4.27: What features to use? What are their intuitions?
- 4.28: How to handle textual or id-based features?
- 4.29: What about counting features? Problems with using counting features?

Modeling & Evaluation

- 4.30: What should the heavy ranking model learn?
- 4.31: What are some reasonable modeling algorithms?
- 4.32: Describe what your model architecture might look like.
- 4.33: What if the model predictions need to be calibrated?
- 4.34: What are some appropriate evaluation metrics and why?
- 4.35: How to balance multiple objectives, e.g., Likes versus Subscribes?
- 4.36: Should we build one multi-task model or combine multiple smaller models?

Deployment & Serving

- 4.37: How to enable the model to serve real-time requests?
- 4.38: What can we cache in the serving system and where?
- 4.39: How often to update the model?
- 4.40: How does online experimentation (A/B testing) work?
- 4.41: How should the server support model experimentation?
- 4.42: Describe some ideas for model experiments.
- 4.43: Suppose we would like to predict which offline evaluation metrics improve online metrics (e.g., engagement). How to go about this?
- 4.44: How to debug if online experiment performance drops over time?

Interview Solutions

Clarify the Problem

Solution 4.1: Why would we want to recommend content to the user?

By aiding users in discovering engaging content amidst a vast amount of data, recommender systems offer several benefits:

1. **Relevance:** Recommender systems recommend items that are tailored to individual users' preferences, leading to a more personalized experience for users.
2. **Discoverability:** Recommender systems make it easier for users to discover new items and can save them time and effort from searching. It also becomes more convenient for user content to be discovered by others.
3. **Increased engagement:** By providing relevant recommendations, recommender systems can increase engagement, driving user retention and organic growth.
4. **Increased revenue:** Recommender systems can directly contribute to increased revenue, as users are more likely to purchase items they are interested in.
5. **Better understanding of customer behavior:** The data collected by recommender systems can provide valuable insights into customer behavior and preferences, which can be used to improve products and services.

Recommender systems play a significant role in various industries. Research conducted in 2013 found that recommender systems accounted for 35% of Amazon's consumer purchases and 75% of the content consumed by Netflix subscribers (MacKenzie et al., 2013). By 2016, this figure was 80% (McAlone, 2016).

Solution 4.2: What metrics do we expect to improve?

Consumer:

From the consumer standpoint, our main goals are to improve the following metrics:

- **User retention**, such as daily and monthly active users, time spent, and churn rate.
- **Positive and negative engagements**, including clicks and reports.
- **User state progression**, which involves converting light users into medium and heavy users.
- **Subscriptions**, also known as “follows”, which increase the density of the network of producers and consumers.

Balancing these metrics, such as determining the relative importance of each metric and achieving a balance between positive and negative engagements, is influenced by business objectives and data science research.

Revenue:

When it comes to revenue, the objectives are more complex. Driving an increase in revenue could potentially lead to negative consequences, such as higher costs for advertisers, which may result in churn. The primary factors to evaluate are:

- **Impression** count, which may fluctuate based on several factors. These factors include the efficiency of the recommendation pipeline (e.g., latency), effectiveness of budget pacing, and the type and quality of the ads displayed.
- **Cost-per-engagement**, where engagements may include impressions and clicks.
- **Monetizable engagements**, which measures the effectiveness of ad relevance. A higher value of this metric indicates that the ads are more relevant and engaging to users.

Generally speaking, the primary objective is to improve **monetizable engagements**, indicating an improvement in ad relevance, while maintaining revenue levels or increasing them. This strikes a balance between user objectives (relevant ads) and business objectives (competitive ad auctions).

If revenue and engagements can be increased simultaneously, it is even more advantageous. However, it is crucial to ensure that the rise in cost-per-engagement is proportional to the improvement in ad relevance.

Solution 4.3: What kinds of content should we recommend?

Consumer:

For the consumer, there are two types of content that the system should recommend:

1. **Non-personalized content:** Non-personalized content may include popular content (frequently engaged with or highly rated by a large number of users), trending content (recently gained popularity), and new content (recently added to the platform). Content may also be popular or trending within specific categories, geographies, or topics.
2. **Personalized content:** Content that is specifically tailored to the user's preferences, taking into account their personal traits and previous interactions. This includes content that is both in-network (specific to the user's current network of connections) and out-of-network (specific to the user's interests and preferences).

Other factors to consider include:

- **Diversity:** Provide a diverse range of content to cater to different user interests and avoid creating a filter bubble. A recommender system should cover a broad range of topics and sources.
- **Serendipity:** Refers to the degree to which the system recommends unexpected or surprising content to the user. The system should include some serendipitous recommendations to keep the user engaged and interested.

Revenue:

On the revenue side, the objective is to display the highest scoring ads while evaluating the following factors:

- Positive and negative predicted **engagements**
- **Bids:** Advertisers employ various strategies, such as max bid and target bid, to place their bids (*Bidding Strategies Overview*, n.d.).
- **Budget pacing:** Campaigns may need to be throttled to ensure smooth spending.
- **Boosting:** Certain types of ad products may be subsidized for business purposes.
- **Revenue share:** This is applicable for ad formats such as video ads (*AdSense Revenue Share - Google AdSense Help*, n.d.).
- Additional **bid optimizations**, such as secondary ad objectives (*Create an App Installs Campaign*, n.d.).

In addition to this, contextual information should be considered. For instance, it is necessary to avoid recommending ads that could potentially conflict with or be deemed offensive to the surrounding organic content. It is also crucial to avoid repeatedly showing the same ad or advertiser to users.

Solution 4.4: How to blend different types of content (e.g., in-network, out-of-network, ads)?

There are several ways that one might blend different types of content. Some of these approaches include:

1. **Interleaving:** This involves alternating between different types of content, such as organic and sponsored content. Interleaving can be non-uniform, such as showing a specific amount of organic content per ad. The interleaving can also be weighted based on factors like relevance and popularity. While this approach can help to diversify recommendations, it is often heuristic-driven and may bias towards certain types of content.
2. **Heuristics:** This involves prioritizing and limiting certain types of content based on factors like diversity, seasonality, serendipity, and business priorities. For example, the system may show fewer ads to new users.

3. **Separated:** This approach recommends one type of content before switching to another, or different types of content in distinct parts of the user interface, such as displaying ads at fixed locations. This can provide a smoother user experience and allow for more targeted recommendations.
4. **Clustered:** This approach groups content into clusters based on shared characteristics or features, such as topics. The system then recommends content from each cluster to provide a diverse set of recommendations on different topics. This can help surface new topics that the user may be interested in.
5. **Global ranking:** This approach ranks all types of content as a hybrid list in a single, unified recommendation. Alibaba developed one such technique (Wang et al., 2019b). This can converge towards an optimal balance of user and revenue objectives, but may not be effective for all types of content. For example, ranking organic and sponsored content together can be a challenge, as it requires determining the relative value from organic content (user engagement and retention) from that of ads (revenue).
6. **Reinforcement learning:** Develop a framework for adapting the insertion strategy for different content types (e.g., organic vs ads) based on interactions with users. Bytedance, the makers of TikTok, designed a two-level system which generates an initial list of recommendations, followed by a decision on where to insert ads (Zhao et al., 2020).

Multiple approaches can be combined, such as using a unified model that ranks both in-network and out-of-network content, with heuristics determining how much content from each type is ranked, or with interleaving applied to the result.

Solution 4.5: Describe operational parameters of this recommender system (e.g., latency).

The operational parameters of a recommender system will vary depending on the specific use case. For example, systems that send emails or push

notifications have significantly different latency considerations than those that provide recommendations in real-time.

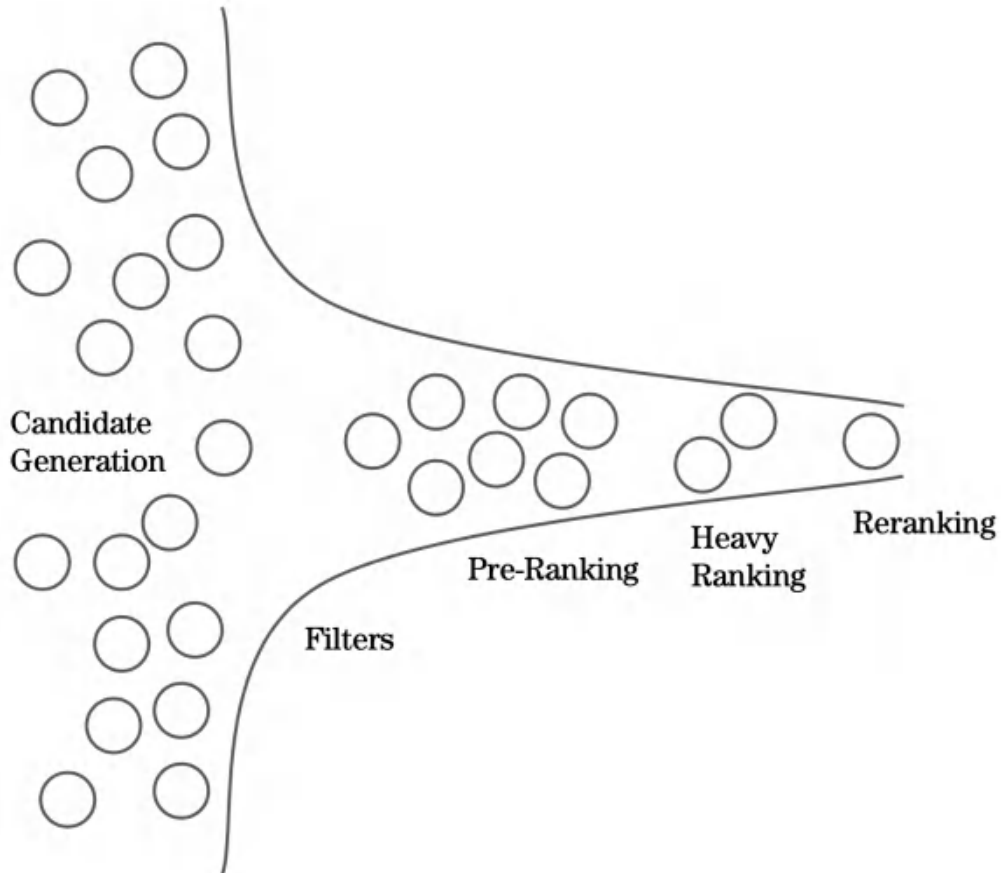
Here are some general operational parameters to consider:

1. **Latency:** The time it takes for the system to generate and deliver recommendations. In a real-time recommender system, low latency is desirable to ensure that users receive timely and relevant recommendations. Latency varies based on system complexity but p99 is typically low 100s of milliseconds due to multiple stages, including candidate, pre-ranking, and full ranking. Pinterest developed a recommender system with a p99 of only 60ms (Eksombatchai et al., 2018). In non real-time systems like email and push notifications, latency is determined by cycle time, which refers to the time it takes to produce recommendations for all users. This can be measured in minutes or even hours.
2. **Throughput:** Refers to the system's ability to process a high volume of requests and deliver recommendations quickly and efficiently. The anticipated throughput of a real-time recommender system depends on several factors such as the platform's active user base, their usage frequency, time of day and year, user interaction patterns, and system design. A figure of several 100s of requests per second for every million users – which refers to recommendation requests sent from clients – is not uncommon.
3. **Number of candidates:** Refers to the total number of items that are considered for recommendation. These candidates are retrieved upstream via candidate generation, and further processed to determine the best items to show to the user. The size varies based on the type and domain of the content, and the content catalog's size: such as tens to hundreds of thousands for ads, or millions for users to follow (Goel et al., 2015). The above-mentioned Pinterest recommender system has a catalog in the billions.
4. **Number of results:** This parameter refers to the actual number of recommended items that are shown to the user. This number is determined by factors such as the user interface and the type of content being recommended. Push notifications, for instance, may only produce one result. A real-time feed (videos, news, posts, etc.) on the

other hand might generate a list of several dozen results for the user to choose from.

High-Level Design

Solution 4.6: Describe the high-level design for this recommender system.



Recommender systems typically use a multistage cascading architecture, or “funnel”, with the following components:

1. **Candidate generation:** The recommender system begins by reducing a vast content catalog to a significantly smaller group of candidates to rank. Different candidate generators can be used, each producing its own distinct set of nominees. The candidate generator for YouTube produces few hundred or thousand potential candidates from a catalog

of billions (*Recommendation Systems Overview / Machine Learning*, 2022), a yield typical in the industry due to the number of subsequent filtration and ranking stages. In ads, this process may also be called “selection” as the system identifies candidates based on specific targeting criteria such as demographics and interests.

2. **Filters:** Filters are employed to narrow down the candidates to a relevant subset of items. This involves assessing each item in the candidate pool with criteria related to quality and health (e.g., content’s age, language, popularity, and author) and in the case of ads, budget and privacy constraints. This stage eliminates 20% to 90% of the remaining candidates.
3. **Pre-ranking:** At the pre-ranking stage, also referred to as “light ranking”, lightweight ranking models narrow down the set of potential candidates to a more manageable size, thereby lessening the computational burden of the subsequent heavy models. The goal of this stage is to maximize recall, and by its conclusion, only a few hundred candidates remain for full ranking.
4. **Heavy ranking:** The final ranking of candidates is generated through heavy ranking, sometimes referred to as “full ranking”, which predicts scores such as the probability of certain engagements. With fewer candidates, more complex models can be employed in this stage.
5. **Reranking:** The system may rerank candidates based on a variety of factors, such as diversity or freshness. To ensure diversity, the system may limit the amount of content from any individual author, topic, or candidate generation source. The system may also favor more recent content (freshness), cull low-scoring candidates, or boost certain candidate types. Additionally, reranking can be done for performance reasons, such as with a lightweight model on the client utilizing only local device features. In ads, predictions from heavy ranking must be combined with the advertiser bids along with other elements² to be reranked for auction.

² There are many additional factors to running an ad auction, such as campaign budget and spend-through, revenue share, and other business rules.

Insider Tip

Here is some additional reading on industrial recommender system designs:

- YouTube: Deep Neural Networks for YouTube Recommendations (Covington et al., 2016)
- Baidu: Towards the Next Generation of Query-Ad Matching in Baidu's Sponsored Search (Fan et al., 2019)
- Alibaba: Cascade Ranking for Operational E-commerce Search (Liu et al., 2017)
- Snapchat: Machine Learning for Snapchat Ad Ranking (*Machine Learning for Snapchat Ad Ranking*, 2022)
- Instagram: Core Modeling at Instagram (Bredillet, 2019)

Insider Tip

Read through Google's course on recommender systems (*Introduction / Machine Learning*, 2022), which goes over high-level concepts.

Solution 4.7: How to generate recommendations for new users and items, aka. the cold start problem?

The cold-start problem is characterized by the absence of behavioral interactions. As most recommender system models heavily rely on user-user and user-item engagements, the system may struggle to generate relevant recommendations. To address this issue, there are several techniques that can be employed:

Data collection techniques:

- **New user onboarding flow:** When a new user signs up, they may not have sufficient data for the recommender system to personalize recommendations. Therefore, it's important to have a comprehensive onboarding flow that gathers as much information as possible about the user's preferences, demographics, and interests. The data collected can establish an initial user profile that is partially personalized. This profile can be bootstrapped with behavioral interactions over time to enhance its accuracy.

- **Heuristics:** Based on the limited information available for new users, heuristics can be used to provide recommendations. One approach is to take the average of the embeddings of users or items with similar parameters (*Collaborative Filtering Advantages & Disadvantages / Machine Learning*, 2022). Alternatively, display non-personalized content, such as popular or trending items, and diverse content. As user engagements are collected, gradually transition to more personalized recommendations.
- **Learning period:** Increase the visibility of new items and users on the platform by boosting their presence in recommendations. This boost can be decayed over time or other signals, such as engagement levels. While this method won't directly help with the user onboarding flow, it aids in making new users and items more discoverable.
- **Contextual bandits:** A systematic approach that selects items in a sequential manner, taking into account contextual information of the user and item (Li et al., 2010, Zhou & Brunskill, 2016). The algorithm adjusts its strategy based on user feedback, to maximize long-term user engagements. For example, the Upper Confidence Bound (UCB) algorithm balances exploration and exploitation by recommending items that have the highest upper confidence bound on reward given the available context – historical and context features of the user and item. As users interact more with the system, the algorithm gains more certainty about the users' preferences and can provide increasingly personalized recommendations.

Modeling techniques:

- **Fallback models:** These models provide recommendations when there isn't enough data available to employ the primary model. The system can monitor feature coverage fed into a model and if it is lower than a threshold, it can have fallback options that are less accurate but only use high-coverage or dense features, such as contextual features. Instagram (Bredillet, 2019) and Facebook (He et al., 2014) have applied this technique to recommend content for new users.
- **Collaborative models:** Interpolate between content-forward modeling techniques in cold start, such as collaborative topic regression with

Latent Dirichlet Allocation (LDA) (Wang & Blei, 2011) or collaborative deep learning with stacked denoising autoencoders (Wang et al., 2015), and conventional modeling techniques when historical features are available. One drawback of this approach has been the complex formulation of the objective function.

- **Hybrid models:** Combine user behavioral information and latent content information using deep learning approaches. Examples include DeepMusic (Oord et al., 2013) and YouTube (Covington et al., 2016), which use hybrid methods for generating recommendations. Heterogeneous information networks, such as TwHIN (El-Kishky et al., 2022), can improve cold-start for candidate generation by incorporating diverse types of data.
- **Dropout:** Force the model to apply dropout, reconstructing input from a corrupted version. The aim is to train a model that can still produce accurate representations even when parts of the input data are missing. DropoutNet (Volkovs et al., 2017) is an example of a model that utilizes this approach, improving its performance in cold-start scenarios.

Data Collection

Solution 4.8: What datasets do we need to collect to build this system?

A recommender system typically requires **user-item interaction data**, such as clicks, purchases, or follows, in order to make recommendations. This data is used to train models that can predict a user's preferences and suggest items they are likely to be interested in. The more data that is available, the more accurate the recommendations can be. Additionally, contextual features, such as demographic data about users, and item metadata, such as category or freshness, can also be used to improve the recommendations.

Apart from this fundamental dataset, other important datasets in a recommender system include:

- **Candidate generation:** For candidate generation algorithms, the training data typically consists of positive examples, or instances where the correct answer is known, and negative examples, or instances where the correct answer is not in the candidate list. Since

user-item interactions are typically very sparse, candidate generators do not train on all of the negatives, or only on the positives. Instead, candidate generation algorithms conduct what is known as negative sampling. Negative sampling involves selecting a small subset of items that are not relevant to use as negative examples. The goal of negative sampling is to provide a representative sample of negative items to optimize the training procedure.

- **Pre-ranking:** Pre-ranking models typically engage in knowledge distillation. For knowledge distillation, the training data typically consists of the inputs to the model, along with the outputs produced by the teacher model (e.g., the heavy ranking model) for those inputs. During training, the student model is optimized to produce outputs that are similar to the teacher's outputs.

Unlike in heavy ranker training, the pre-ranking model does not need ground truth labels from user engagements. As a result, there are more options for collecting the data to train the model. One approach is to sample from all of the input candidates to the pre-ranking model and ask the heavy ranker to score them all offline. By doing this, the heavy ranking model can score all the candidates, including those that were not served to users. This powerful approach not only facilitates various model training and evaluation techniques, but also forms the basis for a robust counterfactual reasoning framework, which we discuss next.

- **Counterfactual reasoning framework:** Counterfactual reasoning is a technique to estimate the effect of changing one or more variables in a system. In the context of a recommender system, this means estimating how a user would have behaved if a different set of candidates were presented, and using that estimate to minimize regret.

Counterfactual reasoning is particularly valuable in recommender systems as there are numerous steps through which candidates may be downranked, boosted, or filtered out. For instance:

1. Heuristics used to merge and prune candidate generator outputs.
2. Application of various filters, such as exclusion of content that is old, avoiding consecutive display of items by the same author, language matching, and so on.

3. Ranking performance of the pre-ranking model.
4. Rules around boosting some types of content over others.
5. Candidates may be reranked for diversity or reasons.

Traditionally, improving a recommender system involved conducting a series of online A/B experiments to make individual component tweaks. However, online experiments can be time-consuming and limited in number, requiring prioritization.

Instead, the counterfactual reasoning framework can be employed offline to tackle this challenge. One approach is to allow all generated candidates for a sample of requests to survive until heavy ranking and annotate each step where a candidate is scored, reranked, or filtered out. Mark the candidates ultimately served. Since all candidates receive a score from the heavy ranker, it's possible to compute the amount of regret that each heuristic, filter, or model contributed to the system. This enables rapid offline experimentation.

While the heavy ranker cannot be considered flawless as it only serves as an estimation of actual user engagement, this framework can help pinpoint potentially fruitful paths for improving the system. These directions can then be validated through online A/B experiments to either support or disprove the hypotheses.

Solution 4.9: How to collect these datasets?

Collecting training data for recommender systems can be a challenging task. The data must be representative of the users and items, it must be large enough to train a robust model, and it must be constructed in a way that minimizes the impact of bias. Let's examine some data collection techniques in several stages of the recommender system.

Candidate generation:

In the candidate generation stage, negative sampling is used in place of optimizing against all items, which is too expensive, or only positive items, which may result in the overprediction of negative items (a phenomenon called "folding"). Negative sampling generates items that are unlikely to be engaged by the user, assisting training algorithms in distinguishing between positive and negative items. Here are some techniques for negative sampling:

1. **Uniform sampling:** Randomly selects items that are not in the user's history as negative samples (Rendle et al., 2009). It is simple and easy to implement, but it may not force the model to effectively distinguish between positive items and negatives that resemble positives.
2. **Popularity-based sampling:** Samples negative items based on their popularity, where less popular items are preferred over highly popular items (Lerer et al., 2019). It can help models differentiate between niche and well-known items, but may result in over-penalization of popular items.
3. **Popularity-dampened sampling:** Dampens the popularity of negative items when sampling (*What Is Candidate Sampling*, n.d.). It can be useful in promoting the discovery of less popular or less known items, but it may also miss some popular items that are still relevant to the user. An example of this is node2vec (Grover & Leskovec, 2016).
4. **Faraway negative sampling:** Selects negative items that are dissimilar to positive items, or items with lower scores. Although this approach seems intuitive by treating nearby nodes as positives and distant ones as negatives, it may also produce uninformative negative samples, much like random sampling.
5. **Hard negative sampling:** This is a popular contemporary approach. Selects negative items that are similar to positive items, or those that are likely to be relevant, such as items with high scores (Zhang et al., 2013). It can help to avoid recommending irrelevant items that are similar to positive items, but it can also be prone to false negatives. There are a number of techniques for this. For instance:
 - A common technique is to sample negative examples in a way that gives higher model scores a greater likelihood of being selected. Examples of this are PinSage (Ying et al., 2018) and RotatE (Sun et al., 2019).
 - Monte Carlo Negative Sampling (MCNS) (Yang et al., 2020) is a technique that utilizes a random walk to identify negative samples through a balance of exploration and exploitation. The

negative samples are selected in proportion to their predicted model scores.

- Generative Adversarial Networks (GAN) define an adversarial learning framework for selecting negatives. KBGAN (Cai & Wang, 2018), for instance, involves using a softmax-based model as the generator to sample negatives likely to confuse the discriminator, and a margin-loss based model as the discriminator. However, this may be inefficient for generating a substantial number of negative samples.

Pre-ranking:

When constructing a training dataset for a pre-ranking (light ranking) model using the knowledge distillation strategy, there are several options to consider:

1. **Biased:** Sample candidates that survive the pre-ranking stage, or from served candidates.
 - Pros:
 - Easy to collect, since these candidates are logged downstream.
 - Allows the model to learn from stronger candidates.
 - Cons:
 - The model will fail to distinguish weaker candidates.
 - Dataset does not accurately represent the input data the model receives.
 - Creates an unfavorable feedback loop for the pre-ranking model.
2. **Uniform:** Sample input candidates completely at random.
 - Pros:
 - Provides an accurate representation of the input data.
 - Most diverse in terms of the training data the model encounters.
 - Cons:

- Inability to evaluate ranking metrics, which are typically the most informative for pre-ranking models.
 - Limitations on training options, such as listwise ranking methods, or training on top- n pointwise labels.
3. **Request-level:** Sample all input candidates for a given request or query.
- **Pros:**
 - Provides an accurate representation of the input data.
 - Unconstrained in terms of training options, evaluation metrics, counterfactual reasoning, and so on.
 - **Cons:**
 - Request diversity will suffer, particularly when the number of candidates per request is large.
 - The model may overtrain on sub-populations of weak or clickbaity candidates that have little representation in the final results.
 - Often the most challenging method to implement in production systems.

In practice, it may be beneficial to use a combination of these approaches to construct a more comprehensive training dataset. For example, the uniform dataset can be used for training, while the request-level dataset can be used for evaluation. Alternatively, the request-level dataset can be subsampled to boost request diversity or to reduce the incidence of spurious candidates. It's important to consider the trade-offs of each approach and how they impact the performance of the model.

Ranking:

In a heavy ranking model, the training loop typically involves a stream of data from candidates that are served to users. This data is then fed back into the model, causing it to update and improve its ranking algorithm over time. When it comes to collecting and using training data for heavy ranking models, there are several important considerations:

- **Update frequency:** Data freshness is an important factor to consider when collecting training data for ranking models. There are several

ways to update ranking models, including batch, online, or continuous updates. See the [ML Infrastructure Design – Offline](#) section for a more detailed description of these options. Depending on the update method used, the data collection process may differ. For batch updates, the data collection process typically involves running offline jobs to aggregate logs and other data into a training repository, which is then used to train the model. In contrast, continuous updates require a continuous stream of data to be sent back to the model in real-time, which can involve more complex data processing and storage infrastructure.

- **Delayed feedback:** In many real-time systems, user interactions are only labeled after a possibly long and random delay (Ktena et al., 2019). For example, in a recommender system, a user may not click on an item until seconds or minutes later. In an extreme case, ad products such as mobile app promotions will not have labels for days or even weeks (*Machine Learning for Snapchat Ad Ranking*, 2022) (if ever! See [Advanced ML Questions – Learning Without Labels](#)). Naive strategies which consider any data point a negative example until a positive label becomes available tend to underpredict those instances. There are several strategies for combating this problem which are discussed in the [Advanced ML Questions](#) section.
- **Negative sampling:** The number of positive samples is typically much smaller than the overall pool of served candidates, leading to a severe class imbalance problem. For example, in a mobile app promotion scenario, the probability of an ad impression leading to an installation on the user's device may be $<0.01\%$. This class imbalance can lead to a training dataset that is skewed towards negative examples, which can negatively impact the performance of the model. To address this issue, sampling strategies can be employed during the data collection process, which we described earlier in [ML Fundamentals – Datasets](#). Oversampling and downsampling are commonly used methods to balance the class distribution. Another approach is cost-sensitive training. Negative sampling is often employed in practice. Read more about negative sampling techniques in the preceding question.
- **Explore / exploit:** In a heavy ranker, the model only has access to labels for items that are served to users. This creates a potential feedback loop issue, where the model gets to select its own training

data. This can lead to various biases in the dataset, such as position bias and serving bias, which can affect the performance of the model over the long-term. To mitigate these biases, it is important to incorporate some level of exploration into the training process. These problems and solutions are discussed in greater detail in the following questions.

Solution 4.10: What sorts of biases might be found in the dataset?

There are various biases that can be found in a training dataset for recommender systems. Some of them are as follows:

1. **Position bias:** Occurs when the items at the top of the list are more likely to be clicked or selected by users, leading to an overemphasis on the importance of those items. This can happen because users may not scroll down to see other items or may believe that the items at the top are more relevant.
2. **Presentation bias:** Occurs when the presentation of the items influences the user's decision (Yue et al., 2010). For example, if certain items are presented with more attractive images or descriptions, they may receive more clicks or engagement than other items.
3. **Trust bias:** Occurs when users have a high level of trust in the recommender system, leading them to only consider the first few results or to only select particular items recommended by the system (O'Brien & Keane, 2006).
4. **Serving bias:** Also known as algorithmic bias, this is the tendency of a system to recommend items similar to those that have historically performed well (e.g., been clicked or consumed), in an attempt to exploit the training data. This can lead to the perpetuation of biases in the model, as it may not accurately reflect the full range of user preferences and experiences.

Solution 4.11: How to mitigate serving bias?

Recommender systems are updated in a training loop to improve prediction accuracy and adjust to evolving user preferences. However, this creates a feedback loop where the model selects its own training data, leading to

serving bias. This causes the model to potentially learn from noise and fail to explore promising candidates.

This exploitation/exploration trade-off is a critical concept in recommender systems. To create an unbiased dataset, items may be uniformly shown to users; however, excessive exploration results in a poor user experience.

There are several ways to mitigate serving bias in recommender systems. Here are some popular ones:

1. **ϵ -greedy exploration:** One simple way to mitigate serving bias is to add some random data to the training dataset. The ϵ -greedy exploration strategy recommends the item with the score with a probability of $1-\epsilon$, while uniformly selecting other items at random with a probability of ϵ . Despite its simplicity, this approach often outperforms more advanced methods like bandit approaches in industrial recommender systems.
2. **Contextual bandits:** Thompson sampling and UCB (Upper Confidence Bound) and Thompson sampling are two contextual bandit algorithms used to address the exploration-exploitation dilemma. Thompson sampling is a probabilistic approach that selects items by randomly sampling from the posterior distribution of the model prediction and selects the items with the highest values. UCB selects items with the highest upper confidence bound, which it computes by adding an uncertainty estimate to the model's prediction, thereby promoting exploration. These approaches have been adopted by Twitter (Guo et al., 2020), Yahoo (Li et al., 2010), and Google (Chen et al., 2019a).
3. **Causal inference:** Causal inference methods aim to estimate the causal effects of alternate actions on outcomes of interest. In the context of recommender systems, this can involve using randomization strategies to obtain accurate counterfactual estimates and performing importance sampling to reweight the data. Microsoft applied such techniques for the ad prediction (Bottou et al., 2013) task.
4. **Learning period:** Boosting new items such as new users, ad campaigns, products, etc. with a decay based on time or other signals like engagements can help to mitigate serving bias effects on the cold start problem. This involves giving higher weights to newer data and gradually decreasing the weight over time. However, this approach

does not address other problems caused by exploitation, such as learning from spurious signals or a lack of diversity or exploration of content.

Theoretically, ranking models should be trained solely on unbiased datasets. However, in reality, the exploration dataset may be too small and may also contain other types of biases depending on the sampling strategy employed. In practice, during training, models often learn from a combination of exploration and exploitation data.

Solution 4.12: What about position bias?

These are some effective ways to mitigate position bias in ranking models for recommender systems:

1. Adding randomness to the position of items can help mitigate position bias during the development stage of a new ranking model. By **randomly shuffling** the positions of the items in the recommendations, the model can learn to identify relevant items independent of their position. However, this approach may degrade user experience.
2. **Add positional features:** Incorporate positional features (Chapelle & Zhang, 2009), such as the position of an item in the list, into the model (Zinkevich, 2023). These features can help capture the effect of position bias. During serving, set all items to have position of 1 to negate the impact of position.
3. **Model position bias:** Develop a model that accounts for position bias by modeling item examination and relevance separately, and then applies position bias as a normalizer or regularizer. This approach assumes that item examination depends on the position, while relevance depends on the context and item. Position bias can be modeled globally (same for all all queries), segmented based on query type, or generalized (train a model to predict position bias). Then, the ranking model can be trained to optimize the inverse propensity weighted score to account for the effect of position bias. See examples from Microsoft (Richardson et al., 2007, Craswell & Taylor, 2008), TripAdvisor (Li, 2020), and Google (Wang et al., 2018, Wang et al., 2016b).

4. **Multi-task model:** Incorporate features contributing to position bias as a wide component in a multi-task ranking model, where each task optimizes for different objectives of the ranking. This can help the model learn to account for the effect of position bias in each task without relying on random experiments to obtain the propensity score. See how YouTube implements this approach (Zhao et al., 2019).
5. **Adversarial training:** Define an auxiliary task that predicts the position of items in the training data. Later, during the backpropagation phase, negate the gradient passed into the model to combat the influence of the position feature on model predictions. This approach is commonly implemented in domain adaptation (Tzeng et al., 2017) and machine learning fairness (Beutel et al., 2017) tasks.

Candidate Generation

Solution 4.13: Where do candidates come from? List potential sources.

Candidate generation in recommender systems is the process of selecting a set of items that could be recommended to a user. There are several sources from which candidates can be generated, including non-personalized sources, personalized in-network sources, and personalized out-of-network sources.

Non-personalized sources:

Non-personalized sources are those that are not specific to the user's interests and preferences. These sources can include:

- **Popular content:** Items that are frequently engaged with or highly rated by a large number of users.
- **Trending content:** Items that have recently gained popularity and are being engaged with or rated more often than usual.
- **New content:** Items that have been recently added to the platform and have not yet been rated or engaged with by many users.
- **Trending content for certain areas:** Items that are trending within specific categories, geographic locations, or topics.

Personalized in-network sources:

Personalized in-network sources are those that are specific to the user's current network of connections and activities. These sources can include:

- **In-network content:** Items that are generated by users or businesses that the user is currently following or connected to.
- **Historical content:** Items that the user has previously consumed or engaged with, such as items they have rated highly or added to their favorites.

Personalized out-of-network sources:

Personalized out-of-network sources are those that are specific to the user's interests and preferences but are not necessarily generated by their current network of connections. These sources can include:

- **Content-based filtering:** Items that are similar to items that the user has previously interacted with or shown interest in. This can be based on characteristics such as genre, keywords, topics, media, or other metadata.
- **Collaborative filtering:** Items that are recommended based on the similarity of the user's preferences to those of other users in the system. This can be based on items that other users have interacted with or shown interest in that are similar to the user's own interests.

By default, when discussing candidate generation in recommender systems without specifying the sources, they are referring to **Personalized Out-of-Network** sources.

Solution 4.14: What are the benefits and drawbacks of each source?

Here are some benefits and drawbacks of each source.

Non-personalized sources:

Benefits:

- Can provide a good starting point for new users who have not yet provided any personal information or ratings.
- Can be helpful for users who are looking for popular or trending items in general, regardless of their specific interests.

Drawbacks:

- May not be tailored to the user's specific interests, resulting in less relevant recommendations.
- Can be biased towards certain types of content or popular trends, leading to limited diversity in recommendations.

Personalized in-network sources:

Benefits:

- Can be highly relevant to the user's specific interests and preferences, as they are based on the user's current connections and activities.
- Can provide opportunities for discovery of new items and users within the user's existing network.

Drawbacks:

- Can be limited by the user's current network, resulting in less diverse recommendations.
- May not be effective for new users who do not yet have a well-established network.

Personalized out-of-network sources:

Benefits:

- Can provide highly personalized recommendations that are tailored to the user's specific interests and preferences.
- Can be effective for users who are looking to discover new items and expand their interests beyond their current network.

Drawbacks:

- Can require significant user data and processing power to generate recommendations based on user similarity or content-based filtering.

- May result in a “filter bubble” effect where users are only recommended items that are similar to what they have interacted with in the past, limiting diversity in recommendations.

Solution 4.15: Describe how candidate generation works at a high-level.

Here are the steps of candidate generation for recommender systems:

1. **Fetch sources:** The first step in candidate generation is to fetch sources that can act as seeds for generating candidates. These sources could be user embeddings, content, and other users that have been recently engaged with. For example, a recommender system for a music streaming service might fetch sources such as the user's listening history, songs on the user's playlists, and artists that the user follows.
2. **Generate candidates:** The next step is to use one or more candidate generation algorithms to generate candidates. There are various algorithms that can be used for candidate generation, including graph-based algorithms, deep neural network techniques, and many others.
3. **Filter candidates:** After generating candidates, the next step is to merge and prune them so that a reasonable number survive to the pre-ranking stage. This is important because too many candidates can lead to longer processing times and reduced recommendation quality.

Candidates can be generated from different sources using different algorithms, making this stage many-to-many. For example, a graph-based algorithm might be used to generate candidates based on songs the user recently listened to, while a deep neural network technique might be used to generate candidates based on artists that the user follows.

For example, Twitter applies dozens of combinations to generate candidates for their recommender systems (*The-Algorithm/CR-Mixer*, n.d.).

Solution 4.16: What are some algorithms for generating candidates?

At a high-level, candidate generation can be performed with several different approaches, including:

1. **Neighborhood-based methods:** These methods leverage user preferences between items or other users to generate candidates. Examples of neighborhood-based methods are Slope One and Pearson R. Slope One is an item-based method that predicts the rating of an item by considering the differences in ratings between pairs of items that users have rated. Pearson R is a user-based method that computes the correlation coefficient between the user and other users to identify similar users and recommend items based on their ratings. These methods do not learn embeddings.
2. **Graph-based methods:** These methods model the relationships between items or users as a graph and use graph algorithms to generate candidates. For instance, a graph-based approach might identify similar items or users based on the connections between them and generate candidates accordingly. There are numerous approaches for this:
 - Amazon (Smith & Linden, 2017) has applied content-based filtering based on association rule mining (Agrawal et al., 1993), or co-visitation counts, for many years.
 - YouTube did the same (Davidson et al., 2010).
 - Twitter leveraged the concept of hubs and authorities in a graph structure (Sharma et al., 2016).

There are also graph-based methods that produce user and item embeddings, known as “graph embeddings”, which we will explore in the next answer.

3. **Latent methods:** These methods use algorithms like matrix factorization and deep neural networks (DNNs) to generate candidates. These methods capture latent relationships that are not directly observable in the data and use them to make predictions. There are numerous approaches, such as:
 - Yahoo (Grbovic et al., 2015) applied neural language modeling techniques for product recommendations.

- PinSage (Ying et al., 2018), developed by Pinterest, is a Graph Convolutional Network (GCN) which aggregates feature information from local graph neighborhoods (such as a user's one-hop neighborhood) using neural networks.
- Google (Yi et al., 2019) uses a two-tower approach to generate item recommendations. We will explore this type of technique in greater depth below.

There are also hybrid approaches, such as the PinSage example above, which combine both content-based information and graph structure.

Solution 4.17: How might you develop an embedding approach to retrieve candidates?

We will now examine three embedding techniques that utilize the methods mentioned earlier: one that utilizes matrix factorization, another that makes use of deep neural networks, and a third that employs graph-based methods.

Matrix factorization:

Through much of the 2010s, a popular approach for candidate generation was matrix factorization (MF) (Koren, 2008). For example, Facebook implemented MF for item recommendation in 2015 (Ilic & Kabiljo, 2015). Matrix factorization methods learn embeddings for users and items by decomposing the rating matrix into low-rank matrices.

One technique for computing the embeddings is through **singular value decomposition (SVD)**, where a low-rank approximation is used to represent the user-item interaction matrix in a lower-dimensional space while minimizing information loss. It works by decomposing the original matrix into three separate matrices – a left, right, and singular value matrix – where the product of these matrices approximates the original matrix. The left matrix is regarded as the embeddings.

Another approach is **weighted matrix factorization**, where an objective function consists of two terms: one for minimizing the error over observed pairs and another for minimizing the error over unobserved pairs. By incorporating the weights into the objective function, the weighted matrix factorization approach can handle datasets with a high degree of sparsity.

Here is an objective function for weighted matrix factorization, where A is the sparse user-item interaction matrix, $\langle U_i, V_j \rangle$ is an inner product of the user and item embeddings, and w_0 is a hyperparameter that balances the two terms.

$$\min_{U \in \mathbb{R}^{n \times d}, V \in \mathbb{R}^{m \times d}} \sum_{(i,j) \in \text{obs}} (A_{ij} - \langle U_i, V_j \rangle)^2 + w_0 \sum_{(i,j) \notin \text{obs}} (\langle U_i, V_j \rangle)^2$$

Optimization techniques such as **stochastic gradient descent (SGD)** and **weighted alternating least squares (WALS)** are used to optimize the embeddings. WALS is an optimization technique that alternates between updating the user and item embeddings.

MF techniques have faced a challenge in effectively incorporating user and item features. Certain methods, including SVDFeature (Chen et al., 2012) and the Factorization Machine (Rendle, 2010), have been developed to capture up to second-order feature interactions.

Deep neural networks:

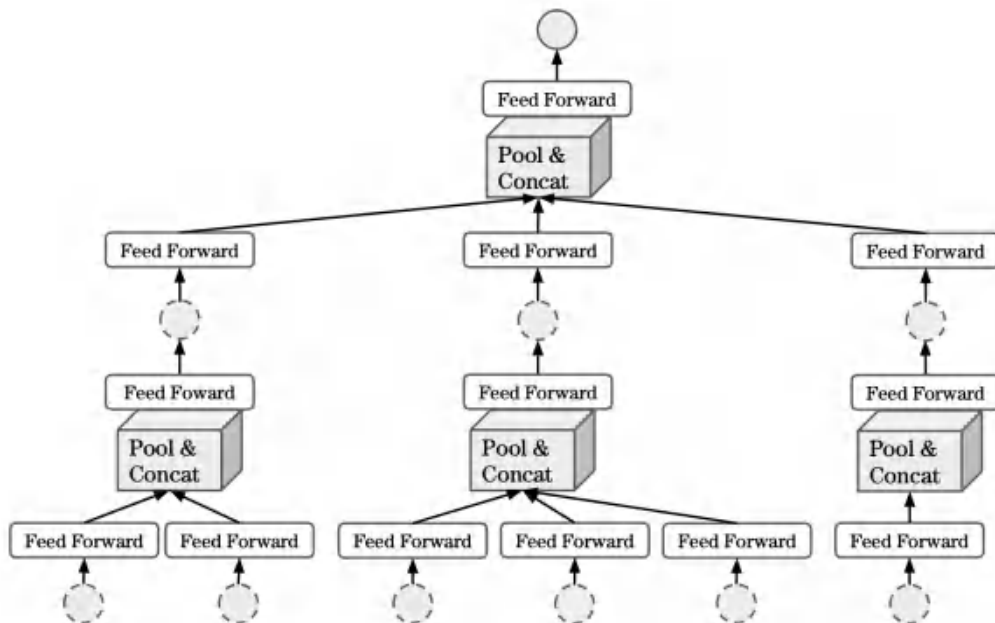
Recent studies have demonstrated the effectiveness of deep neural networks (DNNs) in improving recommendation accuracy over MF techniques (He et al., 2017). Due to their highly nonlinear nature, DNNs are capable of capturing complex feature interactions more efficiently than conventional MF methods.

There are several types of DNNs that can be used to generate embeddings in recommender systems, popular ones including sequence models, graph neural networks, and Two Towers.

- **Sequence models**, such as neural language models, can be used to create user and item embeddings for candidate generation. By treating a user's interaction history as a sequence of “words” (i.e., items), a neural language model can learn embeddings that capture the sequential dependencies between items and the user preferences. Recent advancements have led to the development of models that can leverage attention mechanisms and transformers to generate embeddings. Furthermore, these models can incorporate user and item features in their model structure, enabling them to learn more accurate embeddings that capture both the temporal and contextual aspects of

the data. This technique has been applied at AirBnb (Grbovic & Cheng, 2018), Criteo (Vasile et al., 2016), and Etsy (Zhao et al., 2018). A related method was adopted by Yahoo for news embeddings (Okura et al., 2017).

- **Graph neural networks**, such as Graph Convolutional Networks (GCNs), are a type of network designed to handle graph-structured data. GCNs learn to gather and combine feature information from local neighborhoods of nodes by performing convolutions recursively. A convolution operation aggregates feature information from a node's immediate neighbors and transforms it into a dense representation. By stacking layers of convolutions, graph information can be propagated across distant parts of the graph, allowing GCNs to capture both local and global graph structure. Unlike purely content-based DNNs, GCNs can leverage both content information and graph structure to generate user and item embeddings. PinSage and PinnerSage (Pal et al., 2020) are examples of this.



Note. Visualization of a single node's embedding in a depth-2 GCN. Each dashed node represents a neighboring node in the graph. The node's neighbors are processed incrementally through convolution operations. Each convolution operation consists of a pooling step, a concatenation, and a feed forward network.

Insider Tip

For a survey of graph neural networks, read (Wu et al., 2019).

- The **Two Tower architecture**, also referred to as SplitNet or vector-product models, divides a DNN into two sub-networks that process user and item features separately, and there are no crossing features between them. The output of each network is a fixed-size vector that represents the user and item embeddings, respectively. The score can be computed with different similarity functions, such as an inner product or hadamard product. The Two Tower design offers a notable advantage in that it allows for the independent embedding of new users and items. This is made possible by the isolation of the user and item towers from each other, which is particularly useful in scenarios where there is a constant influx of new users and items. This approach has been adopted by Google (Yi et al., 2019) and Twitter (Wong, 2022) for candidate generation.

Insider Tip

Here are a few other DNN-based approaches for further reading:

- SDNE (Wang et al., 2016a) is an example that incorporates high-order proximity in the node embedding objective.
- More advanced methods use stacked denoising autoencoders to learn node embeddings (Cao et al., 2016).
- Self-supervised techniques train graph encoders (graph neural networks) through actions such as perturbing and reconstructing the graph (Wu et al., 2023a).

Graph embeddings:

Graph embeddings, also known as network embeddings, is a technique used to convert the nodes of large networks into compact low-dimensional representations.

One such approach is **SimClusters** (Sataluri et al., 2020), which is a community-based graph embedding approach developed by Twitter. These

embeddings are used extensively for candidate generation throughout Twitter. In SimClusters, the embedding dimensions are represented by communities with similar followers; they represent groups of users with similar interests or preferences. It works this in this sequence:

1. Follow Graph: Create a follower-followee (also known as consumer-producer) matrix, also known as the Follow Graph.
2. Producer-Producer Similarity: From the Follow Graph, create a producer-producer matrix, computing the cosine similarity between pairs of producers in the Follow Graph based on the fraction overlap between their followers.
3. KnownFor matrix: Densify the Follow Graph into k communities by applying the Metropolis-Hastings algorithm, yielding the KnownFor matrix. This is a lower-dimensional representation of the producer-producer matrix, and it represents the similarity between a producer and the followers of a particular community.
4. User embeddings: to compute user embeddings, multiply the Follow Graph with the KnownFor matrix. They represent the similarity between a user and each community (specifically, the followers of that community), based on who that user follows. These embeddings are also known as user “InterestedIn” embeddings.
5. Item embeddings: compute by aggregating the InterestedIn vector of each user who likes the item (e.g., a tweet). Other interactions can also be used.

Insider Tip

Read more details about Twitter’s SimClusters algorithm on github at ([The-Algorithm/src/scala/com/twitter/simclusters_v2](https://github.com/twitter/simclusters_v2), n.d.).

Another popular technique is using **Heterogeneous Information Networks (HINs)** (Shi et al., 2016). HINs allow us to model networks that contain multiple types of entities (e.g., users, items) and various types of relations (behavioral interactions) among them. In such networks, each entity and relation is represented by an embedding vector.

To generate graph embeddings, the HIN employs a scoring function to triples consisting of the source entity, relation type, and target entity. One such approach is TransE (Bordes et al., 2013), where the scoring function takes the source entity's embedding, translates it by the relation's embedding, and scores it with the target entity's embedding via an inner product. In other words, the scoring function tries to capture the similarity between the embeddings of the entities connected by the relation.

To train the model, the HIN employs a pairwise margin-based ranking criterion. The goal of this criterion is to differentiate between positive observed triples and negative unobserved triples. Negative samples are created by corrupting one of the entities in the positive triple. Twitter has successfully applied HINs (El-Kishky et al., 2022) to represent user and item embeddings for several candidate generation tasks.

There are other graph embedding methods, such as LINE by Microsoft (Tang et al., 2015) which maximizes the probability of observing first- and second-order edges in a graph, given their embeddings. All of these techniques provide scalable and effective ways to embed the nodes of a network into a lower-dimensional space.

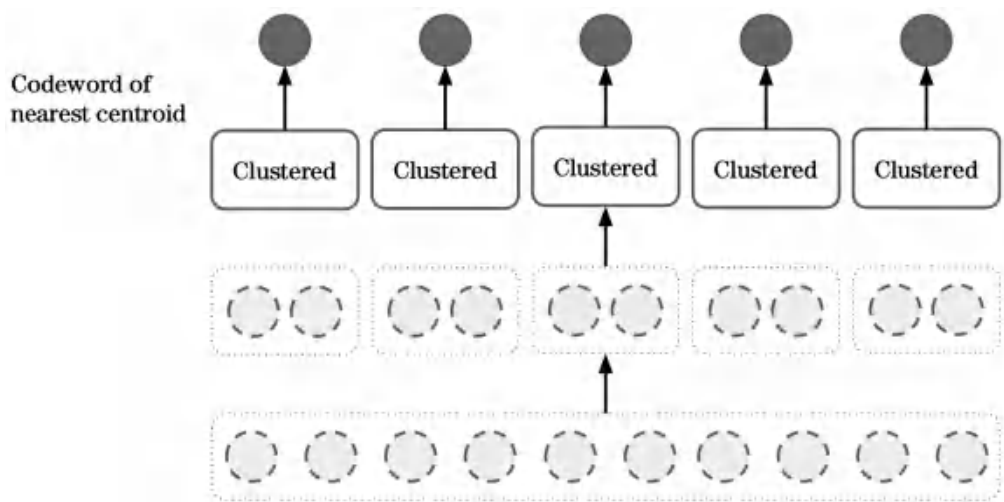
Solution 4.18: At scale, candidate generation cannot score every potential candidate in real-time. What are some ways to solve this problem?

When dealing with large-scale recommender systems, it is often infeasible to score every potential candidate in real-time. To address this issue, here are some potential solutions:

- One popular approach is to use **approximate nearest neighbors (ANN)** to efficiently find the top candidates for each user. ANN is a technique that enables efficient searching of high-dimensional spaces by approximating the distances between points. Here are a few popular methods for implementing ANN:
 - **Locality Sensitive Hashing (LSH):** Efficiently search high-dimensional spaces by hashing similar items to the same bucket with high probability, while data points with low similarity are likely to fall into different buckets. One popular

technique involves random projections, called SimHash. SimHash works by generating binary hash codes based on the sign of the inner product between random vectors and the item vector.

- **Inverted File Index with Product Quantization (IVFPQ):** Designed to handle high dimensional data by compressing and quantizing the data into compact codes. The data is first partitioned into subspaces. Each subspace is quantized by clustering to find centroids, and the data points are assigned to the nearest centroid's codeword for each subspace. The resulting codes are stored in an inverted file index. To search, find the nearest neighbors using the inverted file index by applying the product quantizer to the query vector.



Note. Illustration of product quantization. The original vector is partitioned into subspaces, each subspace is clustered, and the centroids of sub-clusters are represented by codewords.

- **Hierarchical Navigable Small World (HNSW):** Constructs a graph of high-dimensional items with a high clustering coefficient, meaning that nearby nodes tend to be connected to each other, and low diameter, which is the shortest path length between any two nodes, allowing for efficient traversal. In HNSW, the graph is constructed in a hierarchical manner, with each layer representing a different level of resolution (Malkov

& Yashunin, 2020). The top layer contains a small number of highly connected nodes, which are connected to a larger number of less connected nodes in the layer below. This allows for efficient and accurate nearest-neighbor search.

Insider Tip

For an illustration of clustering coefficient, diameter, and how navigable small worlds are constructed, watch the NetSci 10-1 Small World Model Youtube video (*NetSci 10-1 Small World Model*, 2020).

- Another approach is to perform exhaustive scoring offline, **precomputing** and storing a list of the top candidates for each user or item. This can be effective for related-item recommendations, where the pool of candidate items is comparatively smaller and more stable.

Solution 4.19: How to index newly created content?

In many instances, it can be quite straightforward to incorporate new content into the system without the need for model retraining.

- **Matrix factorization:** The matrix factorization approach enables the system to quickly generate an embedding for new users or items. With a small number of behavioral interactions, the system can just solve the following equation, which corresponds to one iteration in WALS (*Collaborative Filtering Advantages & Disadvantages / Machine Learning*, 2022):

$$\min_{v_i \in \mathbb{R}^d} \|A_i - Uv_i\|$$

where A is the user-item matrix, U is the embedding matrix, and v_i is the embedding for the new user or item.

- **DNNs:** The Two Tower approach offers a particularly seamless solution, as passing the new user or item features through the corresponding tower in the DNN and saving the resulting embedding is all that is required. This architecture is purposefully designed to function irrespective of user and item interdependence.

- **Graph embeddings:** In the SimClusters approach, the process is also straightforward since the communities are already predetermined. For users, all that is required is the collection of their follow graph and its multiplication with the existing KnownFor matrix to produce the user's InterestedIn embedding. It is not essential to retrain the KnownFor matrix that represents all communities frequently, since user interests in aggregate do not change rapidly. For items, the process is equally straightforward - one just needs to aggregate the InterestedIn embeddings of the users that have engaged with the item.

Solution 4.20: Suppose we need to limit the number of candidates that survive to ranking. How should we merge and prune candidates from different algorithms?

There are several ways to merge and prune candidates that are generated from several different candidate generation sources and algorithms.

1. **Heuristics:** Use a heuristic to limit the number of candidates from each algorithm or to prefer some algorithms over others. This can be based on the performance of the algorithms on a test set or some prior knowledge about the algorithms. However, the drawback of this approach is that it may not take into account the relative quality of the candidates across algorithms or the overlap between the candidates from different algorithms.
2. **Thresholding:** Adjust the number of candidates from each algorithm based on a threshold defined by the scoring function from that algorithm. However, this approach still does not take into account the relative quality of the candidates across algorithms.
3. **Universal score:** Develop a universal score by calibrating the scoring functions across all algorithms. This involves normalizing the scores from each algorithm based on a teacher model like the heavy ranker prediction. The advantage of this approach is that it takes into account all algorithms while selecting the best candidates from each algorithm. However, some may argue that the scoring function of the candidate generator should not be optimized for ranking. Another drawback is that recalibration can create additional work for ML practitioners.

4. **Regression:** Perform a linear regression based on the scoring function of each algorithm. The dependent variable can be based on a teacher model like the heavy ranker prediction. One drawback of this approach is that most candidates will not be scored by all algorithms. Another drawback is that introducing new algorithms can be slowed down as a result of this approach.
5. **Light ranking:** A fifth approach is to send all the candidates to a very fast light ranker (pre-ranker). The pre-ranker uses more complex models and features that better capture context and user preferences. However, there are drawbacks to this approach. The pre-ranker might be optimized for ranking and not effective in ensuring diversity, which is one aim of using multiple candidate generation algorithms. Moreover, it may not be capable of handling all candidates generated by all algorithms.

Solution 4.21: Why not use the candidate generator to rank items?

While candidate generators do compute a score or similarity measure between items, it is not recommended to rely on them for ranking candidates. Here are some reasons why:

- **Multiple candidate generators** may be used in a system, and their scores may not be directly comparable. Thus, a separate ranking model may be necessary to combine the scores and generate a final ranking.
- The ranking task involves considering a smaller pool of candidates, which allows for the use of **more complex models** and features that better capture context and user preferences. This can lead to more accurate and personalized recommendations.
- The **scoring function** used by the candidate generator may not be optimized for the ranking task, which requires taking into account factors such as optimizing for multiple objectives (such as clicks and likes) and reducing negative engagements. A separate ranking model can be designed to address these factors and optimize for the ranking objectives.

Pre-Ranking

Solution 4.22: What is a pre-ranking (or light ranking) model? What does it do?

A pre-ranking, also known as a light ranking, model is typically a simplified version of a heavy (or full) ranking model. The model is used to quickly rank candidates in advance of the heavy model's application, with the objective of screening for the top candidates. The pre-ranking model primarily focuses on optimizing recall or cumulative gain, as explained in answers below.

The pre-ranking model is employed in situations where the latency of the recommender system is critical, such as in real-time or interactive applications, but also has utility in non real-time recommender systems such as push notifications to reduce overall compute and cycle time.

In order to rapidly evaluate candidates, pre-ranking models employ a reduced set of features and simpler models compared to full ranking models. Through this approach, typically 25% to 90% of candidates are eliminated, thereby allowing the full model to be more complex. A pre-ranking model that sustains a high level of recall or cumulative gain can thus enable the system to generate a more accurate set of recommendations overall.

Solution 4.23: What should the pre-ranking model learn?

Pre-ranking models most commonly use **knowledge distillation**. Knowledge distillation is a technique used to transfer knowledge from a large, complex model (called the teacher model) to a smaller, simpler model (called the student model). The idea is to train the student model to mimic the outputs of the teacher model, rather than trying to train the student model from scratch using the original training data.

The process of knowledge distillation (Ba & Caruana, 2014) involves using the outputs (e.g., probabilities) of the teacher model as the targets for the student model during training. The student model is then trained to minimize the difference between its own outputs and the target outputs. With knowledge distillation, the teacher model can transfer rich information to the student model, while being faster to train and suitable for deployment in resource-limited environments.

The main drawback of knowledge distillation is exactly that same dependence on the teacher model: if the teacher model is not well-trained or is over-parameterized, the distilled knowledge will be erroneous as well.

Another approach is for the pre-ranking model to learn from **ground truth labels** directly, as the full ranker does. However, there are several arguments against this:

- Collecting ground truth for all samples is impractical due to candidates being eliminated during the pre-ranking stage. While it's feasible to collect training data by including eliminated candidates in the final results, this results in a relatively smaller dataset and may lead to underperformance of the recommender system, particularly when considering other exploration conducted by the full ranking model.
- The training data may be imbalanced in ways that's biased towards full ranking tasks due to several factors. First, many recommender systems have several full rankers and only one or two pre-rankers at most, which means the pre-ranking model needs to compress the knowledge from the ensemble. Second, the full ranker training data may be sampled in a way that skews the input distribution to a specific full ranker, which is dissimilar to the input distribution for the pre-ranking model.
- Correlation between stages can be beneficial for reducing system error in aggregate. Furthermore, improvements in the full ranker propagate back to the pre-ranking stage automatically.

That said, it is possible to incorporate ground truth into the knowledge distillation procedure. For example, ground truth labels may be integrated into the training data (Tang & Wang, 2018). The training loss can also be modified to account for ground truth labels (Hinton et al., 2014), or weighted by the sample's significance in relation to the ground truth.

Solution 4.24: Describe some evaluation metrics for the model.

Pre-ranking models aim to maximize the recall of the input candidate pool to retain the top candidates for the full ranker. There are various ways to measure this, depending on factors such as the ultimate user experience:

- **Normalized cumulative gain:** Also known as “cumulative gain ratio”, this is a metric that evaluates the model's ability to identify the best subset of candidates. It is equivalent to the normalized discounted cumulative gain (*Discounted Cumulative Gain*, n.d.) metric but without the position penalty. The gain is determined based on the full ranking prediction, such as the probability of certain engagements, or the predicted revenue of a candidate in ads, excluding elements such as bid and budget pacing parameters that are provided at inference time.
- **Top-k recall:** Evaluates how well the pre-ranking model can identify the top-k candidates, where k is a specified number. For example, if k=1, it measures the model's ability to consistently identify the best candidate, even if it means overlooking the second or third best candidate. This metric is particularly relevant in cases where only one recommendation is presented to the user, like in a push notification, where getting the best recommendation right is essential.
- **Expected top-k loss:** Smoother version of top-k recall that assesses the difference in the gain value of the top-k candidates. This metric is beneficial in predicting the potential loss in engagements or revenue that may arise from utilizing the pre-ranker as a replacement for fully ranking all the candidates.

Solution 4.25: What are some suitable algorithms for this model?

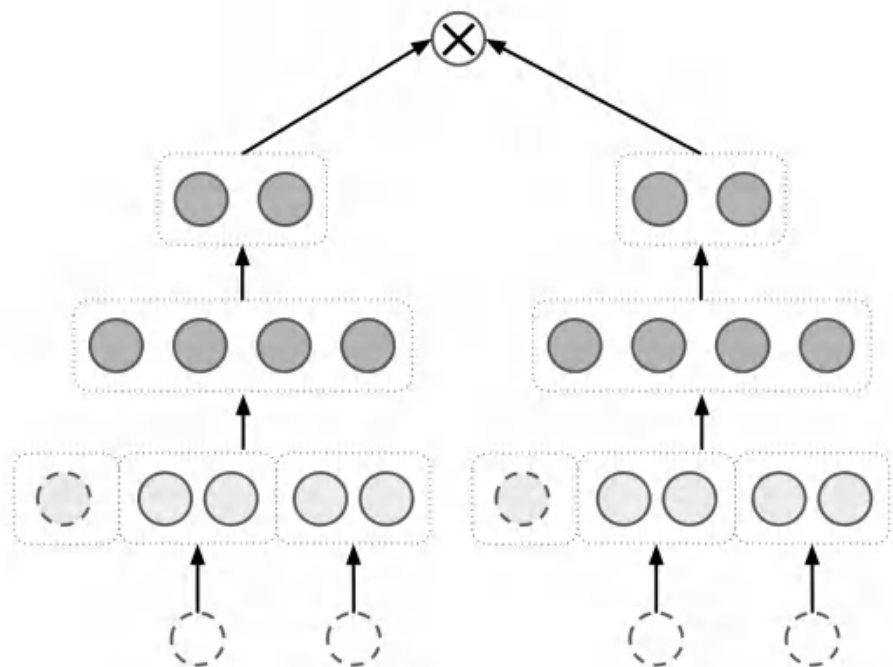
Heuristics to Logistic Regression:

Over the years, the development of pre-ranking models has undergone several iterations. Initially, non-personalized methods like **heuristics** such as averaging predicted engagements or interleaving candidate sources and algorithms were employed, but they were later replaced by **logistic regression** models that were quick to train and could be readily updated through continuous learning (McMahan et al., 2013).

Logistic Regression to Two Tower:

The logistic regression approach was dominant for several years before being eventually supplanted by **Two Tower architectures**, also known as SplitNet or vector-product. These models consist of two parallel sub neural networks,

with one network receiving target features while the other receives candidate features, and no crossing features between them. At the end of each network is a fixed-size vector representing the target and candidate embedding respectively. Efficient predictions are made at serving time by retrieving the embeddings and performing an inner product. This approach has been successfully employed in various platforms like YouTube (Covington et al., 2016), Twitter (Dilipkumar & Chen, 2019), and Alibaba (Wang et al., 2020).



Note. Illustration of the Two Tower network. Each tower takes its respective input features and converts sparse features (dashed, white) into embeddings. These embeddings are concatenated with dense features (dashed, gray) and then fed through fully connected layers. The final layer from each of two towers is combined, usually with an inner product. In some cases, a sigmoid function is then applied to convert the output to a probability. There are numerous variations, for instance Twitter uses a Hadamard product (Wang, 2020) instead of the inner product.

Two Tower to Lightweight DNNs:

Despite the success of the Two Tower approach, one major drawback was its inability to use crossing features, which are often essential for optimal performance of recommender systems and often rank at the top of feature importance analysis.

To balance between speed and accuracy, **lightweight DNNs** have emerged as a viable alternative. These models can approach the speed of Two Tower by employing various optimization techniques, such as caching and model compression. One example of these models is Alibaba's COLD model (Wang et al., 2020), which incorporates a Squeeze-and-Excitation (SE) (Hu et al., 2018) block to dynamically adjust the complexity of the model architecture.

Solution 4.26: Suppose we need to pre-rank a large number of candidates. How to optimize the model to support this?

As previously mentioned, the Two Tower model (also referred to as SplitNet or vector-product model) has become a popular approach for pre-ranking due to its ability to efficiently score candidates at serving time. The latency to produce target and candidate embeddings and perform the inner product can be measured in the single digits of milliseconds.

To further optimize Two Tower, **embeddings can be cached or precomputed**. In the caching approach, candidate embeddings can be stored in a large cache, while target embeddings can be computed early in the request, even before candidate generation. This effectively reduces the latency on the critical path to only the cache fetch and inner product computation. In the precomputing approach, embeddings are precalculated offline by enumerating over users and candidates. This approach has its downsides, such as making model updates more cumbersome due to potential embedding version mismatch issues. Additionally, the cycle time to update the embeddings could result in staleness.

To improve upon the accuracy of the Two Tower model, which is limited by its inability to utilize crossing features, DNNs may be employed. However, since traditional DNNs are generally slower than the Two Tower approach, optimizations must be applied to reduce prediction time:

- **Parallelism** can be achieved at every stage of the process. Ranking requests can be split up into several queries, and sharding is a way to distribute ranking requests while maintaining an efficient cache strategy. Another way to achieve parallelism is by fetching features simultaneously or grouping them to minimize network calls needed to hydrate them. Predictions can be executed in parallel by adjusting the batch size.

- **Optimizations can be made to the inputs.** Inputs can be densified by concatenating features. Additionally, features can be compressed by storing dense embedding representations of target and candidate features during training. Only the embeddings are required during serving and can be directly fed into the model's inner layer.. Separately, in the Alibaba COLD model, features are represented in a column-based format that is cache-friendly.
- To minimize feature hydration latency, **features can be cached at multiple levels**, including in memory and at the datacenter level, to achieve a balance between latency and hit rate. When hydrating target features, which occurs before candidate generation, **crossing features can be prefetched** in a non-blocking manner. However, since features cannot be fetched for all candidates, only the top-k candidates are hydrated, while the remaining crossing features can be backfilled after candidate generation. In a time-critical situation, they could be dropped at a cost to accuracy.
- The **model architecture can be optimized** by reducing the size and computational complexity of the model, while retaining accuracy. Popular techniques for achieving this include quantization, and model compression techniques such as pruning and low-rank factorization. Furthermore, efficient model architectures, such as the Squeeze-and-Excitation (SE) block introduced in the Alibaba COLD (Wang et al., 2020) model, can be applied.
- Finally, there are some ideas for combining the Two Tower model and concepts above. One such method is to utilize a **three-way split network** that can be trained on target, candidate, and crossing features. The embeddings for both target and candidate can be obtained using one of the standard approaches for the Two Tower model. Additionally, the crossing features can be prefetched and/or backfilled in the manner described earlier. This results in a faster inner product that does not require a prediction call to the model server.

In cases where the aforementioned techniques do not yield the desired results, it is possible to **cache pairwise model scores** or even **precompute** them all offline. However, it is important to note that these methods come with certain drawbacks. Caching scores can incur significant storage costs and may not

always achieve a satisfactory hit rate if the time-to-live (TTL) is low. On the other hand, precomputing scores offline can result in delays that may not be amenable to shifts in data distribution.

Feature Engineering

Solution 4.27: What features to use? What are their intuitions?

There are a number of different features that can be used in ranking models for recommender systems. The main feature types include:

- **Target features:** Features that describe the user.
- **Candidate features:** Features describing items that the user might be interested in.
- **Crossing features:** Features that describe the interaction between the user and items.

Numeric features are used heavily throughout industry recommender systems, commonly in the form of historic features such as a user's previous interactions. Historic features are often the most important features (He et al., 2014).

When it comes to historic features, there is a wide range of options available: these include raw counts, decayed counts, windowed counts, durations, ratios, time since events, and binary counting features like “*has done X in Y time window*”.

Taking a closer look at each of these feature types:

Target features:

Features representing the user include **demographics** such as age, gender, and language, as well as relevant **keywords** and the user's **interests** which may be represented by embeddings.

Additionally, there are **request-level features** which include where the content is displayed, such as the specific page of the user interface, the originating query (if applicable), the device, the client, the carrier, the user's network condition, geographic location, time of day, day of week, and so on.

Numeric features representing the **user's past actions**, such as how often they use the product, when they last used the product, and how much content they consume. These features can capture both historic and recent engagements. There are also numeric features that represent the user's presence on the product, such as their subscribers.

Embedding features are commonly used, such as those representing the user itself, embeddings for content comprising the user's recent engagement history, content that seeded the recommendation, embeddings of their subscribers, and so on.

Revenue (ad) models may incorporate additional features, such as the average bid depth, bid value, click-through rate, fill rate³, and network valuation provided by platforms such as Vungle.

Candidate features:

Firstly, **static attributes** of content include its id, category, features extracted from its text and media, the size, the format, and more. There are also static attributes of the author and/or advertiser, such as advertiser type and scale.

Like target features, candidate features also heavily consist of **numeric features** such as the content's recency, as well as historic and recent engagements. There are also **embedding features** representing the content, the content's category, and so on.

Similar features can also be constructed for the content's **author**, and the **advertiser** and **ad campaign**, when applicable.

Additionally, revenue (ad) models use **metadata features** like the type of ad product, the bid type, and currency. They also include features specific to the ad product: for example, for mobile app promotions there are features for app details, app installs, app category, and more.

Crossing features:

³ Fill rate is defined as the ratio of ads impressed divided by the number of total ad requests.

Crossing features play a key role in capturing the user's historical and recent interactions with the content and its author. When it comes to ads, these features can track the user's interactions with the ad itself, the ad campaign, and the advertiser. For example: How much content by this author has the user consumed? When was the last time the user consumed content by this author? How many times has the user interacted with this ad, or campaign, or advertiser?

Moreover, it is possible to cross features by means of inner products within the model architecture. Here are some common methods for crossing features in the model architecture:

- **Explicit crossing:** Primarily used in logistic regression and can also be found in Google's Wide & Deep (Cheng et al., 2016). Since explicit crossing can result in a large number of features, it's important to use domain knowledge or feature selection algorithms to limit the number of crossed features.
- **Embedding crossing:** Involves crossing the embeddings generated by input preprocessing. This technique is employed in recommender systems such as Huawei's DeepFM (Guo et al., 2017a) and Facebook's DLRM (Naumov et al., 2019).
- **Deep crossing:** The Deep & Cross Network (Wang et al., 2017) developed by Google, as well as DCN v2 (Wang et al., 2021), use a technique known as deep crossing, where each cross layer is a function of interactions between the previous cross layer and the original concatenated embedding layer.

Insider Tip

Here are some resources to learn more about the features commonly used in industry recommender systems:

- Ranking Tweets with TensorFlow (Twitter) (Zhuang et al., 2019)
- Multi-Task Learning of Graph-based Inductive Representations of Music Content (Spotify) (Saravanou et al., 2021)
- Leaving No One Behind: A Multi-Scenario Multi-Task Meta Learning Approach for Advertiser Modeling (Alibaba) (Zhang et al., 2022)

- Real-time Short Video Recommendation on Mobile Devices (Kuaishou) (Gong et al., 2022)
- Deep Neural Networks for YouTube Recommendations (Covington et al., 2016)

Solution 4.28: How to handle textual or id-based features?

In deep neural networks (DNNs), there are various methods for preprocessing textual and id-based features:

- If the input has low cardinality it's possible to use a **one-hot encoding**, also referred to as the vocabulary (for textual) or identity (for id-based). As a rule of thumb, low cardinality may be seen as having a number of unique values less than 20% of the dataset size or less than the square root of the dataset size (*Training Using the Built-In Wide and Deep Algorithm / AI Platform Training*, 2023).
- For inputs with higher cardinality, the **hashing trick** can be applied to reduce the dimensionality of the input vector. The hashing trick involves applying a hash function to the features and using the resulting hash values as indices. Besides consuming less memory, the hashing trick facilitates model updates on unseen values without altering the input vector's definition or shape, but may result in collisions.
- **Embeddings** are used to represent categorical features as continuous-valued vectors in a lower-dimensional space. Embeddings are particularly useful when capturing the relationships between input values is important (such as in recommender systems!). However, embeddings may overgeneralize, especially for id-based features that are sparse and high rank.

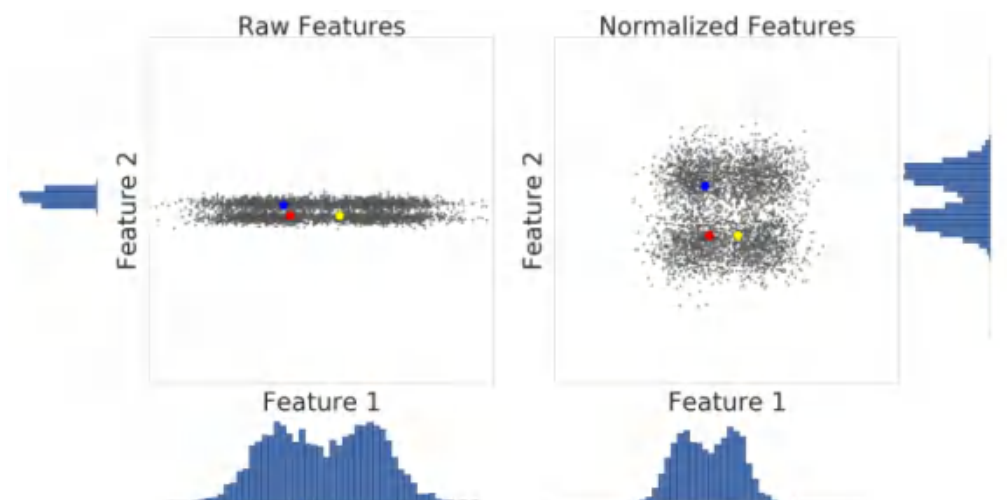
Embeddings are widely used in practice, with both Google's Wide & Deep and Facebook's DLRM recommender system utilizing them for every categorical feature. A common practice is to use one embedding per input feature, with the embedding dimension typically set between 5 and 100, and the ceiling set to the square root of the number of unique values in the input.

Solution 4.29: What about counting features? Problems with using counting features?

Because counting features can get quite large, it's important to standardize them in some way. Failure to do so can lead to several problems during the training process, such as (1) the loss may fluctuate due to significant differences in the input feature ranges, resulting in slow or unstable learning and (2) the model may become vulnerable to noise in the data, resulting in poor performance on new data.

There are several ways to preprocess numeric features in DNNs:

- **Log transformation:** Involves calculating either the natural logarithm or the logarithm of one plus the feature value ($\log(1+x)$). This approach is efficient to compute and is particularly suitable for features that adhere to a power law distribution. However, it does not handle negative values.
- **Z-score scaling:** Converts each value to a standard score, by subtracting from the mean and dividing by the standard deviation. Has the benefit of zero-centricity for normally distributed features, and is a good choice if outliers are not extreme. However, it can be computationally expensive to compute on larger datasets, necessitating sampling.
- **Bucketing:** Also known as binning or discretizing, involves the partitioning of each numeric feature into discrete bins via one of several methods, such as the use of quantile boundaries or the application of minimum description length (MDL) (Fayyad & Irani, 1993), which leverages information gain to identify optimal splits. Bucketing is a valuable technique for datasets characterized by skewed feature distributions. However, note that this approach results in a loss of information.
- **Clipping:** Constrains the range of a feature by bounding all values above and below designated thresholds. This strategy is particularly useful in scenarios where outliers are extreme. Clipping is often employed in conjunction with other preprocessing techniques.



Note. After applying z-score scaling (normalization), features on the x- and y-axes have the same scale. Source: (*Prepare Data / Machine Learning*, 2022).

Performing multiple preprocessing techniques is also a viable option. It is possible to combine techniques, or generate multiple versions of a feature, such as creating both bucketed and z-score scaled versions.

All three techniques are used in practice. For instance, Google's Wide & Deep uses z-score scaling, Huawei's DeepFM uses discretization, and Google's Deep & Cross Network uses log transformation. It is worth noting that for these models, the same preprocessing technique is applied across all numeric features.

Another approach to preprocess numeric features involves applying a multilayer perceptron (MLP) directly to the features and utilizing the output of the hidden layer as the input for the remaining network layers. While this approach is not widely adopted in industry, it is utilized by Facebook's DLRM recommender.

Insider Tip

Test your understanding of various numeric preprocessing techniques with this quiz (*Transforming Your Data: Check Your Understanding / Machine Learning*, 2022) in Google's course on data prep.